

算法的低语

数 学 卷

从公式到推导

本册是《算法的低语》主本的姊妹册。镜像主本上下两半场的七章结构，对每个 AI 概念背后的数学给出两层呈现：

- 直觉 (150-250 字)：回顾概念由来与几何含义
- 严格 (400-800 字 + 公式块)：定义、推导步骤、矩阵维度

面向已具备线性代数、微积分、概率统计本科基础的进阶读者。

建议与主本《算法的低语》配套阅读；读完主本某章后再翻附册对应章节。

大队长 出品

fqsx@mail.ustc.edu.cn

Version 1.5 · 每节增加「这一节在讲什么」导读 + 扩充直觉版

目录

前言：为什么需要这本附录	4
数学卷·第 1 章：信息论与语言的概率结构	5
§1.1 香农的赌局：语言是对抗熵的工具	5
§1.2 从马尔可夫链到 N-Gram：第一代尝试的局限	7
§1.3 跨界映射：语言序列与 DNA 碱基对	9
§1.4 反类比：当 DNA 的隐喻失效	11
数学卷·第 2 章：嵌入空间与高维几何	12
§2.1 词嵌入：为什么字典救不了机器	12
§2.2 语义的坐标系：那个被讲烂了的公式	14
§2.3 高维几何的诡异世界	16
§2.4 跨界映射：从罗马路网到曼哈顿距离	18
§2.5 反类比：当几何隐喻失效	19
§2.6 跨越欧氏：当表示空间不再「平直」	21
数学卷·第 3 章：注意力机制的矩阵微分	23
§3.1 从 RNN 到变换器 (Transformer)：序列建模的范式革命	23
§3.2 注意力公式的拆解：QKV 的真实含义	25
§3.3 矩阵乘法的大规模并发：为什么 AI 是 GPU 的盛宴	27
§3.4 RoPE：把位置编码从加法变成旋转	29
§3.5 FlashAttention：当注意力撞上内存墙	31
§3.6 注意力的真实身份：关联记忆，而非「注意力」	33
§3.7 一个被忽略的争论：注意力权重 解释	35
§3.8 跨界映射：宏观工业供应链	36
§3.9 反类比：当供应链隐喻失效	37
数学卷·第 4 章：非凸优化与超参数标度律	38
§4.1 损失函数：如何用数学定义“机器犯错”	38
§4.2 梯度下降：在黑暗中沿着最陡的方向下山	40
§4.3 高维非凸优化的反直觉：为什么 SGD 居然能工作	42
§4.4 损失景观的可视化：黑暗中的地形图	44
§4.5 P：超参数迁移的“元定律”	45
§4.6 Lion 与 Shampoo：Adam 不是终点	47
§4.7 P 的完整面貌：超参数迁移背后的数学	48
§4.8 跨界类比：复杂系统的工程管理	50
§4.9 反类比：工程管理隐喻的失效边界	51
数学卷·第 5 章：标度律、相变与涌现的数理基础	52
§5.1 尺度律 (Scaling Law)：算力即权力的冷酷物理感	52
§5.2 Chinchilla 的颠覆与再颠覆	54
§5.3 数据墙与 §5.4 推理时计算	56
§5.5 涌现：神话还是幻觉？	58
§5.6–§5.7 Grokking 的数学解剖：从死记到顿悟	60

§5.8 渗流相变：把 Scaling Law 焊到统计物理上	62
§5.9 跨界映射与 §5.10 反类比：相变叙事的边界	64

数学卷·第 6 章：硬件感知算法的数学——屋脊线 (Roofline)、I/O 复杂度与 FlashAttention

推导	65
§6.1 Roofline 模型的形式化	66
6.1.1 算术强度的精确定义	66
6.1.2 性能上界与 Roofline 公式	67
6.1.3 脊点 (Ridge Point) 的推导	67
6.1.4 GEMM 的算术强度分析	67
§6.2 I/O 复杂度的下界 (Hong-Kung 1981)	68
6.2.1 计算模型与红蓝卵石游戏	69
6.2.2 关键引理：S-划分与路径数	69
6.2.3 矩阵乘的 I/O 下界	70
§6.3 FlashAttention 的 I/O 分析 (Dao et al. 2022)	71
6.3.1 标准注意力 (Attention) 的 I/O 复杂度	71
6.3.2 在线归一化指数 (Softmax) 的递推公式	72
6.3.3 FlashAttention 的 I/O 复杂度	73
6.3.4 FlashAttention vs. 标准 Attention 的复杂度比较	74
6.3.5 FlashAttention-2 的工作划分 (Work Partitioning) 改进	74
§6.4 环形注意力 (RingAttention) 的通信复杂度 (Liu et al. 2024)	75
6.4.1 多卡场景设定	75
6.4.2 环形通信方案	76
6.4.3 计算与通信重叠条件	76
6.4.4 环路最大规模推导	77
§6.5 算术强度的演化：从 V100 到 B200	77
6.5.1 各代 GPU 的 Roofline 参数	78
6.5.2 Ridge Point 的单调上行论证	78
§6.6 反类比与边界	80
6.6.1 Roofline 模型的假设局限	80
6.6.2 Hong-Kung 下界的适用范围	81
6.6.3 FlashAttention I/O 分析的隐含前提	81
6.6.4 RingAttention 的通信模型假设	81
§6.7 参考文献	82

数学卷·第 7 章：后变换器 (Transformer) 时代的数学——状态空间模型、线性注意力与扩散语言模型的数理基础

§7.1 状态空间模型的连续形式	83
7.1.0 工程直觉引子：从「滑动窗口 + 缓存状态」走到连续 ODE	84
7.1.1 线性时不变系统的基本方程	85
7.1.2 闭式解的推导	85
7.1.3 卷积视角	85
7.1.4 频域分析与传递函数	86
§7.2 HiPPO 框架的严格推导	86
7.2.1 问题陈述：在线最优投影	87

7.2.2 Legendre 多项式的正交性	87
7.2.3 HiPPO-LegT: 平移 Legendre 测度	88
7.2.4 引理 (HiPPO-LegS 矩阵的推导)	88
7.2.5 最优投影的精确陈述	89
§7.3 S4 的离散化与卷积核	90
7.3.1 零阶保持离散化	90
7.3.2 离散卷积核	91
7.3.3 计算复杂度的降低:Cauchy 核与范德蒙 (Vandermonde) 矩阵	91
§7.4 Mamba 的选择性扫描	93
7.4.1 选择性机制: 从 LTI 到输入依赖	93
7.4.2 选择性 SSM 的计算挑战	94
7.4.3 并行扫描与关联算子	94
7.4.4 Blleloch 并行前缀扫描的复杂度	95
§7.5 线性注意力的核函数视角	95
7.5.1 标准归一化指数 (Softmax) 注意力的复杂度	96
7.5.2 核函数分解	96
7.5.3 特征映射的选择	97
7.5.4 线性注意力与 Fast Weight Memory 的等价性	97
§7.6 Mamba、线性注意力 (Attention) 与 RNN 的统一视角	98
7.6.1 通用递推形式	98
7.6.2 Mamba-2 的结构化状态空间对偶性	99
7.6.3 GLA 的数据依赖门控	99
§7.7 扩散语言模型的数学	100
7.7.1 连续扩散:DDPM 回顾	100
7.7.2 离散扩散: 吸收态过渡矩阵	101
7.7.3 MDLM 的简化目标	101
7.7.4 与自回归模型的本质差异	102
§7.8 世界模型与预测编码的数学	102
7.8.1 预测编码: 层级生成模型与误差传递	103
7.8.2 自由能原理与变分下界	104
7.8.3 JEPA 的能量函数与防 collapse 机制	104
7.8.4 Kalman 滤波作为线性高斯 SSM 的精确推断, 以及神经过程视角	105
§7.9 反类比与边界	106
7.9.1 HiPPO” 最优性” 的测度依赖	107
7.9.2 Mamba 并行扫描的数值稳定性	107
7.9.3 线性注意力在检索任务上的系统性劣势	107
§7.10 参考文献	108
后记	110

前言：为什么需要这本附册

《算法的低语》主本用 7 章篇幅勾勒了大语言模型背后的数学骨架与产业现实——上半场从信息论的香农熵、嵌入空间的高维几何、Transformer 的矩阵微分、非凸优化的鞍点逃逸，讲到涌现现象的渗流相变隐喻；下半场则把镜头拉远，看这套数学如何级联出万亿美元的算力供应链（第六章）、以及 Transformer 之后下一代架构正在从哪里凿开缺口（第七章）。主本的目标是把这条逻辑链讲清楚、讲完整；而本附册的目标是把每一处「为什么是这个公式」「这个公式怎么推出来」补齐。

本册 7 章与主本逐章镜像：前 5 章是数学引擎本身的严格推导；第 6 章《硬件感知算法的数学》覆盖 Roofline 模型、Hong-Kung I/O 下界、FlashAttention 在线 softmax 推导、RingAttention 通信复杂度；第 7 章《后 Transformer 时代的数学》覆盖 HiPPO LegS 矩阵推导、S4 零阶保持离散化、Mamba 选择性扫描的并行关联律、线性注意力的核函数分解、扩散语言模型与 AR 的语义差别。

本册不重复主本的叙事。每个小节默认读者已经读完主本对应章节，因此直接进入数学：

- **直觉版**：用一两段话回顾概念由来与几何含义，铺出数学动机；不含公式。
- **严格版**：给出严格定义、关键公式、必要的推导步骤；所有矩阵显式标注维度，符号尽量与原始论文一致。

读者无需逐节深入。建议在读完主本某一章后，对感兴趣的小节回看附册——把直觉补齐、把公式补全、然后回到主本继续读下一章。

公式约定：行内公式用 $\dots$ ，块公式用

...

，矩阵以 $\mathbb{R}^{m \times n}$ 形式标注维度。底数缺省时 $\log$ 按自然对数处理；信息论场景下若涉及 bit 单位会显式注明 $\log_2$ 。

数学卷·第1章：信息论与语言的概率结构

§1.1 香农的赌局：语言是对抗熵的工具

这一节在讲什么

为什么要给「不确定性」一个数学定义？1948 年之前，工程师只能用「信道质量好」「信号清晰」这种模糊的描述。香农提出：把不确定性当作一个可加、可比、可优化的量，整个通信工程就变成了数学。

第一步：把概率转成「惊讶度」。一件事概率越小，发生时越让人惊讶，信息量越大。所以信息量是 $-\log p(x)$ 。

第二步：取平均。所有可能结果的惊讶度加权平均，就是熵 $H(X) = -\sum p(x) \log p(x)$ 。它告诉你：要编码这个分布的输出，平均最少用多少比特。

第三步：当模型的分布 q 偏离真分布 p 时，要付额外代价。这就是 KL 散度和交叉熵——大模型训练用的损失，本质上就是这个代价。

本节要拿走的一件事：交叉熵不是「随便挑的损失」，它是「让模型分布逼近数据分布」的唯一信息论意义上正确的目标。

直觉

原书以香农的“猜字母实验”引入，指出英语文本的熵率约为 1 bit/字符，远低于 26 个字母均匀分布下的理论上限。这里的关键问题是：用什么数学量度量“不确定性”，以及当一个概率分布与另一个分布不一致时，如何量化其代价？

香农熵 (Shannon entropy) 给出了不确定性的绝对标尺——它的形式 $-\sum p(x) \log p(x)$ 并不是猜出来的，而是由「可加性」「连续性」「单调性」三条公理唯一确定的（香农 1948 年的原始证明）。条件熵刻画了“已知部分信息后剩余的不确定性”——这恰好对应语言模型的预测任务：给定前 k 个 token，下一个 token 还剩多少不确定性。互信息衡量两个变量之间共享了多少信息，是后来 InfoNCE、对比学习的数学源头。

交叉熵和 KL 散度则是模型分布偏离真实分布时付出的“额外代价”。这里有一个关键事实：交叉熵 $H(p, q) = H(p) + D_{\text{KL}}(p \| q)$ ，其中 $H(p)$ 是数据本身的熵，是模型动不了的常数；因此**最小化交叉熵等价于最小化 KL 散度，等价于最大化对数似然**。这三个目标在数学上完全等价，只是从不同角度看同一件事。大模型的训练目标函数——交叉熵损失——正是这条链条的终点。困惑度则是把损失指数化，回到「等效面临几个均匀选项」的直观量纲。

严格

1. 香农熵 (Shannon entropy)

设离散随机变量 X 取值于有限集合 $\mathcal{X}$ ，概率质量函数为 $p(x) = P(X = x)$ 。**香农熵**定义为：

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log p(x)$$

其中约定 $0 \log 0 = 0$ （取极限值）。

对数底数约定：当底数为 2 时，单位为比特 (bit)；当底数为 e 时，单位为奈特 (nat)；当底数为 10

时，单位为哈特 (hart)。大模型训练中几乎统一使用自然对数 (底数 e)，以便与梯度计算衔接；信息论教材常用 $\log_2$ 。本节公式默认底数为 e ，在需要与原书对应的地方显式标注。

熵的基本性质：

- $H(X) \geq 0$ ，等号成立当且仅当 X 为退化分布 (某一值概率为 1)；
- 均匀分布时熵最大， $H(X) \leq \log |\mathcal{X}|$ ；
- $H(X)$ 只依赖于分布 p ，而不依赖于具体取值。

2. 条件熵 (conditional entropy)

设 (X, Y) 的联合分布为 $p(x, y)$ ，**条件熵** 定义为：

$$H(Y | X) = - \sum_{x \in \mathcal{X}} p(x) \sum_{y \in \mathcal{Y}} p(y | x) \log p(y | x)$$

等价地， $H(Y | X) = H(X, Y) - H(X)$ ，即联合不确定性减去 X 本身的不确定性。链式法则由此推广为：

$$H(X_1, X_2, \dots, X_n) = \sum_{i=1}^n H(X_i | X_1, \dots, X_{i-1})$$

3. 互信息 (mutual information)

互信息 衡量 X 与 Y 共享的信息量：

$$I(X; Y) = H(X) - H(X | Y) = H(Y) - H(Y | X) = H(X) + H(Y) - H(X, Y)$$

等价定义为联合分布与边缘分布之积的 KL 散度 (见下文)：

$$I(X; Y) = \sum_{x, y} p(x, y) \log \frac{p(x, y)}{p(x) p(y)}$$

$I(X; Y) \geq 0$ ，等号成立当且仅当 X 与 Y 相互独立。

4. KL 散度 (Kullback-Leibler divergence)

设 p 为真实分布， q 为近似分布，二者定义在同一有限集合上且 $q(x) > 0$ 只要 $p(x) > 0$ 。**KL 散度** (又称相对熵) 定义为：

$$D_{\text{KL}}(p \| q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

性质： $D_{\text{KL}}(p \| q) \geq 0$ (由 Jensen 不等式和 $\log$ 的凹性得到)，等号成立当且仅当 $p = q$ ；一般情况下 $D_{\text{KL}}(p \| q) \neq D_{\text{KL}}(q \| p)$ ，即 KL 散度不是度量 (metric)。

5. 交叉熵 (cross-entropy)

交叉熵 定义为：

$$H(p, q) = - \sum_x p(x) \log q(x)$$

与 KL 散度的关系：

$$H(p, q) = H(p) + D_{\text{KL}}(p \| q)$$

因为真实数据分布 p 的熵 $H(p)$ 是常数（不依赖于模型参数），最小化交叉熵等价于最小化 KL 散度，等价于让模型分布 q_θ 最大化真实数据的对数似然。大模型的训练损失为：

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log q_\theta(w_i | w_{<i})$$

其中 $w_{<i} = (w_1, \dots, w_{i-1})$ 为前缀， N 为序列长度， q_θ 为模型输出的条件概率。

6. 困惑度 (perplexity)

困惑度是评估语言模型的标准指标，定义为交叉熵的指数：

$$\text{PPL} = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log q_\theta(w_i | w_{<i})\right) = \exp(H(p, q))$$

直观解释：困惑度等于模型在每一步预测时“等效面临的均匀选择数”。若 $\text{PPL} = k$ ，模型的表现等价于从 k 个等可能选项中均匀猜测。困惑度越低，模型越确定。注意困惑度的数值大小与 $\log$ 底数直接绑定——若改用 $\log_2$ ，则 $\text{PPL} = 2^{H_2(p, q)}$ ，其中 H_2 表示以 2 为底的交叉熵（单位为 bit）。

与原书呼应：原书 §1.1 指出大模型的训练目标是 minimize 交叉熵损失 $\mathcal{L} = -\frac{1}{N} \sum_i \log P_\theta(w_i | w_{<i})$ ，对应本节交叉熵与 KL 散度的定义，以及困惑度作为其指数化形式。

§1.2 从马尔可夫链到 N-Gram：第一代尝试的局限

这一节在讲什么

为什么 N-Gram 会失败？ 1990 年代之前，几乎所有语言模型都是 N-Gram。它的失败不是「调参不够」，而是数学上注定的。

第一步：N-Gram 的核心是一个粗暴的独立性假设——「下一个词只依赖于前 k 个词」。这等价于说语言是 k 阶马尔可夫链。

第二步：参数空间随 k 指数爆炸。 词表 $V = 50000$ 、 $k = 4$ 时，参数有 $50000^5 \approx 3.1 \times 10^{23}$ 个——比可观测语料的 token 数大得多个数量级。绝大多数计数是零。

第三步：平滑技术也只能续命，不能解围。 Kneser-Ney 是 N-Gram 时代最好的平滑，但当 k 想要捕获长距离依赖时（比如代词指代、章节伏笔），它仍然无能为力。

本节要拿走的一件事：N-Gram 失败的数学根源是 ** 指数爆炸 + 没有参数共享 **。后来的词嵌入、Transformer 之所以能成功，关键在于 ** 用低维稠密向量代替高维稀疏计数 **，让参数能在相似上下文之间共享。

直觉

原书从马尔可夫 1913 年对俄语字母的手工统计出发，介绍了 N-Gram 模型的思路与两个致命缺陷：组合爆炸和无法建模长距离依赖。这两个缺陷不是工程上的小问题，而是**模型形式本身在数学上不可**

避免的代价。

N-Gram 的数学核心是一个严格的条件独立假设：下一个词只依赖于前 k 个词。这在数学上对应 k 阶马尔可夫链。听上去合理——「人在说话时大概也只得记得前面几个词」——但这个假设忽略了语言的本质特征：长距离依赖（代词指代远在 50 个词之前的名词）、嵌套结构（从句中包含从句）、跨段落的语义关联。任何固定的 k 都注定要么太短（丢失长依赖）要么太长（参数爆炸）。

它的参数通过最大似然估计（频率计数）获得，但稀疏性问题使得大量条件概率为零，必须依靠平滑技术修补。**Add-one 平滑**简单粗暴但偏差大；**Kneser-Ney 平滑**引入“延续计数”的精妙思路，是 N-Gram 时代效果最好的方法。但即使最好的 N-Gram + 最好的平滑，在长距离建模上仍然完败于神经网络方法——因为问题不在估计技巧，而在模型形式本身。

严格

1. k 阶马尔可夫性 (k -th order Markov property)

设词序列 $(w_1, w_2, \dots, w_T)$ ，每个 w_t 取值于词表 $\mathcal{V}$ ($|\mathcal{V}| = V$)。 k 阶马尔可夫假设为：

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = P(w_t | w_{t-k}, w_{t-k+1}, \dots, w_{t-1})$$

即给定最近 k 个词后， w_t 与更早的历史条件独立。

N-Gram 的对应关系： k 阶马尔可夫链等价于 $(k+1)$ -gram 模型。

k (马尔可夫阶)	N-Gram 名称	条件窗口大小
0	unigram (1-gram)	无条件 (词独立)
1	bigram (2-gram)	前 1 个词
2	trigram (3-gram)	前 2 个词
k	$(k+1)$ -gram	前 k 个词

整条序列的概率在 k 阶马尔可夫假设下分解为：

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-k}, \dots, w_{t-1})$$

(对 $t \leq k$ 的情形，以序列开头作为边界条件处理。)

2. 最大似然估计 (maximum likelihood estimation, MLE)

给定训练语料， $(k+1)$ -gram 模型的 MLE 参数为计数之比：

$$\hat{P}(w_n | w_{n-k}^{n-1}) = \frac{\text{count}(w_{n-k}, \dots, w_{n-1}, w_n)}{\text{count}(w_{n-k}, \dots, w_{n-1})}$$

其中 w_{n-k}^{n-1} 是从 w_{n-k} 到 w_{n-1} 的子序列， $\text{count}(\cdot)$ 表示该 n -gram 在训练语料中出现的次数。这一估计是无偏的，且在语料趋于无穷时收敛至真实分布。

3. 稀疏性与零概率问题

MLE 的根本缺陷：若某 n -gram 在训练语料中从未出现，则 $\hat{P} = 0$ ，导致任何包含该 n -gram 的句子概率为零。词表大小 V 时， k 阶 N-Gram 的参数空间为 $O(V^{k+1})$ 。当 $V = 50000$ ， $k = 4$ 时，参数量级为 $V^5 \approx 3.1 \times 10^{23}$ ，远超可观测语料的大小，绝大多数组合计数为零。

4. 平滑 (smoothing) 技术直觉

Add-one (Laplace) 平滑: 对每个可能的 n -gram 计数加 1, 等价于在参数的先验 (均匀 Dirichlet) 下取后验均值:

$$\hat{P}_{\text{add-1}}(w_n | w_{n-k}^{n-1}) = \frac{\text{count}(w_{n-k}^{n-1}, w_n) + 1}{\text{count}(w_{n-k}^{n-1}) + V}$$

这保证了所有概率非零, 但对低频 n -gram 的估计偏差较大, 因为它对稀有和常见组合施加了同等的绝对调整量。

Kneser-Ney 平滑的核心直觉 (不做完整推导): 观察到”出现在多样上下文中的词”在未见 n -gram 中应有更高的延续概率。具体地, 用**延续计数** (continuation count) 代替频率计数:

$$P_{\text{KN}}(w | \text{context}) \propto \max(\text{count}(\text{context}, w) - d, 0) + \lambda(\text{context}) P_{\text{KN}}(w)$$

其中 $d \in (0, 1)$ 为折扣参数, $\lambda(\text{context})$ 为归一化常数, $P_{\text{KN}}(w)$ 为低阶 (回退) 分布, 定义为 w 出现在其左侧上下文种类数与总上下文种类数之比。Kneser-Ney 是 N-Gram 时代效果最好的平滑方法, 至今仍是基线评估的参照点。

与原书呼应: 原书 §1.2 指出 k 阶马尔可夫假设是 N-Gram 的数学本质, 并以词表大小 $V = 50000$ 、5-gram 参数量 $V^5 \approx 3.1 \times 10^{23}$ 说明组合爆炸, 对应本节 MLE 定义和稀疏性分析。

§1.3 跨界映射: 语言序列与 DNA 碱基对

这一节在讲什么

为什么要谈「熵率」? 前两节的熵 $H(X)$ 是针对单个随机变量定义的, 但语言是一长串符号。我们需要一个量来刻画「一个无穷长的序列每符号平均携带多少信息」。

第一步: 把序列的联合熵除以长度。 $\mathcal{H} = \lim_{n \rightarrow \infty} \frac{1}{n} H(X_1, \dots, X_n)$ 。如果序列是平稳的 (统计性质不随时间变), 这个极限就存在。

第二步: Shannon-McMillan-Breiman 定理 (AEP) 给出了一个惊人事实——对几乎所有「典型」的长序列, 每个序列的概率都约等于 $2^{-n\mathcal{H}}$ 。也就是说, 长度 n 的序列虽然字面上有 M^n 种, 但 ** 实际上只有 $2^{n\mathcal{H}}$ 个序列扛走了几乎全部的概率质量 **。

第三步: 这对语言模型意味着什么? 训练语料的「有效信息容量」不是字母表大小的幂次, 而是由熵率决定。模型只要覆盖典型序列集, 就能捕获几乎全部概率。

本节要拿走的一件事: 熵率把信息论从「单个变量」推广到「无穷序列」——这是把语言、DNA、时间序列都纳入同一个数学框架的关键一步。

直觉

原书将语言和 DNA 视为”有限符号集 + 组合规则 + 信息传递”的共同框架, 两者都可以用信息论分析。这一节的数学内核是**序列的熵率**: 当序列长度趋于无穷时, 每个符号平均携带的信息量。Shannon-McMillan-Breiman 定理 (典型序列定理) 说明, 对平稳遍历过程, 长序列的概率几乎必然由熵率决定。

为什么这个跨界类比不是修辞而是数学事实？因为信息论的基础概念——熵、互信息、信道容量——都是与符号的物理意义无关的纯统计量。无论符号是英文字母、汉字、DNA 碱基、还是音符，只要它们构成一个平稳遍历过程，熵率定理就同样成立。这就是为什么同一套数学工具能同时分析语言、生物序列、金融时间序列。

熵率的实际数值给出了重要的工程参考：香农估计英语文本的熵率约为 1 bit/字符，DNA 编码区接近其字母表上限 2 bit/碱基。这意味着英语文本的信息密度比 DNA 低（每个符号承载的信息少），但语言用的字母表（汉字/词）比 DNA 大得多，单位时间能传递的总信息量并不低。这种“低密度 + 大字母表”的权衡是语言演化的痕迹——它牺牲了信息效率，换取了抗噪能力和演化灵活性。

严格

1. 信息源与符号序列

设符号序列 $(X_1, X_2, \dots)$ 为定义在有限字母表 $\mathcal{A}$ ($|\mathcal{A}| = M$) 上的平稳随机过程 (stochastic process)。平稳性 (stationarity) 要求联合分布在时间平移下不变：

$$P(X_1 = a_1, \dots, X_n = a_n) = P(X_{1+k} = a_1, \dots, X_{n+k} = a_n), \quad \forall k \geq 0$$

2. 熵率 (entropy rate)

序列的熵率 (entropy rate) 定义为：

$$\mathcal{H} = \lim_{n \rightarrow \infty} \frac{1}{n} H(X_1, X_2, \dots, X_n)$$

对平稳过程，该极限存在。等价地（由链式法则），熵率也等于条件熵的极限：

$$\mathcal{H} = \lim_{n \rightarrow \infty} H(X_n | X_1, \dots, X_{n-1})$$

对语言序列而言， $\mathcal{H}$ 刻画了“在已知所有历史的条件下，下一个 token 还剩多少不确定性”。香农估计英语文本的熵率约为 1 bit/字符 ($\log_2$ 底数)，这正是他猜字母实验的结论。

3. Shannon-McMillan-Breiman (SMB) 定理

对有限字母表上的平稳遍历 (stationary ergodic) 过程，SMB 定理（亦称渐近等分性，Asymptotic Equipartition Property, AEP）成立：

$$-\frac{1}{n} \log P(X_1, X_2, \dots, X_n) \xrightarrow{a.s.} \mathcal{H}, \quad n \rightarrow \infty$$

即序列的经验对数概率以概率 1 收敛至熵率 $\mathcal{H}$ 。其核心推论是：对充分大的 n ，长度为 n 的序列几乎全部集中在约 $2^{n\mathcal{H}}$ ($\log_2$ 底数) 个“典型序列”上，每个典型序列的概率约为 $2^{-n\mathcal{H}}$ 。非典型序列的概率之和趋于零。

对语言模型的意义：SMB 定理说明，一个足够好的模型只需要“覆盖”典型序列集合，就可以捕获几乎全部的概率质量。训练语料的有效信息容量由熵率决定，而不是由字母表大小的幂次决定。

与原书呼应：原书 §1.3 指出语言和 DNA 在“有限符号集 + 组合规则”的信息论框架下具有结构相似性，对应本节序列熵率的定义和 SMB 定理对典型序列的描述。

§1.4 反类比：当 DNA 的隐喻失效

这一节在讲什么

为什么 DNA 的隐喻不能滥用？前一节展示了语言和 DNA 在信息论框架下的共性。这一节给出三堵铁墙：**字母表量级差异、稀疏性差异、冗余的物理本质差异**。

第一步：字母表大小决定熵率上限。DNA 只有 4 个符号，上限 $\log 4 \approx 2$ bit；大模型词表有 5 万 token，上限 $\log 50000 \approx 15.6$ bit——理论信息密度差近 8 倍。

第二步：稀疏性的量级完全不同。DNA 三联体只有 $4^3 = 64$ 种组合，可以穷举统计；语言 5-gram 有 $50000^5 \approx 3 \times 10^{23}$ 种，远超任何可能的语料规模。

第三步：冗余的来源不同。DNA 的简并性是**物理化学约束**（tRNA 摆动配对），是写死的；语言的冗余是**使用者群体统计行为的涌现**，会随时间漂移（流行语、网络用语、新词）。

本节要拿走的一件事：「语言像 DNA」是一个**比喻**，不是定理。它能启发跨学科思考，但当你真的把 DNA 序列建模技术搬到 NLP 时，必须重新面对：更大的字母表、更稀疏的统计、更深的长程依赖——这才是 Transformer 崛起的数学土壤。

直觉

原书在承认语言与 DNA 的结构相似后，立即指出三堵“铁墙”。从数学角度，最直接的一堵墙是量级差异：DNA 的字母表只有 4 个符号，而自然语言的词表有数十万个 token。这一差异不是细节，而是导致熵率、稀疏性、建模难度在量级上截然不同。

更深一层的差异在于**冗余的物理本质**。DNA 的简并性（多个密码子对应同一氨基酸）是写在 tRNA 物理结构里的硬性事实，不会随物种、时代变化。而语言的冗余完全是统计现象——它来源于语法约束、词类共现、上下文语义限制，这些规则没有任何物理化学的必然性，是使用者群体行为的涌现产物。这导致一个深刻的差异：**DNA 的统计性质是定的，语言的统计性质在演化**。一份 1900 年的英文语料训出来的 N-Gram 在 2025 年的网络语言上几乎不可用，但 1900 年小鼠的基因组和 2025 年小鼠的基因组在统计性质上几乎一样。

这一量级差异说明，把 DNA 的序列建模技术直接迁移到语言，不仅需要应对更大的字母表，还需要捕获更深层、更长程的统计依赖——这正是 N-Gram 失败、变换器（Transformer）崛起的数学根源。

严格

1. 字母表大小与熵率上限

对大小为 M 的均匀字母表，熵率上限为 $\log M$ （自然对数，单位 nat）：

$$\mathcal{H} \leq \log M$$

- DNA： $M = 4$ ，熵率上限 $\log 4 \approx 1.39$ nat/符号（约 2 bit/符号）；
- 大模型的 subword 词表（如 GPT-4 的 BPE 词表）： $M \approx 50000$ ，熵率上限 $\log 50000 \approx 10.82$ nat/token（约 15.6 bit/token）。

两者的熵率上限之比约为 $\log 50000 / \log 4 \approx 7.8$ ，即语言 token 级别的理论信息密度是 DNA 碱基的近 8 倍。

2. 稀疏性的量级差异

对 k 阶 N-Gram，参数空间大小为 M^{k+1} 。

- DNA, $k = 2$ (三联体密码子): $4^3 = 64$ 个组合, 全部可在有限实验数据中覆盖。
- 语言, $k = 4$ (5-gram): $50000^5 \approx 3.1 \times 10^{23}$ 个组合, 任何有限语料都无法覆盖其中绝大多数。

因此, DNA 序列在三联体层级上不存在稀疏性问题, 可以对密码表做穷举统计; 而语言序列在 $k \geq 3$ 时稀疏性已经不可避免, 必须依赖平滑或参数共享 (如词嵌入)。

3. 遗传码的信息冗余与语言冗余的本质差异

遗传密码存在**简并性** (degeneracy): 64 个密码子仅编码 20 种氨基酸 (加 3 个终止密码子), 多个密码子映射到同一氨基酸。这是物理化学约束 (tRNA 反密码子的摆动配对) 决定的, 与统计规律无关。

语言的冗余则来自两个来源: 句法约束 (词类共现规则) 和语义约束 (上下文限制词义)。这两类约束没有物理化学的必然性, 它们是使用者群体统计行为的涌现, 因此可以随时间和社群漂移。数学上, 语言的冗余度体现为熵率 $\mathcal{H}$ 远低于 $\log M$: 香农估计英语的实际熵率约为 1 bit/字符, 而字母表 (26 字母) 的上限为 $\log_2 26 \approx 4.7$ bit/字符, 冗余度约为 $1 - 1/4.7 \approx 79\%$ 。DNA 的情况则不同: 以碱基为单位, 其熵率接近理论上限 (基因组中存在大量非编码区, 编码区本身因密码子偏好性略低于上限), 冗余度低得多。

这一量级差异说明, 把 DNA 的序列建模技术直接迁移到语言, 不仅需要应对更大的字母表, 还需要捕获更深层、更长程的统计依赖——这正是 N-Gram 失败、变换器 (Transformer) 崛起的数学根源。

与原书呼应: 原书 §1.4 指出 DNA 字母表 (4 符号) 与语言词表 (数万 token) 的差异是类比失效的起点之一, 对应本节熵率上限的量级比较和稀疏性的参数空间计算。

数学卷·第 2 章: 嵌入空间与高维几何

§2.1 词嵌入: 为什么字典救不了机器

这一节在讲什么

为什么符号表示无法承载语义? 字典用文字解释文字, 递归下去仍是符号游戏。机器需要的不是定义, 而是一种能让「猫」与「狗」天然相邻、「国王」与「总统」有所关联的数学结构。

第一步: 认识 one-hot 编码的根本缺陷。 最朴素的表示方案把每个词映射为一个只有一位为 1 的稀疏向量。词表有 V 个词, 就有 V 维, 任意两词的向量内积都是零——语义相似性完全丢失。

第二步: 用稠密低维向量取而代之。 嵌入矩阵 $E \in \mathbb{R}^{V \times d}$ ($d \ll V$) 把每个词映射到一个 d 维实数向量。查表操作在数学上等价于矩阵乘法, 但因 one-hot 的稀疏性, 实际只需 $O(d)$ 计算。

第三步: 用余弦相似度衡量语义接近程度。 向量的方向而非长度承载语义——归一化后的夹角余弦值能屏蔽高频词模长偏大的噪声, 使「猫」与「狗」在几何上真正相邻。

本节要拿走的一件事: 词的含义不在符号名称里, 在向量空间中的**位置**里——几何关系即语义关系。

直觉

N-Gram 模型把每个词当作原子符号——词与词之间没有任何相似性信息。将「cat」替换为「dog」, 模型毫无察觉, 因为两者在符号层面是完全不同的对象。字典式的递归定义同样无济于事: 用文字解释文字, 最终仍是符号游戏。真正的突破是放弃符号表示, 转而在**实数向量**承载语义。词在高维空间中的位置——而非名字——成为其含义的载体。向量间的几何关系 (距离、角度) 自然地编码语义相

似性，使得「cat」与「dog」天然相邻，而无需任何显式的规则。

这一转变的数学动机并不神秘。one-hot 编码的两个致命缺陷——高维稀疏与任意两词正交——本质上是同一件事的两面：该表示根本没有利用词与词之间的统计共现信息。嵌入矩阵 E 的出现解决了两个问题：它把维度从 V （词表大小，通常十万量级）压缩到 d （数百至万量级），同时允许不同词的向量指向相近的方向，从而让内积和余弦值有意义。嵌入不是手工设计的，而是在训练目标的驱动下从数据中涌现——这意味着词的「含义」是被语料的统计结构塑造的，而非人工规定的。

一个常见误解是把嵌入维度越大等同于越好。维度增大固然能容纳更多的语义方向，但也带来过拟合风险和计算开销。更根本的是，词嵌入的「质量」取决于训练目标和语料，而不仅仅是 d 的大小。另外需要注意：**余弦相似度**选择余弦而非欧氏距离，是有具体理由的——高频词的嵌入向量由于受到更多梯度更新，模长会系统性偏大 (§2.5 详述)，归一化后的角度更稳健地反映语义方向，而不被模长的统计偏置干扰。

严格

One-hot 编码 (one-hot encoding) 是最朴素的词表示。设词表大小为 V ，词 w 对应词表中第 k 个位置，则其 one-hot 向量 $\mathbf{e}_w \in \{0, 1\}^V$ 满足：

$$(\mathbf{e}_w)_i = \begin{cases} 1 & i = k \\ 0 & \text{otherwise} \end{cases}$$

该表示有两个根本缺陷。第一，**稀疏性 (sparsity)**：向量中只有一个非零分量，储存和计算效率极低，在 $V \sim 10^5$ 量级时尤为突出。第二，**维度灾难 (curse of dimensionality)**：任意两个不同词的 one-hot 向量正交， $\mathbf{e}_u^T \mathbf{e}_v = 0$ ，无法表达语义相关性。

嵌入矩阵 (embedding matrix) $E \in \mathbb{R}^{V \times d}$ 将上述问题一并解决，其中 $d \ll V$ （典型地 $d \in [256, 12288]$ ）。矩阵 E 的第 k 行即为词 w_k 的稠密低维嵌入向量。**查表 (lookup)** 操作等价于矩阵-向量乘法：

$$\mathbf{v}_w = E^T \mathbf{e}_w \in \mathbb{R}^d$$

由于 $\mathbf{e}_w$ 是 one-hot 向量，该乘法实际上是直接取 E 的第 k 行，计算复杂度为 $O(d)$ 而非 $O(Vd)$ 。

余弦相似度 (cosine similarity) 是衡量两嵌入向量语义接近程度的标准度量：

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^T \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \frac{\sum_{i=1}^d u_i v_i}{\sqrt{\sum_{i=1}^d u_i^2} \cdot \sqrt{\sum_{i=1}^d v_i^2}}$$

选择余弦而非欧氏距离的动机在于：词频导致高频词的嵌入向量模长系统性偏大 (§2.5 将详述)，归一化后的角度更稳健地反映语义方向。

嵌入矩阵 E 的各分量在训练中通过反向传播学习，初始化通常为小的随机值（如 $\mathcal{N}(0, 1/d)$ ）。最终每一行对应一个词在语义几何空间中的坐标。

与原书呼应：原书 §2.1 指出 N-Gram 无法泛化的根本原因在于缺乏词间相似性信息，对应本节 one-hot 编码正交性缺陷的分析。

§2.2 语义的坐标系：那个被讲烂了的公式

这一节在讲什么

为什么训练一个「预测上下文」的目标，会产出有意义的语义几何？表面上，Skip-gram 只是在分类——给定中心词，猜哪些词会出现在旁边。但这个压力悄悄地把语义关系「写进」了向量方向。

第一步：理解训练目标的数学形式。全词表 softmax 计算代价为 $O(V)$ ，不可接受。负采样用局部二分类替代：每次只判断一对词是否真的共现，计算降至 $O(k)$ ， k 为负样本数。

第二步：理解类比几何的成立条件。 $\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$ 成立，是因为「性别」这一语义关系在空间中近似平行等长地分布——即嵌入空间对该关系具有平移等变性。这是充分条件，不总满足。

第三步：认识公式的边界。原始论文报告类比准确率约 60-70%；在上下文相关嵌入（BERT、LLaMA 隐藏状态）中，简单向量加减更频繁失败，线性表示假说给出了更精确的框架。

本节要拿走的一件事：那个被讲烂了的「国王 - 男人 + 女人 = 女王」不是魔法，是训练目标在数据统计压力下产生的近似几何性质，有明确的成立条件和失效边界。

直觉

Word2Vec 训练目标极为简单：用一个词预测其上下文中的词（或反过来）。统计上出现在相似上下文中的词，被「推到」向量空间中相邻的位置。这一训练压力的副产品是令人惊讶的语义几何：性别、国籍、时态等关系被编码为空间中固定的方向向量，使得向量加减能近似类比推理。但要理解这个现象的边界，必须先看清训练目标的数学形式。

负采样的设计逻辑值得细说。全词表 softmax 的分母要对 V 个词全部求和，当词表有十万个词时，每次梯度更新都要计算十万次指数运算，代价根本无法承受。负采样把这个全局归一化问题拆解成 $k+1$ 个局部二分类：对每一个真实的上下文词对，随机采 k 个「假」的词对，训练模型分辨真假。噪声分布按词频的 $3/4$ 次幂采样，是经验上对高频词进行适度降采样的工程折衷——这个指数并无严格理论推导，但在实践中比均匀采样或直接按词频采样效果都更好。

「国王 - 男人 + 女人 = 女王」这个公式之所以被讲烂，部分原因是它太好用作演示，部分原因是其适用范围被过度夸大。这个近似成立需要的条件是：语义关系在向量空间中近似平行且等长分布。对于「性别」这类在语料中高度规律的关系，条件接近满足；对于「首都 - 国家」这类关系，成功率参差不齐；对于抽象的情感、道德等维度，向量加减几乎失效。此外，上下文相关嵌入（BERT 或大型语言模型的中间层激活）的语义几何更为复杂，简单的静态向量算术不再适用——线性表示假说（§2.2 末）试图把这些现象统一到更严格的框架下，但仍是活跃的研究方向。

严格

Skip-gram 目标函数给定词 w ，最大化其上下文词 c （在窗口 $[-L, L]$ 内，排除自身）的条件概率：

$$\mathcal{L}_{\text{SG}} = \frac{1}{T} \sum_{t=1}^T \sum_{\substack{-L \leq j \leq L \\ j \neq 0}} \log P(w_{t+j} | w_t)$$

条件概率用 softmax 定义，引入两套向量：中心词向量 $\mathbf{v}_w \in \mathbb{R}^d$ （来自矩阵 $W \in \mathbb{R}^{V \times d}$ ）和上下文词向量 $\mathbf{u}_c \in \mathbb{R}^d$ （来自矩阵 $W' \in \mathbb{R}^{V \times d}$ ）：

$$P(c | w) = \frac{\exp(\mathbf{u}_c^T \mathbf{v}_w)}{\sum_{c'=1}^V \exp(\mathbf{u}_{c'}^T \mathbf{v}_w)}$$

分母对全词表求和，计算开销为 $O(V)$ ，在大词表下不可接受。**负采样 (negative sampling, NEG)** 以局部二分类替代全局 softmax：对每个正样本对 (w, c) ，从噪声分布 $P_n(c) \propto f(c)^{3/4}$ ($f(c)$ 为词频) 采 k 个负样本 $c'_1, \dots, c'_k$ ，目标函数变为：

$$\mathcal{L}_{\text{NEG}} = \log \sigma(\mathbf{u}_c^T \mathbf{v}_w) + \sum_{i=1}^k \mathbb{E}_{c'_i \sim P_n} [\log \sigma(-\mathbf{u}_{c'_i}^T \mathbf{v}_w)]$$

其中 $\sigma(x) = 1/(1 + e^{-x})$ 为 sigmoid 函数。典型地取 $k \in [5, 20]$ ，计算复杂度降至 $O(k)$ 。

类比关系的几何解释。 设 $\delta_{\text{gender}} = \mathbf{v}_{\text{man}} - \mathbf{v}_{\text{woman}}$ 。若训练数据中「king」和「queen」出现在几乎相同的上下文中，除性别相关词汇外，则梯度更新会将两者沿性别方向对齐，形成：

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{queen}} \approx \mathbf{v}_{\text{man}} - \mathbf{v}_{\text{woman}} = \delta_{\text{gender}}$$

即：

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$$

这一近似成立的充分条件是：语义关系（如性别）在空间中近似平行且等长地分布，即嵌入空间对该关系具有**平移等变性 (translational equivariance)**。然而，此条件并不总是满足：原始 Word2Vec 论文报告的类比准确率约为 60–70%，且依赖具体词对；在上下文相关嵌入（BERT、LLaMA 隐藏状态）中，简单向量加减更频繁失败。

线性表示假说 (Linear Representation Hypothesis) 将上述观察形式化：语义概念对应残差流 (residual stream) 中的线性方向，沿该方向干预可控制模型输出对应属性 (Park, Choe & Veitch, ICML 2024)。

与原书呼应：原书 §2.2 给出 king–man+woman queen 等若干类比示例，并引入线性表示假说，对应本节 Skip-gram 目标函数及类比几何成立条件的分析。

§2.3 高维几何的诡异世界

这一节在讲什么

为什么三维直觉在数百维的嵌入空间里几乎全部失效？当维度从 3 增长到 512 或 4096 时，「近」「远」「随机」这些概念的含义发生了根本性的变化，忽视这一点会导致对嵌入空间的严重误判。

第一步：球壳质量集中。高维高斯向量的模长以极高概率集中在 $\sqrt{d}$ 附近的薄球壳上——球的「内部」几乎是空的。这意味着嵌入向量之间的欧氏距离差异会被压缩，模长不再是有意义的区分量。

第二步：随机向量近乎正交。两个独立高斯向量的余弦值以 $1/\sqrt{d}$ 的速度趋于零。高维空间能容纳指数级多的「几乎正交」方向，这正是超位置（superposition）现象的数学基础。

第三步：距离保持降维是可行的。Johnson-Lindenstrauss 引理保证：把 n 个点投影到 $O(\log n/\varepsilon^2)$ 维，点对距离的失真不超过 ε 。目标维度与原始维度无关，只与点的数量的对数相关。

本节要拿走的一件事：高维空间「几乎所有方向都互相正交」——这既是词嵌入能承载海量语义方向的几何原因，也是简单向量运算在高维下需要格外谨慎的原因。

直觉

词嵌入的维度通常在数百到万量级。在这样的空间里，我们关于「近」「远」「随机点分布」的三维直觉几乎全部失效。理解高维几何的反直觉性质——质量集中于球壳、随机向量近似正交、内积的统计行为——是理解为何余弦相似度仍然有效、为何超位置现象能够存在的数学基础。

球壳质量集中的直觉可以用一个极端例子来感受：在三维空间里，单位球的体积中有相当大的比例在「内部」；但在一千维空间里，单位球几乎所有的体积都挤在半径接近 1 的极薄球壳里，内部几乎是真空。这对嵌入空间意味着：如果词向量是随机初始化的，它们几乎全部具有相似的模长（约 $\sqrt{d}$ ），欧氏距离对「近」和「远」的区分能力大幅下降。这也解释了为何嵌入空间中余弦相似度比欧氏距离更受青睐——在球壳上，方向差异比距离差异更有信息量。

随机向量近乎正交这一性质有两面性。从负面看，它意味着高维空间中几乎所有随机词对的余弦相似度都趋向于零，差异性被「稀释」，检索任务需要更精细的训练才能产生有区分力的方向。从正面看，指数级多的近正交方向意味着神经网络可以在同一组向量中「叠加」存储远超维度数量的特征——超位置（superposition）现象的数学根基正在于此（§2.3 严格部分推导）。Johnson-Lindenstrauss 引理则给出了一个工程上的安慰：做近似最近邻搜索时，可以把高维向量安全地投影到维度低得多的空间，只损失可控的距离精度，这是向量数据库能够实用化的理论保障。

严格

球壳质量集中 (measure concentration)。设 $\mathbf{x} = (x_1, \dots, x_d)^T$ ，各分量独立同分布 $x_i \sim \mathcal{N}(0, 1)$ 。则 $\|\mathbf{x}\|^2 = \sum_{i=1}^d x_i^2 \sim \chi^2(d)$ ，故：

$$\mathbb{E}[\|\mathbf{x}\|^2] = d, \quad \text{Var}[\|\mathbf{x}\|^2] = 2d$$

由集中不等式， $\|\mathbf{x}\|$ 以高概率集中在 $\sqrt{d}$ 附近，偏离幅度为 $O(d^{1/4})$ 。更精确地，对任意 $\varepsilon > 0$ ：

$$P\left(\left|\|\mathbf{x}\| - \sqrt{d}\right| > \varepsilon\right) \leq 2\exp\left(-\frac{\varepsilon^2}{4}\right)$$

直观含义：在高维下，高斯向量的模长以极高概率位于半径为 $\sqrt{d}$ 的球壳薄层内，球的「内部」几乎是空的。

高维内积分布。 设 $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ 独立同分布 $\mathcal{N}(\mathbf{0}, I_d)$ 。则内积 $\mathbf{x}^T \mathbf{y} = \sum_i x_i y_i$ ，各项独立，方差为 1，由中心极限定理：

$$\frac{\mathbf{x}^T \mathbf{y}}{\sqrt{d}} \xrightarrow{d \rightarrow \infty} \mathcal{N}(0, 1)$$

即 $\mathbf{x}^T \mathbf{y} \sim \mathcal{N}(0, d)$ ，标准差为 $\sqrt{d}$ ，而两向量的模长均约为 $\sqrt{d}$ ，故余弦值：

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} \approx \frac{\mathcal{N}(0, d)}{d} = \mathcal{N}(0, 1/d) \xrightarrow{d \rightarrow \infty} 0$$

随机高维向量几乎正交。这意味着：高维空间能容纳指数级多的「几乎正交」向量，这正是超位置 (superposition) 现象的数学基础。

Johnson-Lindenstrauss 引理 (Johnson-Lindenstrauss lemma)。 设 n 个点 $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^D$ ， $\varepsilon \in (0, 1/2)$ 。存在线性映射 $f: \mathbb{R}^D \rightarrow \mathbb{R}^k$ ，其中：

$$k = O\left(\frac{\log n}{\varepsilon^2}\right)$$

使得对所有点对 (i, j) ：

$$(1 - \varepsilon) \|\mathbf{x}_i - \mathbf{x}_j\|^2 \leq \|f(\mathbf{x}_i) - f(\mathbf{x}_j)\|^2 \leq (1 + \varepsilon) \|\mathbf{x}_i - \mathbf{x}_j\|^2$$

该映射可由随机高斯矩阵 $\Phi \in \mathbb{R}^{k \times D}$ （各元素独立 $\sim \mathcal{N}(0, 1/k)$ ）构造，以高概率满足上述距离保持性质。引理的关键推论：**降维所需的目标维度仅与点的数量对数相关，与原始维度 D 无关。**这解释了为何在高维嵌入中进行近似最近邻搜索是可行的。

余弦相似度在高维的稳定性。 虽然随机向量趋于正交，但经过训练的嵌入向量并非随机——其方向被语义信息约束。余弦相似度的优势在于：它对向量模长不敏感，只关注方向差异。在球壳质量集中的高维空间中，模长携带的信息主要是词频偏置（高频词嵌入模长系统偏大），归一化后恰好消除这一噪声，保留语义方向信号。

与原书呼应：原书 §2.3 列举了高维诡异性的三条现象，并引出超位置与稀疏自编码器 (SAE)，对应本节球壳集中、内积分布与 JL 引理的数学推导。

§2.4 跨界映射：从罗马路网到曼哈顿距离

这一节在讲什么

为什么同一个「距离」概念，换个度量方式就面目全非？欧氏距离只是 L^p 范数家族中的一员。在语义空间中，不同维度的「一个单位」代表的语义差异强度各不相同，一把尺子量不了所有的方向。

第一步：认识 L^p 范数家族。 $p = 1$ 是曼哈顿距离（沿坐标轴走）， $p = 2$ 是欧氏距离（直线距离）， $p \rightarrow \infty$ 是 Chebyshev 距离（取各维最大绝对值）。三者的单位球分别是菱形、圆球、正方形——几何形状截然不同。

第二步：引入加权度量矩阵。若嵌入空间各维度的「语义单位」不等长，可用正定矩阵 G 定义加权内积 $\mathbf{u}^T G \mathbf{v}$ ，对应的马氏距离消除各向异性，等价于在白化后的空间中计算欧氏距离。

第三步：区分内积空间与度量空间。内积空间自然诱导范数和度量；但反过来，满足平行四边形恒等式才能保证一个范数来自内积。词嵌入的「自然度量」实为加权内积，而非普通欧氏内积。

本节要拿走的一件事：选择哪种距离度量，本质上是对「哪些方向上的差异更重要」的假设——不同的 G 对应不同的语义几何观。

直觉

原书用「罗马路网」类比语义空间中的推理路径，这一类比暗示了空间是均匀欧氏的。但更精确的图像是曼哈顿网格：不同语义维度上的「一个单位」代表的语义差异强度各不相同，距离的计算需要一个加权度量。理解 L^p 范数族和内积空间与度量空间的差别，有助于把握嵌入空间几何的实际复杂度。

L^p 范数家族提供了一套从「严格惩罚大偏差」到「宽容大偏差」的连续调节旋钮。 L^1 范数（曼哈顿距离）对所有维度上的偏差一视同仁地累加，不会被某个方向上的极端值主导； L^2 范数（欧氏距离）通过平方放大了大偏差的影响； L^∞ 范数（Chebyshev 距离）则完全只关注偏差最大的那个维度。在稀疏表示和信息检索领域， L^1 范数因其对异常值的鲁棒性和与稀疏性的天然亲和（LASSO 正则化的几何本质）而受到青睐；而在嵌入空间的相似度检索中， L^2 和余弦距离更为主流，因为它们与内积运算直接挂钩，便于通过矩阵乘法高效批量计算。

马氏距离（Mahalanobis distance）是将「白化」思路融入距离定义的结果。当嵌入空间中不同维度的方差相差悬殊时，直接用欧氏距离会被高方差维度主导——就像用不同单位（公里 vs 毫米）混合计算距离一样失真。马氏距离用协方差矩阵的逆 Σ^{-1} 作为度量矩阵 G ，相当于先把数据白化（各向归一）再算欧氏距离。一个常见误解是把马氏距离当作「更复杂的欧氏距离」——实际上它是对欧氏距离的一个系统性修正，当 $G = I$ 时两者退化为同一个量。词嵌入中很少显式使用马氏距离，但白化操作（§2.5）正是这一思路的实际应用。

严格

L^p 范数 (L^p norm) 定义为：

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^d |x_i|^p \right)^{1/p}, \quad p \geq 1$$

几个特殊情形：

- $p = 1$ （曼哈顿距离，Manhattan distance / taxicab norm）： $\|\mathbf{x}\|_1 = \sum_i |x_i|$ 。单位球为超立方体的对角截面（ $d = 2$ 时为菱形）。

- $p = 2$ (欧氏范数, Euclidean norm): $\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$ 。单位球为超球面。
- $p \rightarrow \infty$ (Chebyshev 距离, Chebyshev distance): $\|\mathbf{x}\|_\infty = \max_i |x_i|$ 。单位球为超立方体。证明: $\|\mathbf{x}\|_p = (\sum_i |x_i|^p)^{1/p} \leq (d \cdot \max_i |x_i|^p)^{1/p} = d^{1/p} \max_i |x_i| \rightarrow \max_i |x_i|$ ($p \rightarrow \infty$); 下界同理可得。

加权内积与度量矩阵。若嵌入空间的不同维度具有不同的「语义单位长度」,可引入正定矩阵 $G \in \mathbb{R}^{d \times d}$ ($G \succ 0$), 定义加权内积:

$$\langle \mathbf{u}, \mathbf{v} \rangle_G = \mathbf{u}^T G \mathbf{v}$$

对应的马氏 (Mahalanobis) 距离 (Mahalanobis distance) 为:

$$d_G(\mathbf{u}, \mathbf{v}) = \sqrt{(\mathbf{u} - \mathbf{v})^T G (\mathbf{u} - \mathbf{v})}$$

当 $G = I$ 时退化为欧氏距离; 当 $G = \Sigma^{-1}$ (数据协方差矩阵的逆) 时, Mahalanobis 距离消除各向异性, 等价于在白化 (whitening) 后的空间中计算欧氏距离。

内积空间 (inner product space) vs 度量空间 (metric space)。内积空间配备双线性、对称、正定的内积, 自然诱导范数 $\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$ 及度量 $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$; 反过来, 并非所有度量都来自内积 (满足平行四边形恒等式 $\|\mathbf{u} + \mathbf{v}\|^2 + \|\mathbf{u} - \mathbf{v}\|^2 = 2\|\mathbf{u}\|^2 + 2\|\mathbf{v}\|^2$ 是内积范数的充要条件)。词嵌入通常假设在内积空间中工作, 但其「自然度量」实为加权内积, 而非普通欧氏内积。

与原书呼应: 原书 §2.4 以罗马路网 vs 曼哈顿网格为比喻, 指出词向量空间的「自然内积」为加权内积 $\mathbf{u}^T G \mathbf{v}$, 对应本节 L^p 范数几何含义与度量矩阵的数学形式化。

§2.5 反类比：当几何隐喻失效

这一节在讲什么

为什么嵌入空间的几何比我们想象的更「扭曲」? 词频统计在训练过程中悄悄地扭曲了向量空间的几何结构: 高频词聚集在一个狭窄的锥形区域, 嵌入空间严重各向异性, 余弦相似度和向量加减在这种条件下都面临系统性偏差。

第一步: 认识各向异性。理想的嵌入空间各向同性——向量均匀覆盖所有方向。实际中, GPT-2 各层嵌入的平均余弦相似度可达 0.99, 几乎所有词向量都指向同一方向, 大量语义方向被浪费。

第二步: 理解椎体效应的成因。负采样中高频词被频繁选为负样本, 受到更多「排斥」梯度, 最终被推向空间边缘, 形成低维锥。嵌入空间的有效维度远低于名义维度 d 。

第三步: 白化是可行的修正方案。对嵌入矩阵做 PCA 并按特征值归一化 (白化变换), 能恢复各向同性, 使余弦相似度在语义相似度任务上的表现显著提升。

本节要拿走的一件事: 嵌入空间不是干净的语义坐标系, 而是被词频统计「污染」过的各向异性空间——白化是让几何隐喻重新成立的必要前处理。

直觉

嵌入空间并非理想的欧氏空间。词频统计在空间中留下了系统性的几何扭曲: 高频词的嵌入向量聚集在一个狭窄的椎体 (cone) 中, 而非均匀分布在空间。这种**各向异性 (anisotropy)** 使得简单的

向量算术和余弦相似度都面临失真，并且解释了为何 king-man+woman queen 在现代嵌入中的成功率远低于早期论文所暗示的水平。

各向异性的成因可以从负采样的训练机制直接追溯。负采样以词频的 $3/4$ 次幂为噪声分布采样 (§2.2)，这意味着「the」「of」「and」这类高频词每个训练步都极可能被当作负样本，它们的上下文向量不断受到「远离当前中心词」的排斥梯度。经过数百万步训练，高频词的向量被系统性地推向空间的某个角落，彼此聚集在低维锥内。与此同时，嵌入矩阵的平均向量（词频加权重心）偏离原点，整个空间出现了一个强烈的「平均方向」——几乎每个词向量都与这个平均方向高度对齐，导致任意两词的余弦相似度虚高。这就是 Ethayarajh (2019) 所观测到的现象：BERT 上层的各向异性指标 $\bar{c}\bar{o}s$ 可高达 0.99，即余弦相似度几乎失去了区分不同词对的能力。

白化操作是对各向异性的直接修正，但也有其代价和局限。白化后每个方向的方差相等，余弦相似度恢复了对语义差异的灵敏性；但白化变换依赖于整个词表的协方差矩阵，在嵌入上下文相关时（每个词在不同句子中有不同的激活向量）计算代价较高，且变换会破坏原始嵌入的「线性语义方向」结构，使得类比算术在白化后反而更难奏效。因此，白化并非「万能前处理」，而是一个在特定任务（语义文本相似度评估）上有明确收益、在其他任务（类比推理、向量搜索）上需要权衡的工具。

严格

各向异性 (anisotropy)。理想的嵌入空间应满足各向同性：嵌入向量均匀分布在各个方向，任意随机向量与所有嵌入向量的平均余弦相似度为零。实践中，Ethayarajh (2019) 发现，BERT 的上下文嵌入高度各向异性——大多数嵌入向量与「平均方向」高度对齐，投影到低维子空间后几乎共线。形式化：设嵌入集合 $\{\mathbf{v}_w\}_{w=1}^V$ ，定义各向异性指标：

$$\bar{c}\bar{o}s = \frac{1}{\binom{V}{2}} \sum_{i < j} \cos(\mathbf{v}_i, \mathbf{v}_j)$$

各向同性时 $\bar{c}\bar{o}s \approx 0$ ；实际 GPT-2 的层级嵌入中 $\bar{c}\bar{o}s$ 可达 0.99，即几乎所有嵌入向量指向同一方向。

词频导致的椎体效应 (frequency-induced cone effect)。负采样目标函数 (§2.2) 中，高频词作为负样本被采到的概率高，其上下文向量受到更多「排斥」梯度，导致高频词向量被推向向量空间的边缘，聚集在一个狭窄的锥形区域内。Mimno & Thompson (2017) 将此称为「椎体效应」：频率最高的词（如「the」、「of」）形成一个低维锥，其余词分布在锥外，使得整个嵌入空间的有效维度远低于名义维度 d 。

白化 (whitening) 作为修正。白化变换通过对嵌入集合进行 PCA 并按特征值归一化，消除各向异性偏置：

设嵌入矩阵（去中心化后）的协方差为 $\Sigma = \frac{1}{V} E^T E \in \mathbb{R}^{d \times d}$ ，其特征分解为 $\Sigma = U \Lambda U^T$ ($\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$, $\lambda_1 \geq \dots \geq \lambda_d > 0$)。白化变换为：

$$\tilde{\mathbf{v}}_w = \Lambda^{-1/2} U^T \mathbf{v}_w$$

变换后 $\text{Cov}(\tilde{\mathbf{v}}) = I_d$ ，每个方向的方差相等，空间各向同性恢复。Su 等人 (2021) 的 BERT-whitening 实验表明，白化后用余弦相似度评估语义相似性的效果优于原始 BERT 表示，且在多项语义文本相似度基准上超过更复杂的对比学习方法。

后果总结。各向异性意味着：(1) 余弦相似度在未白化的嵌入空间中存在系统性偏置，高频词对之间的余弦值虚高；(2) 向量加减（如类比算术）在高频词参与时误差更大；(3) 嵌入空间的「有效维度」显著低于 d ，大量方向携带的是词频噪声而非语义信号。

与原书呼应：原书 §2.5 指出几何隐喻失效的三个层次（连续 vs 离散、概率 vs 确定、构造 vs 发现），与本节各向异性和椎体效应共同构成对「嵌入空间是理想欧氏空间」这一假设的否定。

§2.6 跨越欧氏：当表示空间不再「平直」

这一节在讲什么

为什么树形层级结构很难嵌入欧氏空间？ 树的节点数随深度指数增长，而欧氏球的体积只随半径多项式增长。这一根本不匹配意味着：要在欧氏空间中忠实地表示深层树，维度必须线性增长，代价不可接受。

第一步：认识欧氏嵌入的维度瓶颈。 完全 b -叉树有 $O(b^L)$ 个节点。在欧氏空间中，容纳 N 个互相保持距离的点所需维度满足 $d \geq \Omega(L \log b / \log r)$ ，对大深度 L 线性增长，不可避免。

第二步：双曲空间的体积指数增长正好匹配树。 Poincaré 球模型中，以原点为中心、半径为 r 的球的体积 $\propto e^{(d-1)r}$ ，与 b -叉树节点数的指数增长天然吻合。5 维双曲空间已能超过 200 维欧氏空间的层级重建精度。

第三步：球心对应抽象概念，边界对应具体叶节点。 在 Poincaré 球中，节点的层级深度与其到球心的双曲距离直接对应——边界不可达，越靠近边界越「具体」。

本节要拿走的一件事： 选择几何空间本质上是对数据内在拓扑的假设——欧氏空间对混合拓扑最稳健，双曲空间对强层级数据有显著收益，没有普适最优解。

直觉

树形层级结构（动物 → 哺乳动物 → 狗 → 金毛）的节点数随深度指数增长，而欧氏球的体积仅按多项式（ $\sim r^d$ ）增长。这种指数 vs 多项式的不匹配意味着：在欧氏空间中嵌入深层树需要巨大的维度才能保持层级距离关系。双曲空间正好相反——其「体积」随半径指数增长，天然匹配树的拓扑。Poincaré 球模型给出了一个具体且数学上自洽的框架。

双曲空间的几何直觉可以用「无限扩张的平面」来感受。在普通的欧氏平面上，围绕一个点画半径为 r 的圆，圆周长是 $2\pi r$ ，与 r 线性相关。而在双曲平面上，同样半径的圆周长正比于 e^r ——空间越往外越「宽广」，像一个向外无限加速膨胀的曲面。Poincaré 球模型把这种无限的双曲空间「压缩」进一个有限的单位球里，代价是靠近边界时欧氏度量被保角因子 $\frac{4}{(1-\|\mathbf{x}\|^2)^2}$ 急剧放大——边界不可达，等价于双曲距离趋于无穷。这个框架的精妙之处在于：树的深度（层级）直接对应到球中的径向距离，父节点自然位于子节点的「内侧」，不需要任何额外的约束。

一个值得注意的实践问题是：双曲空间中的优化比欧氏空间更棘手。Poincaré 球上的梯度下降需要在黎曼流形上操作，普通的欧氏梯度更新会把点推出单位球边界，必须用黎曼梯度和指数映射替代。此外，双曲嵌入对初始化和学习率极为敏感——靠近边界的点距离变化剧烈，一个小的欧氏扰动对应巨大的双曲距离变化。Nickel & Kiela (2017) 在实现中使用了梯度裁剪和边界约束（ $\|\mathbf{x}\| \leq 1 - \epsilon$ ）来保证数值稳定性。这些工程上的复杂性，加上自然语言语义关系并非纯粹层级拓扑，是主流语言模型至今仍坚持欧氏表示空间的实际原因。

严格

欧氏嵌入的维度瓶颈。 深度为 L 、分支数为 b 的完全 b -叉树有 $N = \frac{b^{L+1}-1}{b-1} = O(b^L)$ 个节点。在欧氏空间 $\mathbb{R}^d$ 中，半径为 r 的球最多包含 $O(r^d)$ 个互相距离 ≥ 1 的点（packing argument）。要将 N 个

节点嵌入使得树上距离被 $(1 + \varepsilon)$ -近似保持，所需维度满足 $r^d \geq N$ ，即：

$$d \geq \Omega\left(\frac{L \log b}{\log r}\right)$$

对大 L 此下界线性增长，不可避免。

Poincaré 球模型 (Poincaré ball model)。 d 维双曲空间 $\mathbb{H}^d$ 的 Poincaré 球模型定义在开单位球 $\mathbb{B}^d = \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\| < 1\}$ 上，配以黎曼度量：

$$ds^2 = \frac{4 \|\mathrm{d}\mathbf{x}\|^2}{(1 - \|\mathbf{x}\|^2)^2}$$

即欧氏度量乘以保角因子 $\lambda(\mathbf{x})^2 = \frac{4}{(1 - \|\mathbf{x}\|^2)^2}$ (共形因子)。当 $\|\mathbf{x}\| \rightarrow 1$ 时 $\lambda \rightarrow \infty$ ，空间被无限「拉伸」——有限欧氏距离对应无限双曲距离，边界不可达。

双曲距离公式。对 $\mathbf{u}, \mathbf{v} \in \mathbb{B}^d$ ，双曲距离为：

$$d_{\mathbb{H}}(\mathbf{u}, \mathbf{v}) = \operatorname{arcosh}\left(1 + 2 \frac{\|\mathbf{u} - \mathbf{v}\|^2}{(1 - \|\mathbf{u}\|^2)(1 - \|\mathbf{v}\|^2)}\right)$$

其中 $\operatorname{arcosh}(t) = \ln(t + \sqrt{t^2 - 1})$, $t \geq 1$ 。

推导梗概。从 $\mathbf{u}$ 到 $\mathbf{v}$ 的双曲测地线长度为 $\int_{\gamma} ds$ ，利用 Möbius 变换将 $\mathbf{u}$ 映射到原点 (原点到 $\mathbf{r}$ 的距离简化为 $\int_0^{\|\mathbf{r}\|} \frac{2 dt}{1-t^2} = 2 \operatorname{arctanh}(\|\mathbf{r}\|)$)，再利用 arcosh 与 $\operatorname{arctanh}$ 的关系 $\operatorname{arcosh}(1 + 2t^2/(1-t^2)) = 2 \operatorname{arctanh}(t)$ ，得到上述闭式表达。

指数体积增长。在 Poincaré 球中，以 $\mathbf{0}$ 为中心、双曲半径为 r 的球的 (欧氏) 体积为：

$$\operatorname{Vol}_{\mathbb{H}}(B(r)) \propto \sinh^{d-1}(r) \sim \frac{1}{2^{d-1}} e^{(d-1)r}, \quad r \rightarrow \infty$$

体积随 r **指数增长**，与 b -叉树节点数的指数增长 $O(b^r)$ (以深度 r 衡量) 匹配，只需令 $b = e^{d-1}$ 。因此，在 $d = 5$ 维双曲空间中，已能容纳 WordNet 约 8×10^4 个名词节点，且层级距离重建精度超过 200 维欧氏嵌入 (Nickel & Kiela, 2017)。

层级位置编码的几何含义。在 Poincaré 球中，节点在层级结构中的深度与其到球心的双曲距离直接对应：

- **球心** ($\|\mathbf{x}\| \approx 0$)：抽象概念、树根 (如「实体」)；
- **球边界** ($\|\mathbf{x}\| \rightarrow 1$)：具体叶节点 (如「金毛犬」)，到边界的距离趋于无穷。

父节点到所有子节点的距离均等且较短，体现层级关系；兄弟节点之间的距离则由双曲距离公式的分母 $(1 - \|\mathbf{u}\|^2)(1 - \|\mathbf{v}\|^2)$ 调节，靠近边界时兄弟之间距离迅速增大，有效隔离不同子树。

为何主流 LLM 仍用欧氏空间。自然语言的语义关系同时包含层级 (is-a) 和网状联想 (associate-with) 结构。欧氏空间对混合拓扑最为稳健；双曲空间在强层级数据 (本体树、分子图) 上有明确收益，但在通用语言建模中的优势尚不稳定。选择几何空间本质上是对**数据内在拓扑**的假设，没有普适最优解。

与原书呼应：原书 §2.6 介绍 Poincaré 嵌入、双曲距离公式及主流 LLM 仍用欧氏空间的原因，对应本节度量推导、指数体积增长的严格分析，以及几何选择的元原则。

数学卷 · 第 3 章：注意力机制的矩阵微分

§3.1 从 RNN 到变换器 (Transformer)：序列建模的范式革命

这一节在讲什么

为什么 RNN 不能并行？ RNN 把序列理解为一个因果链，每步计算依赖前一步的隐状态。这种设计在直觉上符合「逐字阅读」的体验，却注定无法并行——时序依赖锁死了硬件。

第一步：量化梯度消失。 RNN 从时刻 T 向 t 反传梯度，需连乘 $T - t$ 个雅可比矩阵。若其谱半径 $\rho < 1$ ，梯度以 ρ^{T-t} 指数衰减，模型无法学习长距离依赖；若 $\rho > 1$ ，梯度爆炸。LSTM 能缓解但不能根治。

第二步：Transformer 的切断。 Transformer 彻底去掉时序递推：输入矩阵 $X \in \mathbb{R}^{n \times d}$ 整体存在，每个位置的输出通过矩阵乘法直接从全局上下文「读取」， n 个位置完全并行。

第三步：代价是什么。 并行化的代价是失去天然的顺序归纳偏置，必须显式注入位置信息（见 §3.4 RoPE），且计算量是 $O(n^2 d)$ ，序列越长越昂贵（见 §3.5 FlashAttention）。

本节要拿走的一件事： Transformer 的崛起不是「注意力更聪明」，而是「矩阵乘法比时序递推更适合 GPU」——这是一次范式替换，不是渐进改良。

直觉

循环神经网络 (Recurrent Neural Network, RNN) 把序列理解为一个因果链：每一步的计算依赖前一步的结果。这种设计在直觉上符合「阅读」的体验，但同时带来了致命的时序依赖——序列无法并行处理，梯度信号在长链上传播时会随步数指数式放大或衰减。Transformer 的根本突破在于：彻底切断时间依赖，让每个位置的表示直接从全局上下文中「读取」，把序列问题还原为一组独立的矩阵运算，从而让 GPU 的大规模并行硬件得以充分发挥。

理解这一范式替换的关键是看清 RNN 的数学锁链：从最终时刻 T 反传到时刻 t 的梯度，是 $T - t$ 个雅可比矩阵的连乘积。这不是可以绕过的实现细节，而是链式法则的必然结果。LSTM 的门控机制让部分梯度路径的有效谱半径更接近 1，本质上是工程上的「防护栏」，而非数学意义上的根治。相比之下，Transformer 的设计选择是：让任意两个位置之间的信息路径长度固定为 1——无论它们在序列中相距多远，注意力机制都在单次矩阵运算内完成全局聚合。

这一选择带来了三个深远影响。第一，计算图的深度从 $O(n)$ 降至 $O(L)$ (L 为层数)，梯度传播路径大幅缩短。第二，所有位置可完全并行，充分利用 GPU 的 SIMD 架构。第三，但也正因为没有了时序归纳偏置，模型对位置一无所知——必须通过位置编码 (§3.4) 显式补回。常见的误解是认为 Transformer 「更擅长捕捉长距离依赖」，准确的说法是：它的架构不会主动遗忘长距离信号，但能否真正利用它，还取决于参数训练的质量。

严格

RNN 的隐状态 (hidden state) 递推公式为

$$h_t = \sigma(W h_{t-1} + U x_t + b)$$

其中 $h_t \in \mathbb{R}^{d_h}$ 是第 t 步隐状态， $W \in \mathbb{R}^{d_h \times d_h}$ ， $U \in \mathbb{R}^{d_h \times d_x}$ ， σ 为逐元素非线性激活（如 $\tanh$ ）。

梯度消失/爆炸的数学根源

在时刻 T 的损失 $\mathcal{L}$ 对时刻 t 的隐状态求梯度，需通过链式法则展开

$$\frac{\partial \mathcal{L}}{\partial h_t} = \frac{\partial \mathcal{L}}{\partial h_T} \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k}$$

每一步的雅可比矩阵 (Jacobian matrix) 为

$$\frac{\partial h_{k+1}}{\partial h_k} = \text{diag}(\sigma'(Wh_k + Ux_{k+1}))W$$

连乘项的谱范数满足

$$\left\| \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k} \right\|_2 \leq \prod_{k=t}^{T-1} \|\text{diag}(\sigma')\|_2 \cdot \|W\|_2$$

设 $\rho = \|W\|_2 \cdot \max |\sigma'|$ (即该乘积谱半径的上界)，则梯度范数以 ρ^{T-t} 的速率随距离 $T-t$ 指数变化：

- 若 $\rho < 1$ ：梯度指数衰减 (梯度消失, gradient vanishing)；
- 若 $\rho > 1$ ：梯度指数增长 (梯度爆炸, gradient explosion)。

LSTM 通过门控机制 (gating mechanism) 使部分梯度路径的有效谱半径接近 1，但无法从根本上解除连乘结构，长序列仍受限于此。

为什么 Transformer 可并行

Transformer 完全去掉了时序递推：给定输入矩阵 $X \in \mathbb{R}^{n \times d}$ (n 个 token 同时存在于矩阵中)，每个位置的输出仅通过矩阵乘法和 softmax 计算，不依赖任何「前序状态」。具体而言，注意力层的输入输出关系为

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中 Q, K, V 均由 X 一次性线性投影得到，整个计算路径无时间轴上的串行依赖，因此 n 个位置的输出可完全并行计算。

与原书呼应：原书 §3.1 提到 RNN「不能并行」和「长距离依赖丢失」两个缺陷，对应本节雅可比连乘的谱半径分析与并行化条件。

§3.2 注意力公式的拆解：QKV 的真实含义

这一节在讲什么

为什么注意力要用三个矩阵 Q, K, V ？不用三个，用一个或两个行不行？这不是历史偶然，而是「软检索」语义的最小充分参数化。

第一步：把注意力理解为软索引。给定查询 Q ，在键集合 K 中按相似度打分，再用归一化的分数加权混合值 V 。与哈希表不同，这里每个键都有权重，结果是所有值的连续混合。

第二步：为什么要除以 $\sqrt{d_k}$ 。若 q 和 k 各分量方差为 1，它们的内积方差为 d_k ，标准差为 $\sqrt{d_k}$ 。不缩放时 softmax 的输入量级随维度增大，导致概率质量集中于极少数项，梯度趋零。除以 $\sqrt{d_k}$ 将方差还原为 1。

第三步：多头的意义。将 d 维空间切成 h 个子空间，每头在低维子空间中独立做软检索，捕捉不同粒度的语义关系（语法依赖、指代关系、话题关联……），总参数量不变。

本节要拿走的一件事： $\sqrt{d_k}$ 缩放不是调参技巧，是防止高维内积让 softmax 退化为 one-hot 的数学保险。

直觉

把注意力机制想象成一种「软索引」：给定一个查询向量，在一组键值对中按相似度加权检索。与硬索引（哈希表）不同，这里每个「键」都会有或多或少的匹配权重，最终返回的是所有「值」的加权混合。 $\sqrt{d_k}$ 缩放的作用是压制高维内积随维度增大而自然增大的方差，防止 softmax 因输入过大而在某一项上接近饱和、梯度接近零。多头设计则让模型同时在多个线性子空间中做这种检索，捕捉不同粒度的语义关系。

理解 Q, K, V 三者为什么必须是三个独立矩阵而非一个，关键在于分清「检索逻辑」和「被检索的内容」。 Q 和 K 共同决定「谁关注谁」——这是一个关系打分系统，需要两个独立的投影才能让注意力模式具有足够的表达自由度（否则相当于强制 $Q = K$ ，注意力矩阵 QK^T 退化为对称矩阵，丢失方向性）。 V 则是「被提取的内容」，它与打分逻辑解耦：即使两个 token 在某个语义维度上高度相关（高注意力权重），被传递的内容可以是完全不同的特征向量。这种「打分器」与「内容库」的分离，正是 attention 比单纯点积相似度更强大的原因。

常见的误解是把多头注意力理解为「冗余的重复」。实际上，每个头的投影矩阵是独立训练的，它们会在反向传播的压力下自发分工：有的头专注于句法依赖（动词-主语）、有的头追踪指代关系、有的头捕捉局部 n -gram 模式。这种分工是涌现的，不是人为指定的。正因为每头只处理 d/h 维的子空间，参数量与单头相同，但表达能力通过维度分解而倍增——这是多头设计的真正价值。

严格

设输入序列嵌入矩阵 $X \in \mathbb{R}^{n \times d}$ ，可学习投影矩阵

$$W^Q, W^K \in \mathbb{R}^{d \times d_k}, \quad W^V \in \mathbb{R}^{d \times d_v}$$

三个投影矩阵给出

$$Q = XW^Q \in \mathbb{R}^{n \times d_k}, \quad K = XW^K \in \mathbb{R}^{n \times d_k}, \quad V = XW^V \in \mathbb{R}^{n \times d_v}$$

缩放因子 $\sqrt{d_k}$ 的来源

设 $q_i, k_j \in \mathbb{R}^{d_k}$ 各分量独立同分布，均值为 0、方差为 1，则

$$\text{Var}(q_i \cdot k_j) = \text{Var}\left(\sum_{l=1}^{d_k} q_{il} k_{jl}\right) = \sum_{l=1}^{d_k} \text{Var}(q_{il}) \text{Var}(k_{jl}) = d_k$$

即内积 $q_i \cdot k_j$ 的标准差为 $\sqrt{d_k}$ 。不作缩放时，当 d_k 较大（典型值 64–128），内积分布的量级随 $\sqrt{d_k}$ 线性增大，进入 softmax 后所有概率质量集中于极少数最大项，形成近似 one-hot 分布，梯度趋近于零。除以 $\sqrt{d_k}$ 将方差还原为 1，保持 softmax 输入在合理量级。

注意力分数与归一化

注意力分数矩阵

$$A = \frac{QK^T}{\sqrt{d_k}} \in \mathbb{R}^{n \times n}, \quad A_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

softmax 沿第二维（行内）归一化，即对第 i 行独立计算

$$P_{ij} = \frac{\exp(A_{ij})}{\sum_{l=1}^n \exp(A_{il})}, \quad \sum_{j=1}^n P_{ij} = 1$$

这使得 $P \in \mathbb{R}^{n \times n}$ 的每行构成一个概率分布：第 i 个 token 对所有位置的关注权重之和为 1。输出为

$$\text{Attention}(Q, K, V) = PV \in \mathbb{R}^{n \times d_v}$$

多头注意力的参数化

设头数为 h ，每头的子空间维度通常取 $d_k = d_v = d/h$ 。第 i 头 ($i = 1, \dots, h$) 独立维护一套投影矩阵

$$W_i^Q \in \mathbb{R}^{d \times d_k}, \quad W_i^K \in \mathbb{R}^{d \times d_k}, \quad W_i^V \in \mathbb{R}^{d \times d_v}$$

各头输出拼接后再经输出投影 $W^O \in \mathbb{R}^{hd_v \times d}$ ：

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

其中 $\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$ 。多头设计的参数总量与单头相同（ d^2 量级），但在 h 个低维子空间中并行运行不同的「检索模式」。

与原书呼应：原书 §3.2 逐步拆解了 QKV 投影与缩放技巧，对应本节方差推导与 softmax 归一化维度分析。

§3.3 矩阵乘法的大规模并发：为什么 AI 是 GPU 的盛宴

这一节在讲什么

为什么矩阵乘法 and GPU 如此般配？GPU 的设计哲学是「以宽度换速度」——数千个简单核心并发执行同一条指令，作用于不同数据。矩阵乘法天然吻合：每个输出元素是独立内积，可完全并行。

第一步：算术强度决定瓶颈在哪里。 算术强度 $I = \text{FLOPs}/\text{内存字节数}$ 。大矩阵乘法的 $I \approx 2N/3$ 随矩阵尺寸线性增长，轻松超过 GPU 的「屋脊点」，属于计算受限——GPU 算力能被充分榨取。

第二步：softmax 是短板。 逐元素操作（如 $\exp$ ）算术强度接近 1，远低于屋脊点，属于内存受限。标准 attention 中 softmax 和矩阵读写的来回搬运，才是真正的性能瓶颈（见 §3.5）。

第三步：attention 的算力复杂度是 $O(n^2d)$ 。 序列长度 n 一旦增大， n^2 项主导，FLOPs 急剧增长——这是长上下文昂贵的根本原因。

本节要拿走的一件事：「AI 是 GPU 的盛宴」不是比喻，是算术强度分析的结论——矩阵乘法恰好落在 GPU 架构最擅长的计算受限区间。

直觉

GPU 的设计哲学是「以宽度换速度」：数以千计的简单核心并发执行同一条指令，作用于不同数据。矩阵乘法天然符合这一模式——结果矩阵的每个元素是独立内积，可完全并行。反观 attention 的计算，其核心运算 QK^T 、 PV 均为大型矩阵乘法；但真正的瓶颈往往不在浮点计算本身，而在芯片片上缓存（SRAM）与显存（HBM）之间的数据搬运速度——即算术强度（arithmetic intensity）是否高到足以让计算单元保持满载。

要理解为什么「AI 是 GPU 的盛宴」，需要区分两类瓶颈：**计算受限**和**内存受限**。对大矩阵乘法而言，输出的每个元素要做 k 次乘加，但数据（矩阵 A 、 B 、 C ）只需要读写一次；矩阵越大，每次内存访问分摊到的计算量越高，算术强度随 N 线性增长。这使得大矩阵乘法属于计算受限——硬件的算力峰值是瓶颈，而非内存带宽。GPU 的 Tensor Core 正是为此设计的：以矩阵乘法为基本指令，一次处理 16×16 或更大的分块，完全隐藏内存访问延迟。

然而，attention 流程中并非所有操作都是矩阵乘法。softmax 是逐行的 $\exp$ 和归约，其算术强度接近 1 FLOP/byte，远低于 GPU 的屋脊点——它是典型的内存受限操作。更关键的是，标准 attention 实现需要把 $n \times n$ 的注意力矩阵 S 写回 HBM、再读出来做 softmax、再写回、再读出来乘 V ——每轮都要经过慢速 HBM，矩阵乘法的算力优势被数据搬运彻底抹掉。这正是 FlashAttention (§3.5) 的动机：用分块技术让整个「矩阵乘 + softmax + 再乘」的流程在 SRAM 内完成，把 attention 从内存受限拉回计算受限。

严格

矩阵乘法的算术强度

对两个矩阵相乘 $C = AB$ ，其中 $A \in \mathbb{R}^{m \times k}$ ， $B \in \mathbb{R}^{k \times n}$ ， $C \in \mathbb{R}^{m \times n}$ ：

- 浮点运算量（FLOPs）： $2mnk$ （每个输出元素做 k 次乘加）
- 内存访问字节数（读 A 、 B ，写 C ）： $(mk + kn + mn) \times \text{sizeof}(\text{dtype})$

算术强度定义为

$$I = \frac{2mnk}{mk + kn + mn} \quad (\text{FLOP/byte})$$

当 m, n, k 均很大（如 $m = n = k = N \gg 1$ ）时， $I \approx \frac{2N^3}{3N^2} = \frac{2N}{3}$ ，随矩阵尺寸线性增长。典型大矩阵乘法的算术强度远超 GPU 的「屋脊点」（ridge point，即峰值算力 / 峰值内存带宽），因此属于**计算受限**（compute-bound）操作，GPU 的算力可以充分利用。

Roofline 模型简介

屋脊线（Roofline）模型将核函数的实际性能 P （FLOP/s）建模为

$$P = \min(\text{峰值算力}, I \times \text{峰值内存带宽})$$

当 I 低于屋脊点时，性能受内存带宽限制（memory-bound）；高于屋脊点时，性能受峰值算力限制（compute-bound）。对 A100 GPU，屋脊点约为 $312 \text{ TFLOP/s} / 2 \text{ TB/s} = 156 \text{ FLOP/byte}$ ；大矩阵乘法的算术强度通常超过此值，而逐元素操作（如 softmax 的 exp）算术强度接近 1，远低于屋脊点。

Attention 的 FLOPs 分析

对单头注意力，设序列长度 n ，键/查询维度 d_k ，值维度 d_v ：

步骤	计算	FLOPs
$S = QK^T$	$\mathbb{R}^{n \times d_k} \times \mathbb{R}^{d_k \times n}$	$2n^2 d_k$
softmax	逐行归一化	$\Theta(n^2)$ （低算术强度）
$O = PV$	$\mathbb{R}^{n \times n} \times \mathbb{R}^{n \times d_v}$	$2n^2 d_v$

忽略常数，总 FLOPs 为

$$\text{FLOPs}_{\text{attention}} = \Theta(n^2 d)$$

其中 $d = d_k = d_v$ 。对 h 头，乘以 h ，但此时每头 $d_k = d/h$ ，总量不变。 $O(n^2 d)$ 的二次复杂度使得长上下文（大 n ）下计算量急剧增长，是 attention 的核心算力瓶颈。

与原书呼应：原书 §3.3 提到 LLaMA-3 70B 单层 attention 的算力估算及 GPU 的并行哲学，对应本节算术强度与 Roofline 模型的量化分析。

§3.4 RoPE: 把位置编码从加法变成旋转

这一节在讲什么

为什么正弦位置编码不够用？加法叠加把绝对位置混入 token 表示，让模型无法直接感知「两个 token 相距多远」，且在训练长度之外几乎立即失效。

第一步：换一种思路——把位置编码进旋转角。RoPE 将每对相邻维度 (q_{2i}, q_{2i+1}) 看成复平面上的一个点，位置 m 对应旋转 $e^{im\theta_i}$ ，不加法，只旋转。

第二步：旋转的魔法——内积只剩位置差。由于旋转矩阵正交， $\langle \tilde{q}_m, \tilde{k}_n \rangle = q_m^T \mathcal{R}_{n-m} k_n$ ，绝对位置 m, n 从内积中消去，只留下相对位置 $n - m$ 。这正是语言学上真正有用的信号。

第三步：多频率覆盖多尺度。低维度对应高频旋转（短程依赖），高维度对应低频旋转（长程依赖），类似傅里叶分析的多尺度分解。调整底数可延伸训练外推长度。

本节要拿走的一件事：RoPE 把「相对位置」这一语言学核心信号，用旋转矩阵的正交性以数学方式硬编码进注意力内积——不依赖模型去「学」位置关系，而是直接设计进结构。

直觉

原始 Transformer 的正弦位置编码 (Sinusoidal Positional Encoding) 是在嵌入向量上做加法：每个位置对应一个固定向量叠加到 token 表示上。这种绝对编码让模型难以直接利用「两个 token 相距多远」这一语言学上更本质的信号，且在训练长度之外几乎立即失效。旋转位置编码 (Rotary Position Embedding, RoPE) 改变思路：不加法，而是把位置信息编码为向量在复平面上的旋转角度，使得 Query 与 Key 的内积天然只依赖二者的相对位置差，绝对位置信息从内积中消去。

为什么旋转而不是加法能做到这一点？核心在于旋转矩阵的正交性。若对 q_m 施加旋转 $\mathcal{R}_m$ 、对 k_n 施加旋转 $\mathcal{R}_n$ ，则它们的内积为 $q_m^T \mathcal{R}_m^T \mathcal{R}_n k_n = q_m^T \mathcal{R}_{n-m} k_n$ ——绝对位置 m 和 n 在内积中完全抵消，只剩下位置差 $n - m$ 。这不是近似，而是精确的代数恒等式，由旋转矩阵的群结构（角度可加性）保证。加法编码做不到这一点，因为加法后内积会产生绝对位置的交叉项，无法干净分离。

多频率设计则对应多尺度的位置感知。频率参数 $\theta_i = 10000^{-2i/d}$ 随维度下标 i 的增大而减小：低下标维度旋转快（高频，敏感于相邻 token 的差异），高下标维度旋转慢（低频，感知长程距离的变化）。这与傅里叶分析的思路一致：用不同频率的正弦波组合来表示不同尺度的信号。实践中，通过把底数 10000 替换为更大的值（如 YaRN、LongRoPE 等方法），可以在不重新训练的情况下向更长上下文外推——这是 RoPE 区别于加法编码的另一个工程优势。

严格

正弦位置编码的形式

对位置 pos 和嵌入维度下标 $2i, 2i+1$ ：

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

编码后 $x'_{\text{pos}} = x_{\text{pos}} + \text{PE}(\text{pos})$ ，绝对位置以加法混入嵌入，无法分离相对位置信息。

RoPE 的复数化表示

RoPE 将 d 维向量 $q \in \mathbb{R}^d$ 看成 $d/2$ 个复数，第 i 对分量 (q_{2i}, q_{2i+1}) 对应复数 $z_i^q = q_{2i} + i q_{2i+1}$ 。对位于位置 m 的 token，每对分量乘以单位复数 $e^{im\theta_i}$ ：

$$\tilde{z}_i^q = z_i^q \cdot e^{im\theta_i}$$

频率参数

$$\theta_i = 10000^{-2i/d}, \quad i = 0, 1, \dots, \frac{d}{2} - 1$$

低下标 i 对应高频（旋转快），高下标对应低频（旋转慢），覆盖不同尺度的位置信息。

等价实数旋转矩阵

对每对分量，乘以 $e^{im\theta_i}$ 等价于施加二维旋转矩阵

$$R_m^{(i)} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix}$$

对整个 d 维向量，RoPE 的变换可写为块对角旋转矩阵

$$\mathcal{R}_m = \text{blockdiag}\left(R_m^{(0)}, R_m^{(1)}, \dots, R_m^{(d/2-1)}\right) \in \mathbb{R}^{d \times d}$$

位置 m 处的 query 和 key 在施加 RoPE 后分别为

$$\tilde{q}_m = \mathcal{R}_m q_m, \quad \tilde{k}_n = \mathcal{R}_n k_n$$

核心性质：内积只依赖相对位置

由于旋转矩阵是正交矩阵 ($\mathcal{R}_m^T = \mathcal{R}_m^{-1} = \mathcal{R}_{-m}$)，且旋转角度可加 ($\mathcal{R}_m^T \mathcal{R}_n = \mathcal{R}_{n-m}$)，attention score 为

$$\langle \tilde{q}_m, \tilde{k}_n \rangle = (\mathcal{R}_m q_m)^T (\mathcal{R}_n k_n) = q_m^T \mathcal{R}_m^T \mathcal{R}_n k_n = q_m^T \mathcal{R}_{n-m} k_n = \langle q_m, \mathcal{R}_{n-m} k_n \rangle$$

结果仅依赖位置差 $n - m$ ，而不依赖绝对位置 m 或 n 。这一性质使得模型在推理时天然感知相对距离，并且——通过调整频率基数 θ_i 的底数（将 10000 替换为更大的值）或对不同频率段做差异化插值——可以向训练长度以外外推，这是 YaRN 等长上下文扩展方法的数学基础。

与正弦编码的对比如下：

属性	正弦位置编码	RoPE
编码方式	加法叠加	乘法旋转
位置信号	绝对位置	相对位置
内积形式	依赖绝对位置	仅依赖 $n - m$
训练外外推	直接失效	可通过频率调整延伸

与原书呼应：原书 §3.4 介绍了 RoPE 的旋转思想与「位置差从指数里掉出来」的性质，对应本节旋转矩阵正交性的严格推导。

§3.5 FlashAttention: 当注意力撞上内存墙

这一节在讲什么

为什么标准 attention 慢？不是因为 FLOPs 太多，而是因为 $n \times n$ 的注意力矩阵要在慢速 HBM 和快速 SRAM 之间反复搬运，内存带宽成了瓶颈。

第一步：问题出在写-读-写-读的搬运链上。标准实现先把 $S = QK^T$ 写到 HBM，再读出来做 softmax，再写回，再读出来乘 V ——每步都过 HBM，HBM 访问量 $\Theta(n^2)$ 。

第二步：FlashAttention 的思路是「不出芯片」。把 Q, K, V 切成能放入 SRAM 的小块，在片上完成「乘 + softmax + 再乘」全流程，只将最终输出写回 HBM 一次。

第三步：难点在 softmax 需要全局统计量。FlashAttention 用「在线 softmax」解决：用滚动最大值 m_j 和指数和 ℓ_j 递推，无需提前知道全局最大值，可以边读块边更新输出。

本节要拿走的一件事：FlashAttention 的 FLOPs 没有减少，减少的是 HBM 读写量（从 $\Theta(n^2)$ 降至接近线性）——这是一次纯粹的内存工程优化，不改变数学结果，只改变计算路径。

直觉

标准 attention 实现的瓶颈不在浮点运算量，而在显存带宽： $n \times n$ 的注意力矩阵需要在 GPU 片上缓存 (SRAM) 和高带宽显存 (HBM) 之间反复搬运，而 SRAM 的带宽约是 HBM 的 10 倍但容量小四个数量级。FlashAttention 的思路是：把 Q, K, V 切成能放入 SRAM 的小块，在芯片内部完成「矩阵乘 + softmax + 再乘」的全流程，只将最终输出写回 HBM，从而将 HBM 访问量从 $\Theta(n^2)$ 降至接近线性，而浮点运算量不变。难点在于 softmax 需要全局统计量，FlashAttention 通过在线 softmax 技巧解决了这一矛盾。

理解 FlashAttention 的关键是搞清楚「问题出在哪一步」。标准实现中， $S = QK^T$ 计算完后必须写回 HBM (SRAM 放不下)，softmax 再从 HBM 读出来处理， P 写回 HBM，最后再读出来与 V 相乘。这种「写-读-写-读」的链条使得 HBM 访问量正比于 n^2 ，而 HBM 带宽远低于算力峰值，导致 GPU 的大量计算单元处于等待数据的空转状态。FlashAttention 的洞察是：softmax 的「全局归一化」需求（必须看到整行才能除以总和）是阻碍分块计算的唯一障碍——只要解决这个问题，就可以让整个 attention 计算在片上完成。

在线 softmax (online softmax) 递推是解决方案的核心。它维护两个滚动量：当前见过的最大值 m_j 和经过移位的指数和 ℓ_j 。每次读入一个新的 K, V 块时，用新块的局部最大值更新 m_j ，同时用比例因子 $e^{m_{j-1}-m_j}$ 修正旧的指数和，保证数值等价于在所有块上做完整 softmax。这个递推的精妙之处在于：它把一个需要「两遍扫描」的操作（第一遍找最大值，第二遍归一化）变成了「单遍」——代价只是维护两个标量，完全不影响最终的数学等价性。

严格

标准实现的内存访问代价

设序列长度 n ，维度 d ，标准实现的 HBM 读写量（以元素数计）：

1. 计算 $S = QK^T \in \mathbb{R}^{n \times n}$ ，写回 HBM: $\Theta(n^2)$
2. 从 HBM 读 S ，计算 softmax 得 P ，写回 HBM: $\Theta(n^2)$
3. 从 HBM 读 P ($\Theta(n^2)$) 和 V ($\Theta(nd)$)，计算 $O = PV$ ，写回

总 HBM 访问量 $\Theta(n^2 + nd)$ ，长序列下 n^2 项主导，使 attention 成为内存受限操作。

在线 softmax (online softmax) 的递推公式

设将 K, V 按行切成 T 个块，第 j 块对应局部注意力分数向量 $s_j \in \mathbb{R}^B$ (B 为块大小)。维护两个滚动统计量：

- $m_j \in \mathbb{R}$: 前 j 块中 s 分量的最大值 (用于数值稳定移位)
- $\ell_j \in \mathbb{R}$: 前 j 块经移位后的指数和

递推更新为

$$m_j = \max(m_{j-1}, \tilde{m}_j)$$

$$\ell_j = e^{m_{j-1}-m_j} \ell_{j-1} + e^{\tilde{m}_j-m_j} \tilde{\ell}_j$$

其中 $\tilde{m}_j = \max(s_j)$, $\tilde{\ell}_j = \sum_l \exp(s_{jl} - \tilde{m}_j)$ 是当前块的局部指数和。

输出的分块更新

维护运行输出 $O_j \in \mathbb{R}^{d_v}$ ，加载第 j 块时更新为

$$O_j = \text{diag}(\ell_j)^{-1} \left(\text{diag}(\ell_{j-1}) e^{m_{j-1}-m_j} O_{j-1} + e^{\tilde{m}_j-m_j} \tilde{P}_j V_j \right)$$

其中 $\tilde{P}_j = \exp(s_j - \tilde{m}_j \mathbf{1})$ 是当前块的未归一化注意力权重， V_j 是对应的 Value 块。可以验证，遍历所有块后 O_T 恰好等于完整的归一化注意力输出。

复杂度分析

量	标准 Attention	FlashAttention
FLOPs	$\Theta(n^2 d)$	$\Theta(n^2 d)$ (不变)
HBM 读写 (元素数)	$\Theta(n^2 + nd)$	$\Theta(n^2 d/M)$ (M 为 SRAM 大小)
需显式存储 $n \times n$ 矩阵	是	否
额外显存开销 (除 O 外)	$\Theta(n^2)$	$\Theta(n)$ (仅存 m_j, ℓ_j)

FlashAttention 将内存复杂度从 $O(n^2)$ 降至 $O(n)$ (按辅助统计量计)，HBM 带宽使用减少 4–16 倍，使 attention 从内存受限转变为计算受限，GPU Tensor Core 的利用率大幅提升。FLOPs 仍为 $O(n^2 d)$ ，即注意力的算力复杂度没有改变，改变的只是其对硬件内存层级的友好度。

与原书呼应：原书 §3.5 给出了在线 softmax 的递推公式及 HBM 访问复杂度分析，对应本节的逐步推导与复杂度对比表。

§3.6 注意力的真实身份：关联记忆，而非「注意力」

这一节在讲什么

为什么「注意力」这个名字有误导性？ 认知心理学的注意力是有限资源的主动分配；而 softmax attention 在数学上更接近「内容寻址的关联记忆检索」——给定查询模式，在一组存储模式中检索最接近的内容。

第一步：经典 Hopfield 网络的能量函数。 经典版本（1982）用二次能量函数存储模式，存储容量约 $0.14d$ ，线性于维度。

第二步：现代 Hopfield 网络的升级。 将能量函数改为 log-sum-exp 形式，求不动点条件得到更新规则 $\xi^{\text{new}} = X^T \text{softmax}(\beta X \xi)$ ——这与 softmax attention 公式在数学上完全等价（令 $\xi = Q_i$, $X = K$, $\beta = 1/\sqrt{d_k}$ ）。

第三步：存储容量从线性变指数。 现代 Hopfield 网络的容量约为 $O(\exp(\alpha^2 d/2))$ ，指数级于维度。这解释了为何 Transformer 能以有限参数存储和检索海量知识。

本节要拿走的一件事： softmax attention 是现代 Hopfield 网络一步不动点迭代的精确数学等价——「注意力」的本质是指数容量关联记忆的检索，而非心理学意义上的资源竞争。

直觉

「注意力」这个名字容易让人联想到认知心理学中的注意力——有限资源的主动分配。但从数学上看，softmax attention 更接近一种**内容寻址的关联记忆**（content-addressable associative memory）检索：给定一个查询模式，在一个存储了大量模式的系统中检索最接近的内容。现代 Hopfield 网络（modern Hopfield network）的更新规则与 softmax attention 公式在数学上完全等价，而现代 Hopfield 网络的存储容量是维度的**指数级**，远超经典版本的线性容量。

这一等价性不是巧合，而是揭示了注意力机制的真实计算原语。经典 Hopfield 网络用二次能量函数存储离散模式，容量受限于 $0.14d$ （维度的线性函数）；当存储模式数超过这个阈值，检索就会出现「串扰」错误。现代 Hopfield 网络把能量函数换成 log-sum-exp 形式，等价于在存储模式上做指数加权的超叠加——每次不动点更新等价于一次 softmax 加权平均。这让容量从 $O(d)$ 跃升至 $O(\exp(\alpha^2 d/2))$ ，指数级于维度。Transformer 的参数矩阵 W^K 的行向量，从这个视角看，就是关联记忆的「存储库」；推理时的每次注意力计算，是对这个指数容量记忆库的一次软检索。

理解这个等价性还能解释一个常见的困惑：为什么 Transformer 能在参数量相对有限的情况下「记住」如此多的事实性知识？答案在于：高维空间中的指数容量。一个 $d = 128$ 的 key 维度对应的存储容量可以达到 $e^{8192\alpha^2}$ （ α 取决于模式间的最小分离距离），远超任何线性计数的直觉估计。当然，这是上界，实际可用容量取决于模式的相关性和训练动态——但它说明了为什么更大的模型（更大的 d ）不仅仅是「更多参数」，而是在指数意义上具有更强的记忆能力。

严格

经典 Hopfield 网络的能量函数

经典 Hopfield 网络（1982）的能量函数为

$$E_{\text{classic}} = -\frac{1}{2} \xi^T W \xi, \quad W = X^T X - N I$$

其中 $\xi \in \{-1, +1\}^d$ 为查询状态， $X \in \mathbb{R}^{N \times d}$ 为存储模式，存储容量约为 $0.14d$ （线性于维度）。

现代 Hopfield 网络的能量函数

Ramsauer 等 (ICLR 2021) 将能量函数改为

$$E = -\text{lse}(\beta, X\xi) + \frac{1}{2}\xi^T\xi + \frac{1}{2\beta}\log N + C$$

其中 $\text{lse}(\beta, z) = \beta^{-1} \log \sum_i \exp(\beta z_i)$ 是 log-sum-exp 算子, $\beta > 0$ 是逆温度参数, C 是与 ξ 无关的常数。对此能量函数求不动点条件 $\partial E / \partial \xi = 0$, 得到更新规则

$$\xi^{\text{new}} = X^T \text{softmax}(\beta X\xi)$$

与 softmax Attention 的等价性

令 $\xi \leftarrow Q_i$ (某行查询向量), $X \leftarrow K$ (键矩阵的行向量集合), $\beta = 1/\sqrt{d_k}$, 则

$$\xi^{\text{new}} = K^T \text{softmax}\left(\frac{KQ_i^T}{\sqrt{d_k}}\right)$$

这恰好是对单个 query Q_i 的 attention 输出 (忽略 Key 与 Value 不同这一细节; 原始论文中取 $K = V^T$ 的情形即精确等价)。因此, 一次 softmax attention 运算在数学上等价于现代 Hopfield 网络以 $\beta = 1/\sqrt{d_k}$ 做一步不动点迭代。

存储容量的提升

现代 Hopfield 网络的存储容量从经典的 $O(d)$ 提升至

$$C_{\text{modern}} = O\left(\exp\left(\frac{\alpha^2 d}{2}\right)\right)$$

(α 为模式间最小分离距离的某个函数), 即指数级于维度。这解释了为何大型 Transformer 能以有限参数量存储和检索海量知识模式: 权重矩阵 W^K 的行向量构成一个指数容量的关联记忆索引, 推理时的「理解」本质是高维模式检索的级联。

与原书呼应: 原书 §3.6 引用了 Ramsauer 等的等价性证明, 对应本节能量函数推导与不动点更新规则的显式对应。

§3.7 一个被忽略的争论：注意力权重 解释

这一节在讲什么

为什么不能用注意力权重解释模型？ 因为注意力权重 α_{ij} 只是计算图中的一个中间量——它衡量的是 Value 向量的线性组合系数，而非输入 token 对最终预测的「因果贡献」。

第一步：权重衡量的是什么。 α_j 告诉你「第 j 个 Value 向量被赋予了多少权重」，但 Value 向量本身已经经过线性投影，与原始 token x_j 不是同一件事。

第二步：梯度归因走的是不同路径。 x_j 对输出 y 的梯度，不仅经过 Value 分支，还通过 x_j 对 q 、 k_j 、甚至其他位置 k_l 的影响传播，路径复杂得多。

第三步：Jain & Wallace 的反例。 保持输出 y 近似不变的前提下，可以构造与原始分布 ℓ^1 距离很大的替代注意力分布 α' ——当 Value 矩阵行向量相关性高时，不同权重混合可得到相似输出。

本节要拿走的一件事： α_j 大不等于「第 j 个 token 对预测起决定性作用」；真正的因果归因必须通过梯度方法或激活路径分析（circuit analysis）进行。

直觉

直觉上很容易把注意力权重 α_{ij} （即 softmax 输出）理解为「模型因为关注了第 j 个词，所以预测受到它影响」。但从数学上，注意力权重只是计算图中的一个中间量，它对最终输出的实际影响必须通过梯度来量化，而梯度归因与注意力权重并不等价，甚至可以构造出二者大相径庭的反例。

要理解这一不等价性，需要追踪信息流的实际路径。注意力权重 α_j 控制的是 V 矩阵第 j 行在输出中的混合比例——但 V 已经是原始输入 x_j 经过线性投影 W^V 变换后的结果，它与原始 token 的语义已经发生了变换。更重要的是， x_j 对输出 y 的影响不止一条路径： x_j 同时参与了 q （通过 W^Q ）、 k_j （通过 W^K ）、和 v_j （通过 W^V ）的计算，而 k_j 的变化还会影响所有其他位置对第 j 个 token 的注意力权重。梯度归因把所有这些路径都计入在内，而 α_j 只是其中一条路径（Value 分支）的系数。二者的差距，就是「看见了多少」与「影响了多少」之间的根本区别。

Jain 和 Wallace 的反例给出了一个更强的否定结论：不仅梯度归因与注意力权重不一致，甚至注意力分布本身对输出都不具有唯一决定性。当 Value 矩阵的行向量之间高度相关时（实践中这很常见，因为 $V = XW^V$ 中的 X 本身就是相关的 token 嵌入），不同的权重混合可以得到几乎相同的输出向量。这意味着：即使把注意力权重完全打乱成另一种分布，只要 Value 矩阵的列空间结构允许，输出照样可以保持不变。从「模型在看哪里」到「模型的预测取决于什么」，这中间隔着一道无法逾越的鸿沟。

严格

设单层注意力的输出为

$$y = \sum_{j=1}^n \alpha_j v_j, \quad \alpha_j = \frac{\exp(q \cdot k_j / \sqrt{d_k})}{\sum_l \exp(q \cdot k_l / \sqrt{d_k})}$$

注意力权重与梯度归因的不一致性

基于梯度的特征重要性度量定义为

$$g_j = \left\| \frac{\partial \mathcal{L}}{\partial x_j} \right\| \quad \text{或} \quad g_j = \left\| \frac{\partial y}{\partial x_j} \right\|$$

α_j 衡量的是 v_j (Value 向量) 对 y 的**线性组合权重**，而 g_j 衡量的是输入 x_j 对输出 y (或损失 $\mathcal{L}$) 的**微分敏感度**。二者通过不同路径传播： α_j 只经由 Value 分支，而 g_j 还包含 x_j 对 q 、 k_j 乃至其他位置 k_i 的影响，路径更复杂。

可交换性的反例构造

Jain 和 Wallace (NAACL 2019) 指出，可以在保持输出近似不变的前提下构造替代注意力分布 α' ，使得 $\|\alpha' - \alpha\|_1$ 较大。形式化地：对给定输出 $y = \alpha^T V$ ，存在 $\alpha' \in \Delta^{n-1}$ (概率单纯形) 使得

$$\|\alpha' - \alpha\|_1 \geq \epsilon \quad \text{而} \quad \|y' - y\| = \|\alpha'^T V - \alpha^T V\| \leq \delta$$

对足够小的 δ 和足够大的 ϵ ，这说明注意力分布本身对输出不具有唯一决定性——当 Value 矩阵 V 的行向量之间相关性高时，不同的权重混合可以得到接近相同的输出向量。因此， α_j 大不能直接推断「第 j 个 token 对预测起决定性作用」；真正的因果归因必须通过梯度方法 (gradient attribution) 或激活路径分析 (如 circuit analysis) 来进行。

与原书呼应：原书 §3.7 引用了 Jain & Wallace 的核心发现及 induction head 电路分析，对应本节注意力权重与梯度归因不等价的形式化说明。

§3.8 跨界映射：宏观工业供应链

这一节在讲什么

为什么用供应链类比 attention? 注意力机制的「匹配-归一化-加权汇总」流程，在逻辑结构上与全球供应链的「需求发布-能力匹配-资源调度」完全对应，能把抽象矩阵运算还原为可感知的经济直觉。

第一步：Query 是需求方，Key 是供应商标签，Value 是实际物料。 每个生产环节 (token) 发出需求向量 Q ，每个供应商 (token) 公布能力标签 K ，softmax 将匹配分数归一化为调度权重，最终按权重汇总实际物料 V 。

第二步：多头 = 多条并行供应链。 每个注意力头按不同逻辑 (语法、语义、话题) 独立调度，就像多条并行运作的专业供应链，最后在「总装车间」(输出投影 W^O) 合并。

第三步：这个类比的值是量级直觉。 当序列长度 $n = 8192$ ， $h = 32$ 个头，每步注意力相当于 32 条供应链各自对 8192 个供应商做一轮全局询价——这是人类供应链系统完全无法实现的规模。

本节要拿走的一件事：供应链类比把 attention 从「心理学隐喻」还原为「可量化的资源匹配」，但它有根本性的局限——见 §3.9。

直觉

注意力机制有一个简洁的工业类比：全球供应链的动态调度。每个生产环节 (Query) 发出需求，每个供应商 (Key) 公布能力标签，softmax 将匹配度归一化为调度权重，最终按权重汇总实际物料 (Value)。多头注意力则对应多条并行运作的独立供应链，各自按不同逻辑 (语法、语义、话题) 调度，最后在

总装车间合并。这个比喻把注意力从拟人化的「心理过程」还原为可量化的资源匹配，帮助建立对矩阵运算规模的直觉感受。

这个类比在建立「规模直觉」上特别有用。一个现代供应链管理系统可能跟踪数千个供应商，但每次采购决策往往只涉及少数几个候选供应商的详细比较。注意力机制的不同之处在于：每个 Query 向量每次都对所有 n 个 Key 进行全局评分和归一化——在序列长度 $n = 8192$ 的场景下，这相当于每个 token 同时对 8191 个「供应商」进行实时询价。没有任何现实供应链能以这种规模并行运作，但对 GPU 来说这只是一次 $n \times d_k$ 的矩阵乘法，成本是确定的、可分析的。这种「规模可达但人类无法直觉感知」的鸿沟，正是深度学习系统与人类设计的信息处理系统之间最根本的差异。

多头设计对应多条专业化供应链的分工协作。不同的注意力头会在训练中自发专门化：有的头更关注局部语法关系（相邻 token 之间的依存），有的头跨越长距离追踪指代（代词和它指代的名词），有的头捕捉词义的近义词聚类。这种分工不是人为规定的，而是反向传播在参数空间中自然涌现的结果——就像现实中的专业供应商分工是由市场竞争而非中央计划决定的。但请注意，这个类比在两处根本性地失效，详见 §3.9。

§3.9 反类比：当供应链隐喻失效

这一节在讲什么

为什么供应链类比有根本性局限？ 供应链有两个注意力机制没有的核心特征：硬性产能约束和明确的优化目标。这两处差异揭示了 attention 作为数学对象的真实本质。

第一步：Value 向量可被无限次「调用」，没有产能耗尽。 同一个 token 可以同时以最高权重响应序列中所有其他位置，不存在资源争抢。attention 调度的实质是相似度计算，而非真正的权衡分配。

第二步：注意力头没有明确的语义目标。 供应链的优化目标（成本、时效）是可测量的单一指标；而单个注意力头的「功能」是反向传播在高维参数空间中塑造的中间张量，是涌现的、上下文相关的，通常无法用自然语言直接描述。

第三步：attention 数学透明但语义不透明。 注意力权重可以精确计算，但它们不等于解释（见 §3.7）；attention 机制可以精确描述，但单个头的「功能」不能被先验地规定。

本节要拿走的一件事： 好的类比帮助建立直觉，坏的类比固化误解——供应链隐喻的价值在于规模感知，其局限在于它用「有约束的竞争分配」来描述一个「无约束的相似度加权」，会系统性地误导对 attention 瓶颈和可解释性的判断。

直觉

供应链类比在两处根本性地失效。其一：供应链有硬性产能约束，而 attention 中的 Value 向量可被无限次、无竞争地「调用」——同一 token 可同时以最高权重响应序列中所有其他位置，不存在资源争抢，调度的实质只是相似度计算，而非真正的权衡分配。其二：供应链的优化目标（成本、时效）是可测量的单一指标，而单个注意力头的输出没有明确语义目标——它是反向传播在高维参数空间中塑造出的中间张量，其「功能」是涌现的、上下文相关的，且通常无法用自然语言直接描述。这两处失效共同说明：注意力是数学上透明的，但语义上不透明；可精确计算，但难以直接解释。

第一处失效有一个深刻的数学含义。供应链资源的有限性是「分配」语义的来源——正因为物料有限，才需要「权衡」把资源给谁。而 attention 的 softmax 归一化只是确保权重之和为 1，这是一个数学约

束（概率单纯形），而非物理约束（守恒律）。被「高度关注」的 token 并没有减少其他位置访问它的能力，它的 Value 向量可以同时以满权重贡献给序列中的每一个其他位置。这意味着 softmax 的「归一化」不应被理解为「资源的竞争分配」，而应理解为「相似度分数的标准化」——把任意实数映射到概率空间，便于后续的加权平均。用供应链比喻会系统地高估 attention 中「竞争」的程度，并低估单个 token 的信息被复用的广泛程度。

第二处失效则触及可解释性研究的核心困难。在供应链中，每个决策单元的目标函数是外生给定的（成本最小化、时效最优化），可以事后审计每个决策是否合理。而在 Transformer 中，注意力头的「功能」完全由训练数据和损失函数的联合作用决定，没有任何先验的语义规范。机械可解释性（mechanistic interpretability）研究确实发现了一些可命名的头（如「induction head」、「name mover head」），但这些是少数特例，且即使命名了功能，头的激活模式仍然是高维张量，没有对应到供应链语境中那种可直接审计的「调度决策」。类比的局限，正是它无法告诉你一个注意力头「应该做什么」——它只能事后告诉你它「倾向于做什么」。

数学卷·第4章：非凸优化与超参数标度律

§4.1 损失函数：如何用数学定义“机器犯错”

这一节在讲什么

为什么训练前要先定义「犯错」？梯度下降需要一个可微分的数值目标，而「预测对了没有」是离散的，无法直接优化。损失函数就是把「犯错程度」翻译成一个光滑的实数。

第一步：选择什么样的损失？ 分类问题中最自然的损失是交叉熵——它与最大似然估计（MLE）完全等价，同时也等价于最小化模型分布与真实分布之间的 KL 散度，具有坚实的统计学基础。

第二步：为什么交叉熵的梯度如此简洁？ 把 Softmax 与交叉熵联合求导，链式法则中的复杂 Jacobian 自动约分，最终反向传播到 logit 层的误差信号恰好等于「预测概率 - 真实标签」，无需额外计算。

第三步：认清损失函数的三条等价链。 MLE = 最小负对数似然 = 最小 KL 散度，三者只是同一件事的三种视角。

本节要拿走的一件事： 交叉熵不是随手选的损失，它是信息论、统计学与反向传播三条路径共同指向的唯一自然选择。

直觉

训练一个神经网络，本质是找到一组参数，使模型的输出尽可能贴近真实标签。要做到这一点，必须先把“犯错程度”量化成一个数——这个数就是损失函数（loss function）。交叉熵（cross-entropy）损失之所以被普遍采用，不是因为它直觉上最自然，而是因为它与最大似然估计等价，具有坚实的统计学基础。理解这条等价链，是理解为什么“预测概率 vs 正确标签”的对数差距就是最合理的犯错度量的关键。

交叉熵之所以从众多候选损失中脱颖而出，还有一个实用理由：它的梯度形式极其简洁。把 Softmax 层和交叉熵联合处理，经过链式法则化简后，反向传播到最后一层 logit 的误差信号恰好是 $p_i - y_i$ ——预测概率与真实标签之差。这个结果不需要单独计算 Softmax 的雅可比矩阵，两者的耦合反而自动抵消了复杂性。这种数学上的「优雅约分」是交叉熵被工程师大规模采用的重要原因之一，而不只

是统计学上的理论美感。

一个常见的困惑是：均方误差（MSE）能不能用于分类？答案是可以，但有代价。MSE 不具备概率分布的语义，它假设误差服从高斯分布，对于 one-hot 标签的分类任务，MSE 的梯度在预测概率趋于 0 或 1 时会变得极小（梯度消失），而交叉熵的梯度在预测错误时仍然有效。这就是为什么分类任务几乎一律选用交叉熵，而回归任务才用 MSE。

严格

经验风险 (empirical risk) 的定义如下。给定数据集 $\{(x_i, y_i)\}_{i=1}^N$ ，参数化模型 f_θ ，以及单样本损失 ℓ ，经验风险为：

$$\hat{R}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(x_i), y_i)$$

训练目标是 $\theta^* = \arg \min_{\theta} \hat{R}(\theta)$ 。

交叉熵损失的来源。对 K 分类问题，设真实标签以 one-hot 向量 $\mathbf{y} \in \{0, 1\}^K$ 表示，模型输出概率分布 $\mathbf{p} = (p_1, \dots, p_K)$ ，则交叉熵（cross-entropy）损失为：

$$L = - \sum_{k=1}^K y_k \log p_k$$

其来源可从三条等价路径理解：

1. **最大似然估计 (Maximum Likelihood Estimation, MLE)**。数据集的对数似然为 $\log \prod_i P_\theta(y_i | x_i) = \sum_i \log P_\theta(y_i | x_i)$ 。最大化似然即最小化负对数似然 (negative log-likelihood)：

$$\min_{\theta} - \frac{1}{N} \sum_{i=1}^N \log P_\theta(y_i | x_i)$$

对 one-hot 标签，这恰好等于 $\frac{1}{N} \sum_i L_i$ 。

2. **最小化 KL 散度**。设真实数据分布为 q ，模型分布为 p_θ ，KL 散度为：

$$D_{\text{KL}}(q \| p_\theta) = \sum_k q_k \log \frac{q_k}{p_k} = - \sum_k q_k \log p_k + \text{const}$$

其中常数项 $(-\sum_k q_k \log q_k)$ ，即真实分布的熵，不依赖 θ ，故最小化 KL 等价于最小化交叉熵。

三条路径 (MLE = 最小负对数似然 = 最小 KL 散度) 在数学上完全等价。

Softmax + 交叉熵的联合梯度推导。设网络最后一层的 logit 向量为 $\mathbf{z} = (z_1, \dots, z_K)$ ，归一化指数 (Softmax) 将其映射为：

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

对第 i 个 logit，计算 $\partial p_j / \partial z_i$ ：

$$\frac{\partial p_j}{\partial z_i} = p_j (\delta_{ij} - p_i)$$

其中 δ_{ij} 是 Kronecker delta。损失对 z_i 的梯度为：

$$\frac{\partial L}{\partial z_i} = - \sum_k y_k \frac{\partial \log p_k}{\partial z_i} = - \sum_k y_k \frac{1}{p_k} \cdot \frac{\partial p_k}{\partial z_i}$$

代入上式，利用 $\sum_k y_k = 1$ (one-hot)：

$$\frac{\partial L}{\partial z_i} = - \sum_k y_k (\delta_{ki} - p_i) = -(y_i - p_i) = p_i - y_i$$

这是一个极其简洁的结果：反向传播到最后一层 logit 的误差信号，恰好等于预测概率与真实标签之差。不需要单独计算 Softmax 的雅可比 (Jacobian)，两者的链式法则自动约分。

与原书呼应：原书 §4.1 指出交叉熵”把训练目标和统计学中的最大似然估计无缝衔接”，本节给出了这条等价链的完整代数路径。

§4.2 梯度下降：在黑暗中沿着最陡的方向下山

这一节在讲什么

为什么梯度反向就是下降最快的方向？这来自一阶 Taylor 展开的几何含义：在单位球约束下，最大化损失减少量的方向恰好与梯度方向相反。

第一步：步长不能任意大。函数曲率 (Lipschitz 常数 L) 给出步长上界 $\eta < 2/L$ ，超出这个界限，更新会越过极小值导致震荡甚至发散。

第二步：mini-batch 引入的随机噪声有双重作用。它让每步计算变快，而且噪声本身在高维优化中有隐式正则化效果——这在 §4.3 中会看到它帮助逃离鞍点。

第三步：动量和 Adam 各解决一个问题。动量用历史梯度的指数加权平均提供惯性，帮助穿越平坦区域；Adam 用二阶矩自适应调整每个参数的有效步长，使各维度更新更均匀。

本节要拿走的一件事：Adam 的偏差修正不是工程技巧，而是初始化为零导致的系统偏差的数学必然修复。

直觉

在一个高维空间中找函数的极小值，最自然的思路是：每一步都沿着函数下降最快的方向走一小步。梯度反向就是下降最快的方向，这来自一阶泰勒 (Taylor) 展开的几何含义。步长 (学习率) 不能任意大——太大会越过极小值，甚至发散；适当的上界由函数的曲率 (李普希茨 (Lipschitz) 常数) 决定。实践中用 mini-batch 近似梯度引入的噪声，在高维优化中反而有正则化效果。动量法和 Adam 是对纯梯度下降的两类重要改进，各有清晰的数学动机。

动量法的本质是给优化器加上「惯性」。想象一个小球在损失景观上滚动：纯梯度下降在平坦区域会极度缓慢，因为梯度接近零；而有动量的小球会利用历史速度继续前行，穿越平坦区域。动量还有抑制振荡的效果：在一个狭长的峡谷形损失景观中，梯度在峡谷横截方向上频繁改变符号，动量会让横向分量互相抵消，纵向分量则累积加速，最终沿峡谷方向加速收敛。

Adam 的二阶矩估计解决的是另一个问题：不同参数的梯度幅值可能差异极大 (有的参数对损失非常敏感，有的几乎无影响)，纯梯度下降用同一个学习率处理所有参数，效率低下。Adam 的 $\sqrt{\hat{v}_t}$ 相当

于对每个参数做了「历史梯度幅值的归一化」，让稀疏梯度的参数（如词嵌入中罕见词对应的权重）也能获得有效更新。初期的偏差修正 $1/(1-\beta^t)$ 则是因为动量从零初始化，早期估计系统性偏小，修正项让前几步的更新量恢复到正常尺度。

严格

一阶 Taylor 展开导出更新规则。在 θ_t 处展开：

$$L(\theta_t + \Delta\theta) \approx L(\theta_t) + \nabla L(\theta_t)^\top \Delta\theta$$

在约束 $\|\Delta\theta\| = \epsilon$ 下，使 L 下降最多的方向是 $\Delta\theta = -\epsilon \frac{\nabla L(\theta_t)}{\|\nabla L(\theta_t)\|}$ 。令步长 $\eta = \epsilon/\|\nabla L\|$ ，得到梯度下降（gradient descent）更新规则：

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

步长上界与 Lipschitz 常数。若 L 的梯度是 L -Lipschitz 连续的，即：

$$\|\nabla L(\theta) - \nabla L(\theta')\| \leq L\|\theta - \theta'\|, \quad \forall \theta, \theta'$$

则可证明当 $\eta < 2/L$ 时，梯度下降保证每步损失不增（对凸函数则保证单调收敛）。直觉： L 是曲率的上界，步长过大会“跨过”极小值导致震荡。

SGD 的方差与 mini-batch。随机梯度下降（Stochastic Gradient Descent, SGD）用 mini-batch $B \subset \{1, \dots, N\}$ 估计梯度：

$$\tilde{g}_t = \frac{1}{|B|} \sum_{i \in B} \nabla \ell(f_\theta(x_i), y_i)$$

$\tilde{g}_t$ 是真实梯度 $\nabla L(\theta_t)$ 的无偏估计。其方差为：

$$\text{Var}(\tilde{g}_t) = \frac{1}{|B|} \text{Var}(\nabla \ell_i)$$

方差随 batch size $|B|$ 增大而线性减小。更大的 batch 更准确，但引入的随机噪声也更少——这一噪声在 §4.3 中将看到有正面意义。

动量法 (momentum)。动量法引入速度向量 v_t ，模拟质点在损失景观中的惯性运动：

$$v_{t+1} = \mu v_t + \nabla L(\theta_t), \quad \theta_{t+1} = \theta_t - \eta v_{t+1}$$

其中 $\mu \in [0, 1)$ 是动量系数。展开递推可得：

$$v_t = \sum_{k=0}^t \mu^k \nabla L(\theta_{t-k})$$

即当前速度是历史梯度的指数加权和。物理类比：质点在黏性介质中滚动——历史方向提供惯性，有助于穿越平坦区域、抑制振荡方向的震荡。

Adam 的二阶矩估计与偏差修正。Adam (Adaptive Moment Estimation) 同时维护一阶矩 m_t 和二阶矩 v_t ：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

其中 $g_t = \nabla L(\theta_t)$ ，逐元素平方。初始时 $m_0 = v_0 = 0$ ，导致早期估计偏小（偏向零）。偏差修正（bias correction）为：

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

最终更新：

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

核心思想：用 $\hat{v}_t$ 自适应地缩放每个参数的有效学习率——历史梯度幅值大的参数步长缩小，历史梯度幅值小的参数步长放大，使各参数的更新幅度更均匀。典型超参数： $\beta_1 = 0.9$ ， $\beta_2 = 0.999$ ， $\epsilon = 10^{-8}$ 。

与原书呼应：原书 §4.2 给出了 Adam 的更新规则并描述其为“为每个参数自适应地调整学习率”，本节完整推导了一阶 Taylor 展开来源、步长上界约束，以及偏差修正的数学必要性。

§4.3 高维非凸优化的反直觉：为什么 SGD 居然能工作

这一节在讲什么

为什么高维非凸优化看起来不可能，却在实践中奏效？经典理论告诉我们梯度下降会陷入局部极小值，但真正的障碍根本不是局部极小值——而是鞍点。

第一步：高维空间里局部极小值极为罕见。随机矩阵理论表明，一个临界点是纯局部极小（Hessian 全正定）的概率随维度 d 指数衰减，参数量达 10^8 时几乎为零。

第二步：SGD 的噪声恰好能逃离鞍点。噪声协方差矩阵 $\Sigma \approx \frac{\eta}{b} C(\theta)$ 在鞍点附近沿负曲率方向提供随机扰动，batch size 越小、学习率越大，逃逸越快。

第三步：极小值之间存在低损失连通路径。不同初始化训练出的解不是孤立点，而是一个连通流形——SGD 从几乎任何起点出发，都能漂流到这个流形上。

本节要拿走的一件事：SGD 能工作不是「运气好」，而是高维几何的必然——维度本身消灭了局部极小值的威胁。

直觉

神经网络的损失函数是高维非凸的，经典优化理论预言梯度下降几乎必然陷入局部极小值。但现实中，从任意初始化出发训练大模型都能得到好解。这个矛盾的答案藏在高维几何里：高维空间中局部极小值极为稀少，鞍点才是主要障碍；而 SGD 的噪声恰好有能力逃离鞍点；此外，极小值并非孤立，而是构成连通的流形。

要理解为什么局部极小值如此罕见，可以做一个思想实验：一个临界点要成为局部极小，它在所有方向上的曲率都必须为正。在二维情况下这并不苛刻，但在 10^8 维空间中，每一个方向的曲率都恰好为

正的概率是一个指数衰减的小数——如同要求 10^8 枚硬币全部正面朝上。随机矩阵理论给出了这个概率的精确估计：它随维度指数衰减。换句话说，高维空间给了「逃跑」无数条出路，任何不是全局最优的地方，几乎必然存在某个方向可以继续下坡。

鞍点虽多，却并不是无解的障碍。在鞍点附近，函数在某些方向上是下坡的，SGD 的随机噪声提供了一个「随机搜索」机制：只要噪声协方差在负曲率方向有非零投影，模型就会以一定概率向下滑走。这里一个微妙的地方是：小 batch size 和大学习率都会增大 Σ 的尺度，从而加速鞍点逃逸——这就是为什么「小 batch 训练更快收敛」有时在实践中成立，即使每步的梯度估计更嘈杂。

严格

鞍点主导：随机矩阵理论的结论。 设损失函数在临界点 θ^* 处的 Hessian 矩阵（海森矩阵）为 $H = \nabla^2 L(\theta^*) \in \mathbb{R}^{d \times d}$ 。局部极小值要求 H 正定（所有特征值 > 0 ），鞍点（saddle point）则要求 H 至少有一个负特征值。

Dauphin 等人（NIPS 2014）借助随机矩阵理论（random matrix theory）中的 Bray-Dean 计算，对高维随机非凸函数得出以下结论：在 d 维空间中，临界点处 Hessian 的负特征值比例 $\rho \in [0, 1]$ 决定其类型—— $\rho = 0$ 对应局部极小， $\rho > 0$ 对应鞍点。纯局部极小（ $\rho = 0$ ）的数量占总临界点数的比例随维度 d 指数衰减：

$$\Pr[\text{纯局部极小}] \propto e^{-\Omega(d)}$$

这意味着在 $d \sim 10^8$ 量级的参数空间中，SGD 几乎不会遇到局部极小值——它面对的几乎全是鞍点。

Hessian 谱的半圆律（semicircle law）。 在高维随机设定下，随机对称矩阵的特征值经验分布收敛到 Wigner 半圆分布：

$$\rho(\lambda) = \frac{1}{2\pi\sigma^2} \sqrt{4\sigma^2 - \lambda^2}, \quad |\lambda| \leq 2\sigma$$

深度网络损失 Hessian 的谱在过参数化区域近似符合此规律（带有一部分大特征值的“bulk + spike”结构），说明 Hessian 的负特征值占主体，纯局部极小在谱意义上极为罕见。

SGD 噪声协方差与鞍点逃逸。 SGD 每步的梯度噪声向量 $\xi_t = \tilde{g}_t - \nabla L(\theta_t)$ 的协方差矩阵近似为：

$$\Sigma(\theta) \approx \frac{\eta}{B} \cdot C(\theta)$$

其中 B 是 batch size, $C(\theta) = \frac{1}{N} \sum_i \nabla \ell_i \nabla \ell_i^\top - \nabla L \nabla L^\top$ 是样本梯度的协方差矩阵。在鞍点附近， $C(\theta)$ 的特征向量与 Hessian 负曲率方向有重叠， Σ 提供了沿负曲率方向的随机扰动，使模型以 $O(\eta/B)$ 的速率逃离鞍点。减小 batch size 或增大学习率，均可加强这种逃逸能力。

模式连通性（mode connectivity）。 Garipov、Izmailov 等人（NeurIPS 2018）实验表明：两个独立训练得到的神经网络极小值 θ_A, θ_B ，可以用一条损失值始终低的曲线 $\phi: [0, 1] \rightarrow \mathbb{R}^d$ 连接，即 $L(\phi(t)) \leq \max(L(\theta_A), L(\theta_B)) + \delta$ 对所有 t 成立， δ 很小。这说明极小值不是孤立的点，而是连通流形（connected manifold）的一部分，SGD 从几乎任何初始化出发均可漂到其中某处。

与原书呼应：原书 §4.3 引用 Dauphin et al. 2014 的随机矩阵论据，并描述了 SGD 噪声的隐式正则化效应，本节给出了鞍点主导的指数估计、谱分布形式，以及噪声协方差 $\Sigma \approx \frac{\eta}{B} C(\theta)$ 的结构。

§4.4 损失景观的可视化：黑暗中的地形图

这一节在讲什么

为什么要可视化损失景观？ 高维函数不能直接画出来，但沿随机二维切面采样可以揭示架构选择对优化难度的影响——这不只是好看的图，而是有可量化意义的曲率信息。

第一步：filter-wise 归一化消除尺度干扰。 不归一化时，不同 filter 的参数尺度差异会主导切面图，掩盖真实曲率；按 filter 归一化后，才能公平比较不同架构的景观平坦度。

第二步：平坦极小值和泛化能力有可证明的关联。 Hessian 谱半径 $\lambda_{\max}$ 越小，泛化上界越紧——小 batch SGD 倾向于找到这类平坦解，大 batch SGD 则常陷入尖锐极小值。

第三步：SAM 把「找平坦解」显式化为优化目标。 它最小化参数邻域内最坏情况的损失，强制解落在平坦区域，代价是每步需要两次前向-反向传播。

本节要拿走的一件事： 损失景观的「平坦度」不是审美概念，而是泛化界中可以量化的几何量，skip connection 是让景观变平坦的最简单结构改动。

直觉

高维损失函数无法直接可视化，但可以沿随机选取的二维切面观察其形状。这种切面图揭示了架构选择对“损失景观平坦度”的影响：有 skip connection 的网络损失景观宽阔平滑，而无 skip connection 的网络则陡峭混乱。平坦极小值（flat minima）不仅好训练，还与更好的泛化能力有数学上可证明的联系。

Skip connection 为什么能让景观变平坦？直觉是：残差连接提供了梯度的「高速公路」，使得反向传播的梯度信号不必穿越所有非线性层，损失函数对参数的依赖变得更加平滑。没有残差连接的深层网络，其损失在参数空间里会有许多陡峭的峡谷和尖峰，这些尖峰对应着特定层参数的微小变化导致整体输出剧烈变化的情况。可视化的意义在于把这种直觉量化：通过 Hessian 最大特征值 $\lambda_{\max}$ ，可以数值地比较不同架构、不同 batch size 下解的「尖锐程度」。

一个容易被忽视的细节是 filter-wise 归一化的必要性。如果不做归一化，直接用随机方向在参数空间画切面，会发现不同层的权重尺度差异极大（有的层范数是另一层的百倍），这会让切面图完全失去几何意义——你看到的不是真实曲率，而是参数尺度的差异。filter-wise 归一化把每个方向向量的每个 filter 分量缩放到与对应参数 filter 等范数，确保切面图反映的是真实的曲率信息，而不是人为的尺度效应。

严格

Filter-wise normalization 切片方法。 Li 等人（NeurIPS 2018）的可视化方案：选取两个随机方向 $d_1, d_2 \in \mathbb{R}^P$ ，对每个方向按 filter 归一化（filter-wise normalization）——将 d_1 的每个 filter 缩放使其与 θ^* 中对应 filter 的 Frobenius 范数相等，消除参数尺度差异。随后在二维坐标 (α, β) 上绘制：

$$L(\alpha, \beta) = L(\theta^* + \alpha d_1 + \beta d_2)$$

不做 filter-wise normalization 时，方向尺度与参数尺度不匹配，可视化会被参数量级的差异所主导，无法反映真实曲率。

平坦极小值 vs 尖锐极小值 (flat vs sharp minima)。 设极小值处 Hessian 的最大特征值（谱半径）为 $\lambda_{\max}$ 。Keskar 等人（ICLR 2017）的实验表明：

- 大 batch SGD 收敛到尖锐极小值 (large $\lambda_{\max}$)，测试集性能差；
- 小 batch SGD 收敛到平坦极小值 (small $\lambda_{\max}$)，测试集性能好。

泛化界 (generalization bound)。对平坦极小值，存在如下形式的泛化上界：

$$R(\theta) \leq \hat{R}(\theta) + O\left(\sqrt{\frac{\lambda_{\max} \cdot n}{N}}\right)$$

其中 $R(\theta)$ 是真实风险， $\hat{R}(\theta)$ 是经验风险， n 是参数数量， N 是样本数。该界说明： $\lambda_{\max}$ 越小（极小值越平坦），泛化误差上界越紧，模型在未见数据上的性能越有保障。直觉：平坦极小值意味着参数在其邻域内小扰动不改变模型输出，模型“不依赖特定参数值”，故泛化更好。

SAM (Sharpness-Aware Minimization)。Foret 等人 (2021) 将平坦性显式化为优化目标：

$$\min_{\theta} L^{\text{SAM}}(\theta) := \max_{\|\epsilon\| \leq \rho} L(\theta + \epsilon)$$

即在参数邻域内最坏情况的损失最小化，强制解落在平坦区域。SAM 需要两次前向-反向传播（先求 ϵ^* ，再计算更新），计算开销加倍，但泛化通常改善显著。

与原书呼应：原书 §4.4 描述了 Li et al. 2018 的可视化工作及 skip connection 对损失景观的影响，本节补充了 filter-wise normalization 的数学必要性、泛化界的具体形式，以及 SAM 的优化目标。

§4.5 P：超参数迁移的“元定律”

这一节在讲什么

为什么小模型调好的学习率不能直接用于大模型？ 标准初始化下，激活的更新量 Δh 随宽度 n 以 $O(\sqrt{n})$ 发散，最优学习率必须随宽度缩放——迁移根本不成立。

第一步：找到发散的根源。 参数更新量 $\Delta W \sim O(\eta/\sqrt{n})$ ，乘以宽度为 n 的激活求和，得到 $\Delta h \sim O(\eta\sqrt{n})$ ——宽度一翻倍，激活更新翻 $\sqrt{2}$ 倍。

第二步：P 通过层依赖缩放修复这个发散。 隐藏层学习率 (Adam 下) 保持 $O(1)$ ，输出层按 $O(1/n)$ 缩减，初始化方差也按层类型相应调整，使所有层 $\Delta h = O(1)$ 。

第三步：P 是唯一保持「特征学习」的宽度极限。 NTK 参数化下 $\Delta h \rightarrow 0$ ，模型退化为固定核方法；标准参数化下 Δh 发散；只有 P 在 $n \rightarrow \infty$ 时维持 $O(1)$ 的特征更新。

本节要拿走的一件事： P 不是调参技巧，而是唯一让超参数在宽度变化时保持稳定的代数约束。

直觉

在标准初始化下，当模型宽度增大时，各层激活的更新量随宽度发散，导致最优学习率随宽度漂移——在小模型上调好的超参数无法直接用于大模型。P (Maximal Update Parametrization) 通过精心设计初始化方差和学习率随宽度的缩放规则，保证所有层的激活更新量在宽度极限下保持 $O(1)$ ，从而使最优超参数在不同宽度下稳定不变。

为什么这个问题如此难以察觉？因为在小模型上，激活发散的效果被掩盖了：模型规模小，发散幅度尚在可接受范围内，网络勉强能训练起来，超参数搜索也能找到一个「局部最优」的学习率。但当宽度扩大 10 倍甚至 100 倍时，激活更新量随 $\sqrt{n}$ 放大，在大模型上直接套用小模型学习率会导致训练

不稳定或次优收敛。P 的工程价值正在于此：在代理模型（宽度较小、训练成本低）上做超参数搜索，然后直接零成本迁移到生产规模的大模型，省去在大模型上反复试错的巨大算力开销。

P 与 NTK 参数化之间的区别揭示了一个更深的权衡：要让宽度极限下的极限理论有意义（特征不动，可以用核方法精确分析），就必须接受网络退化为固定核，失去特征学习能力；要保留特征学习，就必须让参数在训练中有 $O(1)$ 的实质更新，而这正是 P 所保证的。P 可以看作「在保留特征学习的前提下，让超参数具有规模不变性的唯一方式」。

严格

标准参数化 (Standard Parametrization, SP) 的尺度失控。 设隐藏层宽度为 n ，权重 $W \in \mathbb{R}^{n \times n}$ 按 He 初始化 $W_{ij} \sim \mathcal{N}(0, 1/n)$ ，学习率为 η 。考察单步更新对激活的影响：

- 前向激活： $h^{(l)} = Wh^{(l-1)} \in \mathbb{R}^n$ ，方差 $\text{Var}(h_j^{(l)}) = O(1)$ （设计目的）；
- 反向梯度： $g^{(l)} = W^\top g^{(l+1)} \odot \sigma'(z^{(l)})$ ，尺度 $O(1/\sqrt{n})$ （因 $W^\top$ 的谱范数 $\sim O(1)$ 但分量求和数目为 n ，结果分量方差为 $O(1/n)$ ，尺度为 $O(1/\sqrt{n})$ ）；
- 参数更新： $\Delta W = -\eta g^{(l+1)}(h^{(l)})^\top$ ，尺度 $O(\eta/\sqrt{n})$ ；
- 激活变化： $\Delta h^{(l)} = \Delta W \cdot h^{(l-1)}$ ，尺度 $O(\eta\sqrt{n})$ ——**随宽度发散**。

为使 $\Delta h = O(1)$ ，需令 $\eta = O(1/n)$ ，即**最优学习率随宽度线性衰减**，迁移不成立。

P 的缩放规则。 Yang & Hu 在《Tensor Programs V》中证明，通过以下层依赖缩放可实现所有层 $\Delta h = O(1)$ ：

层类型	初始化方差 σ^2	SGD 学习率	Adam 学习率
输入嵌入 $V \rightarrow n$	$O(1)$	$O(1)$	$O(1)$
隐藏 $n \rightarrow n$	$O(1/n)$	$O(1/n)$	$O(1)$
输出 $n \rightarrow V$	$O(1/n^2)$	$O(1/n)$	$O(1/n)$

其中 V 为词表大小。关键点：在 Adam 下，隐藏层学习率无需随宽度衰减（因 Adam 的二阶矩归一化已吸收了梯度尺度的变化），仅输出层需要 $O(1/n)$ 的衰减。

与 SP 和 NTK 参数化的对比。 神经切线核 (Neural Tangent Kernel, NTK) 参数化是另一种极限：隐藏层初始化方差 $O(1/n)$ ，学习率 $O(1/n^2)$ ，使得整个训练过程中参数几乎不动，模型退化为固定核方法（“懒学习”，lazy regime），无法真正学习特征。P 则保持了“特征学习极限” (feature learning limit)：参数在训练中有 $O(1)$ 的实质更新，模型确实在学习新的特征表示。

$$\text{NTK} \subset \text{SP} \subset \text{P}$$

（从左到右：核极限 标准参数化 特征学习极限）

三者的本质区别在于激活变化 Δh 的尺度：NTK 下为 $O(1/n)$ ，SP 下为 $O(\sqrt{n})$ （随宽度发散），P 下为 $O(1)$ （稳定）。只有 P 在宽度 $n \rightarrow \infty$ 时保持非平凡的特征动力学。

与原书呼应：原书 §4.5 描述了 P 的超参数迁移实证效果，本节补充了尺度失控的完整推导链 ($\Delta h \sim O(\eta\sqrt{n})$) 以及 SP/NTK/P 三者的对比表与特征学习含义。

§4.6 Lion 与 Shampoo: Adam 不是终点

这一节在讲什么

为什么要挑战 Adam? Adam 需要同时存储一阶矩和二阶矩，优化器状态内存占总显存的大头；同时，Adam 对学习率和权重衰减的耦合在大规模训练中引发微妙的调参问题。

第一步：Lion 用符号代替幅值，内存减半。更新方向只取动量的 sign，每个参数每步只做 $\pm\eta$ 的跳跃，丢弃梯度幅值信息，只需维护一份动量而非两份。

第二步：Shampoo 把二阶方法用 Kronecker 分解重新带回来。维护左右两个小矩阵 $L_t \in \mathbb{R}^{m \times m}$ 和 $R_t \in \mathbb{R}^{n \times n}$ ，计算复杂度从 $O((mn)^3)$ 降到 $O(m^3 + n^3)$ ，近似精确 Gauss-Newton 预条件子。

第三步：SOAP 融合两者之长。把梯度旋转到 Kronecker 矩的特征基中再运行 Adam，兼得二阶预条件与自适应性，减少约 40% 训练步数。

本节要拿走的一件事：优化器的竞争不是「谁更聪明」，而是在内存、算力、收敛速度三者之间寻找新的帕累托前沿。

直觉

Adam 统治 LLM 训练近十年，但近年出现了两条值得关注的挑战路线。Lion 是通过符号优化搜索演化出的算法，只用梯度的符号而非幅值，内存需求减半。Shampoo 则将二阶方法用克罗内克 (Kronecker) 分解重新带入大规模训练，以可接受的计算代价换取更快的收敛。

Lion 的设计哲学与 Adam 截然不同。Adam 相信「梯度的幅值包含了关于步长的有用信息」，因此用二阶矩来自适应缩放每个参数的步长；Lion 则持相反观点：在损失景观高度各向异性的情况下，梯度幅值包含的尺度信息更多是噪声，方向（符号）才是可靠的信号。每个参数每步做等幅的 $\pm\eta$ 更新，确保各维度的更新量完全均等，避免了 Adam 中某些参数因二阶矩极小而获得超大大有效步长的问题。Lion 还有一个实用优点：只需存储一份动量 m_t ，而 Adam 需要 m_t 和 v_t 两份；在混合精度训练中，优化器状态往往占据总显存的主要部分，减少一份 fp32 动量对大规模训练意义显著。

Shampoo 代表的是完全不同的思路：与其放弃幅值信息，不如引入更精确的曲率信息。完整的二阶方法（牛顿法）需要计算和存储 $d \times d$ 的 Hessian 逆，对 $d \sim 10^9$ 的模型完全不可行；Shampoo 的聪明之处在于对每个权重矩阵单独维护左右两个低维的二阶统计，Kronecker 积结构使得计算量从参数总数的三次方降到两个小矩阵的三次方之和，同时近似质量远超对角预条件子（Adam 本质上是一个对角预条件子）。

严格

Lion (EvoLved Sign Momentum) 的更新规则。Chen 等人 (ICML 2023) 通过符号优化搜索得到：

$$\begin{aligned}c_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ \theta_{t+1} &= \theta_t - \eta \cdot \text{sign}(c_t) - \eta \lambda \theta_t \\ m_t &= \beta_2 m_{t-1} + (1 - \beta_2) g_t\end{aligned}$$

其中 $g_t = \nabla L(\theta_t)$ ， λ 是权重衰减系数， $\text{sign}(\cdot)$ 逐元素取符号。

核心区别：更新方向取 c_t 的符号，每个参数每步只做 $\pm\eta$ 的更新，完全丢弃梯度幅值信息。内存开

销：只需维护 m_t （一份动量），而 Adam 需要 m_t 和 v_t 两份，节省约 1/3 的优化器状态内存（在混合精度训练中，Adam 优化器状态约占总内存的 2/3，Lion 将其减半）。

理论动机：当损失景观高度各向异性（不同方向曲率差异巨大）时，梯度幅值包含大量尺度噪声，符号反而是更稳健的方向信号。Lion 可以看作带动量的符号随机梯度下降（SignSGD with momentum），其收敛性在一定条件下可证明。

Shampoo 的 Kronecker 分解预条件。对权重矩阵 $W \in \mathbb{R}^{m \times n}$ ，梯度 $G_t \in \mathbb{R}^{m \times n}$ ，Shampoo 维护左右二阶矩统计：

$$L_t = \sum_{s=1}^t G_s G_s^\top \in \mathbb{R}^{m \times m}, \quad R_t = \sum_{s=1}^t G_s^\top G_s \in \mathbb{R}^{n \times n}$$

更新规则为：

$$W_{t+1} = W_t - \eta \cdot L_t^{-1/4} G_t R_t^{-1/4}$$

预条件子 $P_L = L_t^{-1/4} \in \mathbb{R}^{m \times m}$ ， $P_R = R_t^{-1/4} \in \mathbb{R}^{n \times n}$ 近似了精确高斯-牛顿（Gauss-Newton）预条件子 $(L_t \otimes R_t)^{-1/2}$ 的 Kronecker 分解。计算复杂度：矩阵根 $L_t^{-1/4}$ 需要 $O(m^3)$ ， $R_t^{-1/4}$ 需要 $O(n^3)$ ，远低于精确 Hessian 逆的 $O((mn)^3)$ 。对 $m, n \sim 10^4$ 量级（变换器（Transformer）典型隐藏维度），每步预条件计算是可承受的。

SOAP（2024）在 Shampoo 基础上融合 Adam：将梯度旋转到 Kronecker 矩的特征基中，再在特征基里运行 Adam，兼得二阶预条件与 Adam 自适应性，报告相比 AdamW 减少约 40% 训练步数。

与原书呼应：原书 §4.6 描述了 Lion 的符号更新与内存优势，以及 Shampoo 的 Kronecker 分解框架和 SOAP 的融合结果，本节给出了完整的更新方程和计算复杂度分析。

§4.7 P 的完整面貌：超参数迁移背后的数学

本节的关键论证：（1）在无穷宽极限下，前向激活方差应保持 $\Theta(1)$ （**激活稳定性条件**）；（2）反向梯度方差也应保持 $\Theta(1)$ （**梯度稳定性条件**）；（3）同时满足两条 $\rightarrow$ 输入层、隐藏层、输出层的学习率必须按 $\Theta(1/n)$ 、 $\Theta(1/n)$ 、 $\Theta(1)$ 缩放（**P 缩放律**）。所有「凭空出现」的 $1/n$ 因子都来自这两条平衡条件。

这一节在讲什么

为什么需要一个理论框架来理解 P? §4.5 给出了工程规则，但没有解释为什么这些规则是唯一的，以及违反它们会发生什么——Tensor Programs 框架给出了完整的答案。

第一步：宽度极限下激活收敛到高斯过程。当 $n \rightarrow \infty$ 时，随机初始化网络的所有层间激活的经验联合分布弱收敛到一个高斯过程，这使得极限行为可以用核函数精确描述。

第二步：abc 参数化把所有选择统一到三个指数。初始化指数 a 、学习率指数 b 、输出缩放指数 c 决定了网络属于哪个极限。特征学习要求 $a + b = 1$ （隐藏层，Adam），违反则要么激活爆炸要么退化为核方法。

第三步：P 是唯一的特征学习极限。NTK 参数化下 $\Delta h \rightarrow 0$ （核极限），标准参数化下 Δh 发散，只有满足 $a + b = 1$ 的 P 在 $n \rightarrow \infty$ 时保持 $O(1)$ 的非平凡特征动力学。

本节要拿走的一件事：abc 参数化约束 $a + b = 1$ 是「超参数在宽度变化下保持稳定」的充要条件，这条等式是所有 $1/n$ 缩放因子的统一来源。

直觉

§4.5 给出了 P 的工程规则。这里深入到其理论基础：Tensor Programs 框架。其核心是：当宽度 $n \rightarrow \infty$ 时，网络各层激活的分布在某种弱收敛意义下有极限，而不同参数化方案决定了这个极限是平凡的（核极限）还是非平凡的（特征学习极限）。理解这一框架，揭示了为什么 P 是唯一“正确”的宽度极限。

Tensor Programs 框架的关键洞察是：宽度无穷大的极限并不是一个「模糊的近似」，而是一个精确可分析的数学对象。在这个极限下，网络行为完全由有限维的核函数递推决定，所有关于「激活分布是什么」的问题都有闭合形式的答案。不同参数化方案（NTK、SP、P）的区别，完全体现在这个极限下参数更新 ΔW 反馈给激活变化 Δh 的尺度上： $\Delta h \rightarrow 0$ 意味着特征冻结（核极限）， $\Delta h \rightarrow \infty$ 意味着训练不稳定（发散），只有 $\Delta h = O(1)$ 才是有意义的特征学习。

abc 参数化把所有可能的选择统一到三个指数 (a, b, c) 上，使得「是否有特征学习」变成一个简单的代数不等式。约束 $a + b = 1$ 是一个边界条件： $a + b < 1$ 对应激活爆炸， $a + b > 1$ 对应核极限，恰好等于 1 是唯一的「刀刃上的平衡点」。P 的所有缩放规则——包括那些看起来有些任意的 $1/n$ 因子——都是这个约束在不同层类型上的具体实现。理解了 $a + b = 1$ 这一条，所有细节都变得清晰而非任意。

严格

Tensor Programs 的核心思想。Yang 的 Tensor Programs 系列论文 (I-V) 建立了一套分析框架，核心结论是：在宽度 $n \rightarrow \infty$ 的极限下，随机初始化神经网络的所有层间激活（以及梯度、参数更新）的经验联合分布弱收敛（converge weakly）到一个高斯过程（Gaussian process）。具体地，对输入 $x^{(1)}, x^{(2)} \in \mathbb{R}^V$ ，第 l 层激活向量的内积 $\frac{1}{n} h^{(l)}(x^{(1)}) \cdot h^{(l)}(x^{(2)})$ 在 $n \rightarrow \infty$ 时依概率收敛到确定极限，极限的递推由核函数给出。

abc 参数化 (abc-parameterization)。对权重矩阵 $W \in \mathbb{R}^{n \times n}$ ，引入三参数族：

$$W = n^{-a} \tilde{W}, \quad \tilde{W}_{ij} \sim \mathcal{N}(0, 1)$$

学习率 $\eta = n^{-b} \tilde{\eta}$ ，输出层缩放 n^{-c} 。其中 a （初始化指数）、 b （学习率指数）、 c （输出缩放指数）是三个可选参数。

Yang & Hu 证明，要使宽度极限下存在非平凡特征学习（即 $\Delta h = O(1)$ ），三参数必须满足约束：

$$a + b = 1 \quad (\text{hidden layers, Adam})$$

具体而言，对隐藏—隐藏权重： $a = 1$ 对应初始化方差 $O(1/n)$ ， $b = 0$ 对应 Adam 学习率 $O(1)$ （不随宽度缩放）。违反此约束会导致要么激活爆炸（ $a + b < 1$ ），要么激活冻结退化为核极限（ $a + b > 1$ ）。

特征学习极限 (feature learning limit) vs 核极限 (kernel limit / NTK)。在 NTK 参数化下， $a = 1/2$ (He 初始化)， $b = 0$ (固定学习率)， $a + b = 1/2 < 1$ (对 SGD)， $\Delta W = O(\eta/\sqrt{n})$ ， $\Delta h = O(\eta) \rightarrow 0$ (宽度极限下特征不变)，网络退化为固定核方法，训练动力学由初始 NTK 完全决定，与宽度无关（不再学习新特征）。

P 下， $a = 1$ (隐藏层初始化方差 $O(1/n)$)， $b = 0$ (Adam)， $\Delta h = O(1)$ ，网络在宽度极限下仍有 $O(1)$ 的特征更新量——这是“特征学习极限”。它是唯一使特征动力学在 $n \rightarrow \infty$ 时既非爆炸也非消失的参数化，也是使超参数在宽度极限下有稳定极值的唯一参数化。

与原书呼应：原书 §4.7 描述了 Tensor Programs 框架与 abc 参数化的核心思想，以及特征学习极限与核极限的本质区别，本节给出了弱收敛的形式化陈述、约束方程 $a + b = 1$ 的来源，以及两类极限的对比。

§4.8 跨界类比：复杂系统的工程管理

这一节在讲什么

为什么用「工程管理」来类比神经网络训练？ 大模型训练涉及数以亿计的参数协同更新，每个参数的调整都影响全局，这与超大型组织的绩效分摊问题在结构上高度相似。

第一步：反向传播对应责任分摊。 链式法则把全局损失精确分解为每个参数的边际贡献，这与绩效考核系统把组织目标分摊到每个员工的逻辑同构——都是线性叠加结构。

第二步：Adam 对应个性化绩效考核。 历史梯度方差大的参数自动获得更保守的步长，这与「不确定性越大调整越保守」的管理原则一致。

第三步：P 对应规模不变的制度设计。 无论组织规模（模型宽度）如何变化，管理制度（超参数）保持有效——这是代数上的 n -均匀性，不是工程直觉。

本节要拿走的一件事：类比的目的在于提供直觉入口，但类比本身不能替代数学——下一节 (§4.9) 将指出类比的失效边界。

直觉

原书将训练大模型类比为“指挥数千万员工的超大型工程项目”：反向传播对应绩效分摊系统，学习率对应调整幅度，Adam 对应个性化绩效考核，P 对应规模不变的管理制度设计。

这个类比之所以有效，在于它抓住了神经网络训练中一个真实的结构特征：责任分摊的线性叠加性。在工程管理中，一个大项目的整体绩效可以（在线性假设下）分解为各个团队、各个员工的边际贡献之和；在反向传播中，全局损失对某个参数的梯度，通过链式法则被精确分解为该参数所在层之后所有层的贡献之积。两者的共同点是：个体的调整幅度由其对全局目标的边际影响决定，而非由中央统一指派。这种去中心化的调整机制，是梯度下降能在数十亿参数规模下高效运作的根本原因。

类比还帮助我们理解 Adam 的直觉。如果一个「员工」（参数）的历史绩效波动极大（梯度方差大），说明其表现不稳定、预测性低，管理者应该保守地调整其薪酬/职责；而长期表现稳定但贡献偏小的员工，应该给予更积极的激励（更大的有效学习率）。Adam 的 $\hat{v}_t$ 正是「历史波动性」的量化指标，自适应步长 $\eta/(\sqrt{\hat{v}_t} + \epsilon)$ 则是这种「波动越大越保守」原则的数学实现。

严格（简版）

类比的数学核心在于链式法则的线性性：每个参数的梯度 $\partial L / \partial \theta_i$ 通过链式法则被精确分解为各层贡献之和，这与责任分摊 (responsibility allocation) 的线性叠加结构同构。Adam 的自适应步长 $\eta/(\sqrt{\hat{v}_t} + \epsilon)$ 对历史梯度方差大的参数自动降低有效步长，这与「不确定性越大，调整越保守」的管理逻辑一致。P 的「规模不变性」对应于管理制度在员工数量变化时的结构不变性——是代数上的 n -均匀性 (n -homogeneity)，而非工程直觉。

§4.9 反类比：工程管理隐喻的失效边界

这一节在讲什么

为什么要主动拆解上一节的类比？好的类比是思维工具，但过度依赖类比会产生误导——了解类比在哪里失效，比知道它在哪里成立更重要。

第一步：训练没有「完成」这个状态。工程项目有二值判断 (done/not done)，训练的停止是一个连续的截断决策（验证集曲线斜率、算力预算），本质上是截断而非收敛。

第二步：神经元的职责是涌现的，不是预先指定的。工程模块有明确的功能分工；网络中每个参数的「语义」由训练动力学事后决定，需要可解释性工具才能反推，不存在先验的功能指定。

第三步：损失景观是不可消除的非凸。工程优化通常可转化为凸规划，有全局最优保证；深度学习的成功依赖高维几何的「幸运」结构（鞍点主导、模式连通性），而非可控的算法保证。

本节要拿走的一件事：我们能利用高维优化的几何性质，但无法从工程意义上「控制」它们——这是第四章最重要的警示。

直觉

类比在三点上失效：深度学习训练没有明确的「完成」状态；神经元的职责是涌现的，不是预先指定的；损失函数是高维非凸的，不像工程优化那样可分解为凸子问题。

第一个失效点——「没有完成状态」——比表面看起来更深刻。工程管理之所以能有效运转，部分原因在于每个子任务都有清晰的完成标准，管理者可以做出「是否完成」的二值判断并据此分配资源。但训练的停止完全是一个连续的权衡：验证集损失还在缓慢下降，但算力预算已经耗尽；或者验证集损失已经停止下降，但没有理论保证这是全局最优。「早停」是一个工程截断，不是数学意义上的收敛。这个区别意味着，工程管理中「项目完成后的复盘」逻辑在训练中并不适用——你永远不知道继续训练是否还能改善。

第二个失效点更根本。工程系统的模块化分工依赖于预先设计的接口和规范——每个模块知道自己应该做什么。神经网络没有这种预先规范：一个注意力头最终是否专注于句法、语义还是位置信息，完全由训练数据和动力学决定，是训练完成后才能用可解释性工具（如激活补丁、稀疏自编码器）反推出来的。这种「事后涌现的职责分工」从根本上不同于「事前设计的模块功能」，类比中「员工职责明确」的前提在这里完全不成立。理解这一点，也是理解为什么神经网络可解释性是一个独立的研究领域、而不能简单从架构设计中读出的原因。

严格（简版）

1. **无终止条件**。工程完工有二值判断 (done/not done)，训练的停止准则是连续的（验证集 loss 曲线的斜率阈值或算力预算约束），本质上是一个截断 (truncation) 而非收敛 (convergence)。
2. **功能涌现 (emergent function)**。每个参数的“语义”由训练动力学决定，事后才能通过可解释性方法（如激活补丁、稀疏自编码器）反推，不存在先验的功能指定。这与工程系统的模块化分工在信息论层面不同构。
3. **非凸 vs 凸**。工程最优化问题（成本最小化、路径最短化）通常可转化为凸规划，有全局最优保证。深度学习损失景观的非凸性不可消除，训练成功依赖高维几何的“幸运”结构（鞍点主导、模式连通性），而非可控的算法保证。第四章的核心教训是：**我们能利用这些几何性质，但无法从工程意义上“控制”它们。**

数学卷·第5章：标度律、相变与涌现的数理基础

§5.1 尺度律 (Scaling Law)：算力即权力的冷酷物理感

这一节在讲什么

为什么损失会随算力呈幂律下降而不是指数下降？因为语言本身在词、短语、段落、篇章各层次上具有自相似结构，幂律正是自相似系统的典型指纹。如果收益以指数速率衰减，很快就会归零；幂律收益永不截断，在双对数坐标下是一条无止境的直线。

第一步：三条单变量幂律。固定其他条件，损失分别对参数量 N 、数据量 D 、算力 C 呈 $\mathcal{L} \sim x^{-\alpha}$ 的形式下降，拟合指数均在 0.05–0.10 之间——这意味着每增加十倍投入，损失约降低 10%–20%。

第二步：联合损失函数 $L(N, D)$ 。Hoffmann 等把参数项与数据项合并为 $E + A/N^\alpha + B/D^\beta$ ，其中 E 是语言本身的不可归约熵——降不到零的下界。

第三步：拉格朗日求最优配比。在固定算力 $C \approx 6ND$ 的约束下，对 $L(N, D)$ 做拉格朗日优化，得到 $N^* \propto C^{\beta/(\alpha+\beta)}$ ——告诉你算力应以何种比例分配给参数和数据。

本节要拿走的一件事：幂律让模型性能变成可预测的「工程量」，而最优配比公式是 §5.2 Chinchilla 争论的数学起点。

直觉

原书 §5.1 揭示了一个令整个工业界沉默的事实：语言模型的损失值在七个数量级的范围内，对参数量 N 、数据量 D 、算力 C 呈精确的幂律下降。这意味着模型性能可以被预测，AI 研究从「艺术」变成「重工业」。

为什么会是幂律而不是指数律？直觉是：指数律意味着每增加一个单位投入，收益以固定比率衰减——最终收益几乎为零；幂律则意味着收益以递减但永不截断的方式下降，在双对数坐标上表现为一条无限延伸的直线。幂律描述的是自相似 (scale-free) 结构，而语言的复杂性恰好在很多层次上具有自相似性（词 → 短语 → 段落 → 篇章）。这不是巧合，而是语言统计结构的深层反映。

理解幂律的另一个角度是：它的斜率（幂指数 α ）是对「投入边际回报」的精确量化。 $\alpha_N \approx 0.076$ 意味着参数量翻十倍，损失仅降约 15%； $\alpha_D \approx 0.095$ 意味着数据量翻十倍，损失降约 19%。两个指数相

近，但差异足以影响最优配比的结论——这正是 Kaplan（参数优先）与 Hoffmann（等比例扩展）争论的根源。真正决定配比的是比值 α/β ，而不是两者的绝对大小（见「严格」部分的拉格朗日推导）。

一个常见的误解是把「不可归约损失 E 」当作可以继续压缩的目标。 E 代表语言本身的条件熵——即使给定所有上下文，下一个词仍有的随机性，它与模型大小无关。在实践中， E 被估计为约 1.69 nats（对应困惑度约 5.4）；任何训练方案都无法突破这个下界，盲目扩大 N 或 D 只会让 A/N^α 和 B/D^β 趋近于零而整体损失平台化，而非继续下降。

严格

Kaplan 单变量幂律。 设 N 为非嵌入参数量， D 为训练 token 数， C 为训练总 FLOPs，Kaplan 等（2020）拟合出：

$$\mathcal{L}(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad \mathcal{L}(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad \mathcal{L}(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C}$$

其中 $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$, $\alpha_C \approx 0.050$, N_c, D_c, C_c 为拟合常数。

幂律 vs 指数律的渐近行为。 设 $f(x) = x^{-\alpha}$ （幂律）， $g(x) = e^{-\lambda x}$ （指数律）。对任意 $\alpha, \lambda > 0$ ，有：

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} x^{-\alpha} e^{\lambda x} = +\infty$$

即指数律衰减比任意幂律都快——对数坐标下，幂律是直线，指数律是向下弯曲的曲线。双对数坐标下， $\log \mathcal{L} = -\alpha \log N + \text{const}$ ，斜率即幂指数 $-\alpha$ 。

Hoffmann 联合损失函数（为 §5.2 铺垫）。 原书在 §5.1 引入 Kaplan，在 §5.2 引入 Chinchilla 的联合形式：

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

其中 $E \geq 0$ 是不可归约损失（irreducible loss），即语言本身的熵下限； A/N^α 表示参数不足导致的欠拟合； B/D^β 表示数据不足导致的欠拟合。 $\alpha, \beta > 0$ 为正数，意味着两项均单调递减。

Compute-optimal 的拉格朗日推导。 给定固定算力预算 C ，变换器（Transformer）的标准 FLOPs 近似为：

$$C \approx 6ND$$

（每个参数在前向 + 反向传播中各贡献约 $2D$ 次乘加，共 $6ND$ 。）目标是在约束 $6ND = C$ 下最小化 $L(N, D)$ 。构造拉格朗日函数：

$$\mathcal{L}(N, D, \lambda) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} - \lambda(6ND - C)$$

对 N 和 D 分别求偏导并令其为零：

$$\frac{\partial \mathcal{L}}{\partial N} = -\frac{\alpha A}{N^{\alpha+1}} - 6\lambda D = 0 \quad \Rightarrow \quad \frac{\alpha A}{N^{\alpha+1}} = 6\lambda D \quad (1)$$

$$\frac{\partial \mathcal{L}}{\partial D} = -\frac{\beta B}{D^{\beta+1}} - 6\lambda N = 0 \quad \Rightarrow \quad \frac{\beta B}{D^{\beta+1}} = 6\lambda N \quad (2)$$

由 (1) 和 (2) 消去 λ :

$$\frac{\alpha A}{N^{\alpha+1}} \cdot \frac{1}{D} = \frac{\beta B}{D^{\beta+1}} \cdot \frac{1}{N} \Rightarrow \frac{\alpha A}{N^{\alpha} D} = \frac{\beta B}{N D^{\beta}} \Rightarrow \frac{\alpha A}{N^{\alpha}} = \frac{\beta B}{D^{\beta}}$$

即在最优点处, 参数项与数据项的边际贡献之比满足:

$$\frac{A/N^{\alpha}}{B/D^{\beta}} = \frac{\beta}{\alpha} \quad (3)$$

结合约束 $6ND = C$, 由 (3) 解出:

$$N^* \propto C^{\beta/(\alpha+\beta)}, \quad D^* \propto C^{\alpha/(\alpha+\beta)}$$

当 $\alpha \approx \beta$ (Hoffmann 拟合值 $\alpha \approx 0.34$, $\beta \approx 0.28$ 近似) 时, $N^* \propto C^{0.5}$, $D^* \propto C^{0.5}$, 即参数量与 token 数应等比例扩展。

更一般地, 若记 $r = \alpha/\beta$, 则:

$$N_{\text{opt}}^* \propto C^{1/(1+r)}, \quad D_{\text{opt}}^* \propto C^{r/(1+r)}$$

当 $r > 1$ (参数的边际回报衰减更快) 时, D^* 的增长指数大于 N^* , 表明应更多增加数据。

与原书呼应: 原书 §5.1 提到「模型的性能可以被精确预测」, 对应本节幂律拟合与拉格朗日推导给出的 compute-optimal 配比公式 $N_{\text{opt}}^* \propto C^{\beta/(\alpha+\beta)}$ 。

§5.2 Chinchilla 的颠覆与再颠覆

这一节在讲什么

为什么同一个损失函数会给出截然不同的结论? 因为拟合方法不同。Kaplan 用参数固定、数据变化的实验, Hoffmann 用等算力切片 (IsoFLOP), 两者在信息矩阵的条件数上差异巨大, 导致最优配比从「参数优先」翻转为「1:20 等比扩展」。

第一步: 三种拟合方法的方向一致性。 IsoFLOP (方法一)、等损失曲线 (方法二)、全局参数化最小二乘 (方法三) 均指向同一结论: 每个参数对应约 20 个训练 token, 而非 Kaplan 建议的 5 个。

第二步: 方法三的置信区间为何过窄。 非线性最小二乘的 CI 宽度正比于 $\hat{\sigma}/\sqrt{n}$, 但信息矩阵 $J^T J$ 条件数极大, 参数 α 与 A 强相关, 导致对角元素被低估——400 个训练点撑不起如此紧致的置信区间。

第三步: Sardana 修正——把推理成本纳入预算。 真实世界里推理调用数十亿次, 约束从 $6ND \leq C$ 扩展为 $6ND + 2N \cdot n_{\text{inf}} \leq C_{\text{total}}$, 最优解向「更小的 N 、更多的 D 」偏移。

本节要拿走的一件事: Chinchilla 的方向性结论 (等比扩展) 仍有效, 但其置信区间的统计学基础存疑; 推理成本一旦纳入, 最优点还会进一步向小模型偏移。

直觉

Hoffmann 等 (2022) 用同样的联合损失函数 $L(N, D)$ ，却得出了与 Kaplan 截然不同的结论：参数量与训练 token 数应以约 1:20 的比例同等扩展，而非「参数优先」。这一修正背后有三种不同的拟合方法，它们给出的结论方向一致，但置信区间宽窄差异悬殊。Epoch AI 的复现工作揭示了其中方法三的统计学问题，而 Sardana 等 (ICML 2024) 进一步指出，即使 Chinchilla 最优解在训练成本上正确，也忽略了推理成本——真实世界中推理调用数十亿次，应将推理成本纳入总优化目标。

方向性结论的翻转为什么发生？关键在于 Kaplan 的实验设计是固定 D 变化 N ，而 Hoffmann 的 IsoFLOP 实验是固定算力 C 同时扫描 (N, D) 组合——后者直接在约束面上测量最优点，信息更为直接。当真实最优点处 $N^*/D^* \approx 1/20$ 时，Kaplan 的单变量扫描因数据量偏少而系统性地高估了参数的边际价值，给出了偏向大 N 的错误建议。这不是理论推导的错误，而是实验设计覆盖范围的局限。

方法三的统计学问题值得特别注意：它不是「结论错了」，而是「置信区间夸大了精度」。如果真实不确定性更大，那么 Hoffmann 报告的 $\alpha = 0.34 \pm 0.02$ 实际上可能跨越一个更宽的范围，而 Kaplan 与 Hoffmann 之间的差异有相当部分可能落在这一更宽的 CI 之内。这提醒我们：对 scaling law 参数的精确数值保持审慎，它们的意义更多在于数量级和比例关系，而非精确常数。

严格

Hoffmann 三种拟合方法。设损失模型为：

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} \quad (4)$$

共有 5 个参数 $\{E, A, \alpha, B, \beta\}$ 待拟合。Hoffmann 等使用了三种方法：

- **方法一 (IsoFLOP)**：固定总算力 $C = 6ND$ ，在多组 (N, D) 上测量损失，拟合每条等算力曲线的最优 N^* ，再对 $N^*(C)$ 做幂律回归，得出 $N^* \propto C^{0.50}$ 。
- **方法二 (IsoLoss)**：找出达到相同损失值所需的 (N, D) 组合，沿等损失曲线回归，得出类似结论。
- **方法三 (参数化拟合)**：直接对全部 (N, D, L) 观测数据用最小二乘拟合方程 (4) 的 5 个参数，Hoffmann 报告的置信区间极窄（如 $\alpha = 0.34 \pm 0.02$ ）。

三种方法的方向性结论一致：最优 N^*/D^* 比约为 1/20（即每个参数对应约 20 个训练 token），与 Kaplan 建议的 1/5 差异显著。

Epoch AI 复现与置信区间统计学根源。Epoch AI (2024) 指出，方法三的置信区间过窄，在统计学上源于以下机制：

方法三对有限的训练点做非线性最小二乘，设观测数为 n ，参数数为 $p = 5$ ，残差方差为 $\hat{\sigma}^2$ ，则参数 $\hat{\theta}$ 的渐近协方差矩阵为：

$$\text{Cov}(\hat{\theta}) \approx \hat{\sigma}^2 (J^\top J)^{-1}$$

其中 $J \in \mathbb{R}^{n \times p}$ 是雅可比矩阵。置信区间宽度正比于 $\hat{\sigma}/\sqrt{n}$ 。Chinchilla 论文实际训练的模型数量约为 400 个，但非线性拟合中信息矩阵 $J^\top J$ 的条件数往往很大（参数 α, β 与 A, B 之间存在强相关性），导致 $(J^\top J)^{-1}$ 的对角元素被低估，最终报告的 CI 比实际参数不确定性窄得多。要支撑如此紧致的置信区间，理论上需要数十万量级的独立观测点。

Sardana 推理时计算修正。设 C_{train} 为训练总算力， C_{inf} 为单次推理算力（约 $2N$ FLOPs）， n_{inf} 为生命周期内推理调用次数，总算力预算为 C_{total} 。扩展后的优化问题为：

$$\min_{N,D} L(N,D) \quad \text{s.t.} \quad C_{\text{train}} + n_{\text{inf}} \cdot C_{\text{inf}} \leq C_{\text{total}}$$

其中 $C_{\text{train}} \approx 6ND$, $C_{\text{inf}} \approx 2N$, 故约束变为:

$$6ND + 2N \cdot n_{\text{inf}} \leq C_{\text{total}} \quad \Rightarrow \quad N(6D + 2n_{\text{inf}}) \leq C_{\text{total}} \quad (5)$$

当 $n_{\text{inf}} \gg 3D$ (即推理调用次数远超训练 epoch 数的三倍) 时, 约束 (5) 的主导项变为 $2Nn_{\text{inf}}$, 即推理成本主导。此时拉格朗日最优解偏向使用**更小的** N (配合更多训练 token D), 因为减小 N 既降低推理成本, 又可将节省下来的预算投入更多训练数据。

这是 LLaMA-3 8B 使用 15T tokens (约为 Chinchilla 最优的 100 倍) 训练的理论依据: 在推理量极大的部署场景下, 小模型 + 超量训练比大模型 + 少量训练更经济。

与原书呼应: 原书 §5.2 提到「Chinchilla 所指出的方向性结论仍然具有重要影响」及 Epoch AI 对「方法三置信区间过窄」的批评, 对应本节对信息矩阵条件数问题的分析, 以及 Sardana 修正公式 $\min L(N,D) \text{ s.t. } C_{\text{train}} + n_{\text{inf}} \cdot C_{\text{inf}} \leq C_{\text{total}}$ 。

§5.3 数据墙与 §5.4 推理时计算

这一节在讲什么

为什么 scaling 会撞墙? 人类产生的文本总量有限——约 3×10^{14} token——而当前扩展速率将在数年内耗尽这一资源。数据墙不是性能瓶颈, 而是一个约束边界: $D \leq D_{\text{max}}$, 一旦触碰, 损失函数里 B/D^β 项就被钉死, 无论 N 多大, 损失都有一个受数据支配的下界。

第一步: 合成数据的陷阱——模型崩溃。 用模型自己生成的数据反复训练, 每一代都相当于对真实分布做有噪采样, 低概率的尾部被截断, 经多代迭代后分布退化为一个点——Wasserstein-2 距离随代数单调发散。

第二步: 推理时计算作为「第二曲线」。 Best-of- K 采样在推理阶段投入更多算力, 绕过训练数据的上限。每 Best-of-3 采样等价于约 1 个额外训练 token 的算力——这个折算率对可验证任务 (数学、代码) 高, 对不可验证任务 (创意写作) 低。

第三步: 两条曲线的关系。 推理时计算不是训练时计算的替代, 而是在数据墙之后的补充路径; 两者的最优组合取决于任务是否有客观验证信号。

本节要拿走的一件事: 数据墙迫使 scaling 从「烧更多训练数据」转向「在推理时花更多算力」, 而这一转变的数学基础是等价 token 折算率 $D_{\text{equiv}}(K) = K/3$ 。

直觉

原书 §5.3 提出数据墙 (data wall): 公开人类文本总量约 3×10^{14} token, 在当前 scaling 速率下将于 2026–2032 年耗尽。§5.4 描述了「第二曲线」——推理时计算缩放 (inference-time scaling), 通过在推理阶段投入更多算力 (链式推理、N 选优 (Best-of-N) 采样) 来补偿训练数据的不足。

数学上, 数据墙是一个约束边界: $D \leq D_{\text{max}} \approx 3 \times 10^{14}$ 。一旦 D 到顶, 若仍要降低 L , 只能继续增大 N ——但 B/D^β 项固定, 损失存在一个受数据量支配的下界。

合成数据看似提供了绕过这一上限的途径, 但模型崩溃的数学机制揭示了它的根本局限: 每一代生成

都是对真实分布的一次有噪采样，低概率区域——也是语言中最有信息量的罕见表达——会在采样过程中被系统性地丢弃。一代一代迭代下去，方差单调收缩，最终分布退化到一个几乎无信息的退化点。这不是工程缺陷，而是采样过程的数学必然：有偏的采样器在有限样本下必然截断尾部，而尾部的缺失无法通过后续训练恢复，因为训练数据本身就缺少了那些信息。

推理时计算之所以能作为「第二曲线」，根本原因是它把测试时的算力投入转化成了对输出空间的更充分探索。Best-of- K 本质上是用 K 倍算力换取对输出分布更高分位数的近似采样——在有客观评分函数（数学答案正确性、代码测试通过率）的任务上，这种探索有明确的方向；在无法客观评分的任务上，「更多采样」只是更多噪声，折算率趋近于零。

严格

模型崩溃 (model collapse) 的分布退化。 Shumailov 等 (Nature 2024) 证明，若模型递归训练于自身生成的数据，第 n 代模型的输出分布 $\mathcal{D}_n$ 的方差满足：

$$\Sigma_n \xrightarrow{a.s.} 0 \quad \text{as } n \rightarrow \infty$$

直觉：每代生成数据相当于对真实分布做一次有噪采样后再拟合，低概率区域（尾部）在采样时被截断，经多代迭代后尾部彻底消失，分布收敛到退化点。Wasserstein-2 距离满足：

$$\mathbb{E}[\mathbb{W}_2^2(\mathcal{N}(\mu_n, \Sigma_n), \mathcal{D}_0)] \rightarrow \infty$$

即合成数据与原始真实分布的距离随代数增加而发散。

推理时计算的等价 token 折算。 设基础模型参数量为 N_0 ，推理时使用 Best-of- K 采样：从 K 个独立采样中选最优输出，验证成本约为 $K \cdot 2N_0$ FLOPs。如果以「等价训练 token」衡量推理投入，令 $C_{\text{test}} = K \cdot 2N_0$ ，则其对应的等价训练 token 数为：

$$D_{\text{equiv}}(K) = \frac{C_{\text{test}}}{6N_0} = \frac{K}{3}$$

即每 Best-of-3 采样大约相当于额外提供 1 个训练 token 的算力。实践中，推理时计算与训练时计算的「兑换率」取决于任务类型：可验证任务（数学、代码）兑换率高，因为有客观奖励信号指导采样选择；不可验证任务（创意写作）兑换率低。

与原书呼应：原书 §5.3 引用 Shumailov 等人的「模型崩溃」结果，对应本节 $\Sigma_n \rightarrow 0$ 的方差退化；§5.4 描述推理时计算缩放，对应本节 $D_{\text{equiv}}(K) = K/3$ 的等价折算关系。

§5.5 涌现：神话还是幻觉？

这一节在讲什么

为什么「涌现」可能只是测量幻觉？ 研究者用的是非线性评估指标（如全序列精确匹配），把底层平滑增长的 token 级能力压缩成了视觉上的阶跃——用阶梯状温度计量连续升温的水，读数也会「突变」。

第一步：非线性指标放大了导数。 长度为 ℓ 的精确匹配指标等于 $p(N)^\ell$ ，在 p 过临界值时，导数被 $\ell \cdot p^{\ell-1}$ 放大，序列越长，「阶跃」越陡峭——这是数学必然，而非新能力的出现。

第二步：换连续指标，涌现消失。 Schaeffer 等用 token 级 log-likelihood 重新评估原本呈现「涌现」的任务，曲线全部变为平滑的幂律增长，与预训练损失吻合。

第三步：保留少数真实阈值。 Olsson 等对 induction head 电路的机制研究表明，in-context learning 能力的出现即使在连续指标下也呈现阶跃——但这是电路形成的结构性事件，与热力学相变仍有本质区别（见 §5.10）。

本节要拿走的一件事： 大多数「涌现」是 $dm/dN = \delta(p(N) - \theta) \cdot dp/dN$ 的指标伪影；真正可信的涌现需要在所有连续可微指标下均持续存在。

直觉

原书 §5.5 的核心争论：Wei 等（2022）记录了许多任务上随模型规模增大出现「突然跃升」的准确率——被称为涌现能力（emergent abilities）。Schaeffer、Miranda 与 Koyejo（NeurIPS 2023 杰出论文）反驳说，这种「涌现」是测量幻觉：研究者选择了分段不连续的评估指标，把底层平滑的能力增长映射成了视觉上的阶跃。

类比：用阶梯状的温度计（只有 0 和 1 两个刻度）去量一壶从室温缓慢升温的水，读数会从 0 突然跳到 1——但水温本身是连续上升的。换连续刻度的温度计，「突变」就消失了。

理解这一争论的关键是区分两个层次：**底层能力**（token 级的 log-likelihood，连续平滑增长）和**表现指标**（精确匹配、多选准确率，高度非线性）。非线性指标把底层平滑信号通过 $m(p)$ 这个非线性函数投影到观测空间——投影之后的导数 $m'(p) \cdot dp/dN$ 可以在特定点处极大，制造出视觉上的阶跃。这本质上是一个信号处理问题：非线性变换会在频域引入谐波，把低频（平滑）信号变成高频（突变）信号。因此，「涌现」的存在性在某种程度上取决于我们用什么「滤镜」看待能力增长。

然而，不能把所有涌现都归结为指标伪影。Olsson 等的机制可解释性工作提供了一条独立证据链：他们直接追踪了模型内部 induction head 电路的形成时刻，发现即使不看任何外部指标，训练过程中存在某个特定步骤，电路结构从「不存在」变为「稳定存在」——这是内部表示层面的结构性事件，而非评估指标的函数值跳变。这说明两类「涌现」需要区别对待：一类是指标伪影（可以通过换指标消除），另一类是电路层面的涌现（换指标也消除不了，但与物理相变仍有本质差异）。

严格

指标非线性导致的表现涌现。 设模型规模为 N ，token 级正确率（token-level accuracy）为 $p(N)$ ，假设其随 N 连续、单调递增（例如随 $\log N$ 线性增长）。

对长度为 ℓ 的序列，「全序列正确」（exact match）指标为：

$$M_{\text{exact}}(N) = \prod_{i=1}^{\ell} p_i(N) \approx p(N)^\ell$$

其中假设各 token 独立且每位正确率相同。当 $p(N) < 1/\ell$ 时, $M_{\text{exact}} \approx 0$; 当 $p(N)$ 略超过 $1/\ell$ 时, M_{exact} 迅速从 0 跳向正值。具体地, 在 $p(N^*) = 1/\ell^{1/\ell}$ 附近, M_{exact} 对 N 的导数为:

$$\frac{dM_{\text{exact}}}{dN} = \ell \cdot p(N)^{\ell-1} \cdot \frac{dp}{dN}$$

当 ℓ 大时 (如 $\ell = 10$ 的三位数加法), $\ell \cdot p^{\ell-1}$ 在过渡区附近极大, 导致 M_{exact} 对 N 呈近似阶跃响应。Schaeffer 将此一般化为: 若评估指标 $m(p)$ 是关于 p 的非线性函数 (如 $m(p) = \mathbf{1}[p \geq \theta]$), 则:

$$\frac{dm}{dN} = m'(p) \cdot \frac{dp}{dN}$$

对阶梯函数 $m(p) = \mathbf{1}[p \geq \theta]$, 其导数为 Dirac delta: $m'(p) = \delta(p - \theta)$, 故:

$$\frac{dm}{dN} = \delta(p(N) - \theta) \cdot \frac{dp}{dN}$$

这是一个在 $p(N^*) = \theta$ 处集中的脉冲——视觉效果即「涌现」。而换用连续指标 $m(p) = p$ (token-level log-likelihood) 时, $m'(p) = 1$, 曲线即还原为 dp/dN 的平滑形状。

连续度量下的平滑性验证。Schaeffer 等在实验中替换指标后验证: 原本呈现「涌现」的任务, 在 token-level log-likelihood 下均表现为平滑的幂律增长, 与预训练损失曲线吻合。这一结果的可证伪推论是: **若某任务的「涌现」在所有连续可微指标下均持续存在, 则该涌现更可能是真实能力阈值, 而非指标伪影。**

反驳路线保留的空间: Olsson 等 (2022) 的 induction head 工作从机制层面证明, in-context learning 能力的出现与模型内部 induction head 电路的形成在训练步骤上高度同步, 呈现明确的阶跃特征——即使换用连续指标, 该阶跃仍然可见。这说明存在少数真实的能力阈值, 但它们与物理意义上的「相变」仍有本质区别 (见 §5.10)。

与原书呼应: 原书 §5.5 引用 Schaeffer 等 (NeurIPS 2023 杰出论文), 对应本节 $dm/dN = \delta(p(N) - \theta) \cdot dp/dN$ 的形式分析; 原书提及换用 token-level log-likelihood 曲线变平滑, 对应本节连续指标下的 $m'(p) = 1$ 情形。

§5.6–§5.7 Grokking 的数学解剖：从死记到顿悟

这一节在讲什么

为什么模型会先「死记」再「顿悟」？两套电路在同一个网络里竞争：稠密记忆电路快速拟合训练集但参数范数大，稀疏傅里叶电路需要更长时间发展但范数小。权重衰减持续惩罚大范数，最终让记忆电路衰竭、傅里叶电路胜出。

第一步：任务的代数结构决定了「顿悟」的形式。模算术 $a + b \bmod p$ 恰好对应圆周群上的相位旋转，而圆周群的表示理论正是傅里叶基——模型学到的不是查表，而是三角恒等式 $\cos(\theta_a + \theta_b) = \cos \theta_a \cos \theta_b - \sin \theta_a \sin \theta_b$ 。

第二步：两阶段动力学。记忆阶段训练损失快速下降、测试损失不动；竞争阶段两套电路并存、傅里叶分量逐渐增强；清理阶段记忆权重被压到阈值以下，测试损失突然归零。

第三步：增大权重衰减系数 λ_{wd} 可以提前 Grokking 时刻——这是正则化驱动的竞争过程，而非任何意义上的相变。

本节要拿走的一件事：Grokking 的延迟泛化不是神秘的「顿悟」，而是权重衰减在两套电路之间执行的一场慢速淘汰赛，任务的代数结构决定了最终胜出电路的形式。

直觉

原书 §5.6–§5.7 描述了 Grokking 现象 (Power 等, 2022)：在模算术任务 $a + b \bmod p$ 上，模型先 100% 记忆训练集（测试准确率约 0%），再经数万步训练后测试准确率突然跃升至 100%。这被称为「延迟泛化」。

Nanda 等 (ICLR 2023) 的机制可解释性研究揭示，模型并非在「顿悟」——它悄悄学到了离散傅里叶基表示，并用三角恒等式实现了模加法。权重衰减 (weight decay) 是驱动这一转变的关键正则化力量：它持续压缩稠密记忆权重，直到更紧凑的傅里叶电路「胜出」。

为什么模算术这个任务的「正确表示」恰好是傅里叶基？这来自任务本身的代数结构。整数 $\{0, 1, \dots, p-1\}$ 在模加法下构成一个循环群（圆周群 $\mathbb{Z}/p\mathbb{Z}$ ），而圆周群的不可约表示恰好是复指数函数 $e^{2\pi i k a/p}$ ——也就是傅里叶基。模型通过梯度下降自发地「发现」了群论的表示理论：这不是研究者施加的先验，而是任务结构通过损失函数传递给网络权重的隐式约束。换句话说，如果任务有一个漂亮的代数结构，网络最终会找到与之同构的漂亮表示——前提是正则化足够强，能清除掉「丑陋但有效」的记忆捷径。

一个常见的误解是把「延迟泛化」理解为「模型突然学会了新东西」。实际上，傅里叶电路在记忆阶段就开始悄悄发展，只是被更强大的记忆电路掩盖，在指标上看不到。测试准确率的阶跃并不对应任何新知识的获取，而只是权重衰减把记忆电路的范数压到了某个功能阈值以下——就像一条原本被大声噪音掩盖的旋律，在噪音衰减后突然变得清晰可闻。这意味着 Grokking 本质上是一个**信噪比问题**：傅里叶信号一直在那里，只是一直被记忆噪声淹没。

严格

任务设置。取素数 $p = 113$ ，输入为 $(a, b) \in \{0, \dots, p-1\}^2$ ，目标为 $(a + b) \bmod p$ 。训练集取所有 p^2 对中的 30%，测试集为剩余 70%。

Embedding 矩阵的傅里叶基收敛。Nanda 等逆向工程发现，训练后模型的 embedding 矩阵 $E \in \mathbb{R}^{p \times d}$ (d 为隐藏维度) 在若干频率 k 处形成了如下结构：

$$E[a, 2k] = \cos\left(\frac{2\pi ka}{p}\right), \quad E[a, 2k+1] = \sin\left(\frac{2\pi ka}{p}\right), \quad k \in \mathcal{K}$$

其中 $\mathcal{K}$ 是模型自发选择的少数几个频率（实验中共约 5 个）。这等价于将整数 a 映射为在特定频率下的复数相位 $e^{2\pi i ka/p}$ 。

注意力层实现频域加法。设 attention 层在频率 k 处的操作将 a 和 b 的嵌入合并，实现：

$$\cos\left(\frac{2\pi k(a+b)}{p}\right) = \cos\left(\frac{2\pi ka}{p}\right)\cos\left(\frac{2\pi kb}{p}\right) - \sin\left(\frac{2\pi ka}{p}\right)\sin\left(\frac{2\pi kb}{p}\right)$$

这正是余弦的加法定理。模型利用 bilinear 注意力结构（query 与 key 的内积）自然地实现了这一乘积——两个 embedding 向量的内积提取了跨频率的 $\cos(\theta_a)\cos(\theta_b)$ 和 $\sin(\theta_a)\sin(\theta_b)$ 项。

输出层的逆傅里叶解码。网络最终输出 logit 向量 $\hat{y} \in \mathbb{R}^p$ ，其中：

$$\hat{y}_c \propto \sum_{k \in \mathcal{K}} w_k \cos\left(\frac{2\pi k(a+b-c)}{p}\right)$$

对 c 取 $\arg\max$ ，即等价于用若干傅里叶频率的叠加来定位 $(a+b) \bmod p$ 。

两阶段动力学：权重衰减驱动的延迟泛化。定义两个进度指标：

$$L_{\text{Fourier}}(t) = L(\text{仅保留傅里叶频率分量的模型}), \quad L_{\text{mem}}(t) = L(\text{去除傅里叶分量后的模型})$$

训练动力学分三阶段：

阶段	步数 ($p = 113$)	训练 loss	测试 loss	L_{Fourier}	L_{mem}
记忆	0 – 1k	快速下降	不动	高	快速下降
电路竞争	1k – 9k	接近 0	不动	下降	下降
清理	9k – 14k	≈ 0	突然下降至 0	≈ 0	≈ 0

权重衰减的数学角色：设权重为 θ ， ℓ_2 正则化强度为 λ_{wd} ，则训练目标为：

$$\mathcal{J}(\theta) = L_{\text{CE}}(\theta) + \frac{\lambda_{\text{wd}}}{2} \|\theta\|_2^2$$

稠密记忆电路需要大量权重（大 $\|\theta\|_2^2$ ），而稀疏傅里叶电路用少数频率分量表示同样的功能，权重范数更小。正则化项持续惩罚记忆电路，使其在竞争中逐渐处于劣势；当记忆电路权重衰减至低于傅里叶电路的表达阈值时，后者独占输出，测试 loss 才突然降至 0。

这解释了 Grokking 的关键实验观察：增大 λ_{wd} 可以提前 Grokking 时刻，减小 λ_{wd} 则推迟甚至阻止 Grokking——表明这是正则化驱动的竞争过程，而非任何形式的真实相变。

与原书呼应：原书 §5.7 引用 Nanda 等 (ICLR 2023)，对应本节 embedding 的傅里叶基收敛公式与三角恒等式推导；§5.6 提及「权重衰减驱动延迟泛化」，对应本节正则化目标 $\mathcal{J}(\theta)$ 及两阶段动力学表格。

§5.8 渗流相变：把 Scaling Law 焊到统计物理上

这一节在讲什么

为什么渗流理论是描述涌现的合适框架？因为它正好捕捉了「单元素平滑增长 → 整体性质急剧变化」的模式：每个格点的占据概率 p 连续增大，但最大连通簇的尺寸在临界点 p_c 处从对数级跳到线性级。这与「单概念能力平滑增长、复合推理突然可行」的结构同构。

第一步：二维渗流的精确临界指数。序参量 $P_\infty \sim (p - p_c)^\beta$ ($\beta = 5/36$)，关联长度 $\xi \sim |p - p_c|^{-\nu}$ ($\nu = 4/3$)，均为严格数学结果，来自共形场论和 SLE 理论。

第二步：能力涌现的渗流映射。把单概念掌握概率 $p(N)$ 建模为 N 的连续函数，复合能力 $C(N)$ 在 $p(N)$ 过 p_c 时导数呈类 delta 脉冲——可证伪预测：推理链越长，涌现越陡峭。

第三步：神经网络剪枝验证。Pesce 等 (2026) 发现剪枝到约 98% 边时网络功能崩溃，功能-保留率曲线与渗流序参量 $F \sim (\bar{q} - \bar{q}_c)^\beta$ 形式同构。

本节要拿走的一件事：渗流给涌现提供了一个有精确临界指数、可做定量预测的物理框架——但这个框架能否真正用于神经网络，取决于「单概念掌握」这一映射是否成立，这是尚待验证的假设。

直觉

原书 §5.8 将「涌现」与统计物理中的渗流理论 (percolation theory) 相联系：把模型掌握的「概念」想象成图上的节点，复合能力对应图上的连通路径。当单概念掌握概率越过临界值 p_c ，最大连通簇尺寸从对数级跳到线性级——这给「涌现」一个可证伪的渗流解释。

渗流为什么是合适的类比？因为它恰好是「单元素平滑增长 → 整体性质急剧变化」的典型模型，与「底层能力平滑增长、表观性能阶跃」的结构吻合。

渗流框架的真正价值在于它的**定量可证伪性**。物理相变有精确的临界指数， $\beta = 5/36$ 和 $\nu = 4/3$ 不是经验估计，而是通过共形场论严格推导出来的。如果神经网络的涌现行为真的属于同一普适类，那么它应该满足相同的指数——这是一个可以检验的预言，不是一个模糊的比喻。这正是渗流框架相比于「复杂系统涌现」这类笼统说法的优越之处：后者什么都能解释（因此什么都不能预测），前者提供了一个明确的定量约束。然而，从「模型掌握某概念」到「格点被占据」的映射是否成立，目前仍是假设而非已验证的事实；在这一步被严格建立之前，渗流类比的定量预测能力不能被过度引申。

神经网络剪枝实验则提供了一条独立的实证路径：它不需要「概念 → 格点」的映射假设，而是直接在网络连通性层面测量功能-保留率曲线，发现其形式与渗流序参量同构。这让渗流解释从纯粹的类比上升为有独立实验支撑的结构假说——尽管剪枝网络的「相变」与自然训练网络的「涌现」是否属于同一物理机制，仍需更多工作来确认。

严格

二维 site percolation 的临界现象。考虑二维正方形格点 $\mathbb{Z}^2$ 上的伯努利渗流 (Bernoulli site percolation)：每个格点以概率 p 被「占据」，彼此独立。临界概率 $p_c \approx 0.5927$ (数值精确值)。

设 $P_\infty(p)$ 为无穷大格点中随机选取的占据格点属于无穷大连通簇的概率 (序参量)。在 p_c 附近：

$$P_\infty(p) \sim (p - p_c)^\beta, \quad p \rightarrow p_c^+$$

其中临界指数 $\beta = 5/36$ 。关联长度 (correlation length) ξ ，即连通簇线性尺度的特征值，满足：

$$\xi(p) \sim |p - p_c|^{-\nu}, \quad \nu = 4/3$$

$\xi \rightarrow \infty$ 意味着在临界点附近，任意尺度的涨落均存在——这是连续二阶相变的标志。序参量 P_∞ 在 p_c 处连续地从 0 上升（不发生不连续跳变），故为二阶（连续）相变。

这两个精确临界指数（ $\beta = 5/36$, $\nu = 4/3$ ）来自二维伯努利渗流的精确结果，是严格数学意义上（通过共形场论和 SLE 理论）已经证明的，并非数值近似。

能力涌现的渗流映射。将单概念 c 的「掌握概率」建模为规模的函数：

$$p(N) = \sigma(\alpha \log N + \beta_0) = \frac{1}{1 + N^{-\alpha} e^{-\beta_0}}$$

其中 σ 为 sigmoid 函数， $p(N)$ 关于 N 连续单调递增。

定义复合能力函数 $C(N)$ 为「存在长度 $\geq k$ 的全连通路径」的概率（即多跳推理成功的概率）。在渗流框架下，当 $p(N)$ 越过 p_c 时：

$$\frac{dC}{dN} \approx \delta(p(N) - p_c) \cdot \frac{dp}{dN}$$

单概念能力 $p(N)$ 平滑，但复合能力 $C(N)$ 在 p 过 p_c 时呈现类 delta 脉冲。可证伪预测：**能力涌现的陡峭程度应正比于推理链长度 k** ——单步任务平滑，多步任务陡峭。

神经网络剪枝的相变行为。 Pesce、He 与 Caldarelli (2026) 研究神经网络剪枝时发现：随着删除边的比例 q 从 0 增大，网络功能（以某任务准确率衡量）在临界比例 q_c 处发生从「基本保持」到「完全失能」的急剧转变，且临界指数与渗流的标度律一致。实验表明，最高可移除约 98% 的边而性能损失很小，越过 q_c 后才发生崩塌。

设网络功能 F 为边保留率 $\bar{q} = 1 - q$ 的函数，则：

$$F(\bar{q}) \sim (\bar{q} - \bar{q}_c)^\beta \cdot \mathbf{1}[\bar{q} > \bar{q}_c]$$

其中 $\bar{q}_c \approx 0.02$ （约 2% 保留）对应 $q_c \approx 98\%$ 。这与渗流的 $P_\infty \sim (p - p_c)^\beta$ 在形式上同构，指向同一物理机制：**网络功能是连通性的函数。**

与原书呼应：原书 §5.8 提到二维 site percolation 的临界指数 $\beta = 5/36$ 、 $\nu = 4/3$ 及 Pesce 等 (2026) 剪枝临界比例约 98%，对应本节严格推导；渗流序参量连续上升（二阶相变）的说明对应原书「标准二维伯努利渗流是连续（二阶）相变」的表述。

§5.9 跨界映射与 §5.10 反类比：相变叙事的边界

这一节在讲什么

为什么「AI 涌现是相变」这个类比虽然有吸引力但存在根本性缺陷？物理相变有三个严格要求：热力学极限 ($N \rightarrow \infty$)、普适临界指数、时间反演对称（可逆）——AI 涌现一个都不满足。

第一步：有限系统只有 crossover，没有相变。自由能密度 $f(T) = \lim_{N \rightarrow \infty} F(N, T)/N$ 的非解析性只在 $N \rightarrow \infty$ 时出现；有限 N 的 $f(N, T)$ 是有限个指数之和的对数，永远解析——无论网络多大。

第二步：后视镜叙事无法预测。历史上的文明「相变」是事后贴上的标签；事前不知道序参量 ϕ 的形式，就无法预测转变时刻 t^* 。AI 涌现同理，「下一次涌现何时发生」没有任何理论依据。

第三步：不可逆性打破动力学同构。物理相变（一阶或二阶）沿相图均可逆；AI 能力涌现和文明转变均不满足时间反演对称——已习得的能力无法通过逆向训练消除。

本节要拿走的一件事：用「相变」描述 AI 涌现是一种生动但不严格的比喻；有限系统、后视镜叙事、不可逆性这三条界限，划定了这个类比能走多远。

直觉

原书 §5.9 列举了农业革命、城邦兴起、工业革命、互联网等历史拐点，将它们类比为文明的「相变」，并指出 AI 涌现被许多业界领袖以「phase transition」的语言包装。§5.10 则给出三条反驳：物理相变要求热力学极限；文明「相变」是后视镜叙事，不是预测工具；物理相变（除特殊情形外）可逆，文明涌现不可逆。

这三条反驳的力度并不相同。**热力学极限**是最硬的数学约束：解析函数的有限和仍是解析函数，这是高中级别的数学事实，不存在任何争议；任何有限参数量的网络，无论多大，都必然只能有 crossover 而非严格相变。**后视镜叙事**的反驳力度次之：它指出的是方法论问题——序参量的选择是事后确认的，不能被当作预测工具——但这并不意味着「历史上没有发生真实的结构性转变」，只是说我们在事前无法精确定位这些转变。**不可逆性**的反驳针对动力学层面：物理相变保留时间反演对称（至少在统计力学框架内），而社会转变和能力习得均不可逆，因此两者的动力学机制不同构。

理解 crossover 与相变的区别，对解读 scaling 实验结论有直接意义：如果「涌现」只是 crossover，那么它的「陡峭程度」依赖模型细节，没有普适临界指数；换不同架构、不同任务，陡峭程度会不同。这意味着不能把某个模型上观察到的「涌现阈值」当作普遍的物理常数，也不能用它来预测另一个模型在什么规模下会出现同样的「涌现」。这与幂律的情况正好相反——幂律指数在不同架构间相对稳定，crossover 的陡峭程度则不然。在宣称「某能力在 N^* 规模时涌现」时，应当意识到这个 N^* 不是一个普适常数，而是特定评估指标与特定模型细节共同决定的系统参数。

严格

热力学极限的必要性。设系统有 N 个粒子（或参数）。自由能密度为 $f(N, T) = F(N, T)/N$ ，其中 T 为温度（或某控制参数）。严格意义上的相变（即 f 在某 T_c 处的非解析性）仅在热力学极限下存在：

$$f(T) = \lim_{N \rightarrow \infty} \frac{F(N, T)}{N}$$

对有限 N ， $f(N, T)$ 是有限个指数函数的对数之和，解析函数的有限和仍是解析函数，因此严格奇异点不存在。有限系统观察到的只是平滑的**交叉（crossover）**行为，而非奇异点。

大语言模型即使有 10^{12} 个参数，仍是有限系统；其「涌现」行为即使在某些指标下看似陡峭，在严格数学意义上也只是 crossover，而非相变。这一区别不仅是字面上的：crossover 没有普适临界指数，其「陡峭程度」依赖模型细节，无法从统计物理的普适类理论中推导出任何定量预测。

后视镜叙事的不可预测性。设文明状态变量为 $\mathbf{x}(t) \in \mathbb{R}^d$ ，「相变时刻」为 t^* 使得某序参量 $\phi(\mathbf{x}(t))$ 在 t^* 附近急剧变化。历史学家能在 $t > t^*$ 时用 ϕ 描述转变，但事前预测 t^* 需要知道 ϕ 的具体形式——而这在转变发生前往往未知。AI 语境中，「下一次涌现何时发生」同样无法从现有理论推导，Schaeffer 等工作进一步表明，若涌现是指标伪影，则「预测涌现」更无理论基础。

不可逆性。一阶相变（如水的液固转变）满足 Clausius-Clapeyron 方程，可沿相图逆向路径恢复原相；二阶相变（如铁磁相变）在翻转控制参数后同样可逆。文明相变（如工业化）和 AI 能力涌现均不满足时间反演对称性——已习得的能力无法通过逆向训练消除。这一不可逆性意味着，物理相变与 AI/文明涌现之间的类比缺乏动力学层面的同构，只是在序参量行为的表现形式上相似。

特别说明：有限 N 系统只能有 crossover。以上分析的核心结论值得单独强调：热力学极限 $N \rightarrow \infty$ 是严格相变的必要条件，任何有限参数量的神经网络，无论其涌现行为看起来多么陡峭，数学上均为 crossover，而非奇异点。这一限制不依赖于模型大小的具体数值，而是有限系统的普遍性质。

与原书呼应：原书 §5.10 明确指出「热力学极限是严格相变的必要条件，有限 N 系统只能有 crossover」，对应本节自由能密度对 N 极限的分析；「物理相变背后有严格统计力学理论；AI 涌现背后只有经验观察」，对应本节后视镜叙事与不可逆性的讨论。

本章对应原书第五章「量变引发的智能飞跃——统计物理与复杂系统涌现」（§5.1–§5.10）。主要参考文献：*Kaplan et al. (2020)*; *Hoffmann et al. (2022)*; *Epoch AI (2024)*; *Sardana et al. (ICML 2024)*; *Schaeffer, Miranda & Koyejo (NeurIPS 2023)*; *Nanda et al. (ICLR 2023)*; *Power et al. (2022)*; *Pesce, He & Caldearelli (2026)*; *Shumailov et al. (Nature 2024)*。

数学卷·第6章：硬件感知算法的数学——屋脊线 (Roofline)、I/O 复杂度与 FlashAttention 推导

导读：主本第六章以产业链视角讲述为什么 HBM 已成为万亿美元生意；本章在此基础上，逐一补全那些被主本略去的不等式推导。读者需要掌握矩阵分析、基础算法复杂度与变换器 (Transformer) 结构的预备知识（可参照数学卷第三章 §3.5 关于 FlashAttention 直觉版的铺垫）。本章的核心线索是：**带宽是常数，算力在增长，因此算法必须在数学层面不断”提升算术强度”，才能不被内存墙钉死。**

§6.1 Roofline 模型的形式化

这一节在讲什么

为什么一个 kernel 跑得很慢，根本原因可能不是代码写得差？硬件有两个独立的天花板：算力和带宽。Roofline 模型给你一把尺子，精确测量你的 kernel 撞上的是哪一面墙。

第一步：定义算术强度 I 。每搬运 1 字节数据，kernel 完成多少次浮点运算？这个比值就是算术强度。 I 低说明数据搬得多、算得少——内存墙是瓶颈； I 高说明数据复用充分——算力才是天花板。

第二步：两个上界取 $\min$ 。内存受限时性能上界是 $I \cdot \beta$ （带宽乘以算术强度）；计算受限时上界是 P_{peak} （硬件峰值算力）。两者相交的脊点 $I^* = P_{\text{peak}}/\beta$ 是分水岭。

第三步：看 GEMM 的两个极端。大矩阵乘的 $I \rightarrow \infty$ （计算受限）；矩阵-向量乘的 $I \approx 1$ （深度内存受限）。H100 的 $I^* \approx 295$ ，推理解码阶段与脊点相差 295 倍——这就是 HBM 带宽决定推理速度的数学根源。

本节要拿走的一件事：脊点 $I^* = P_{\text{peak}}/\beta$ 是判断 kernel 瓶颈的分界线；低于它的所有优化，只有提升算术强度才有根本效果，调 CUDA 核心利用率是徒劳的。

直觉

Roofline 模型的核心洞察是：一块 GPU 同时受到两个独立约束的限制——浮点峰值算力 P_{peak} 和 HBM 带宽 β ，而一个 kernel 的实际性能只能由这两者中更紧的那个决定。算术强度 I （FLOP 数除以字节搬运量）是衡量 kernel 「计算密度」的唯一关键指标： I 越大，说明每搬运一字节数据能完成的运算越多，越接近算力受限； I 越小，说明数据搬运是瓶颈。脊点 $I^* = P_{\text{peak}}/\beta$ 是两个区域的分界线，它的物理含义是：算力和带宽「刚好匹配」时的算术强度门槛。

矩阵乘是理解这个模型的最佳测试场。大方阵 (m, n, k 同阶增长) 的算术强度随规模线性增长，最终进入计算受限区；而 batch size = 1 时的矩阵-向量乘，算术强度约为 1 FLOP/byte，与 H100 脊点 295 相差近三百倍——这不是偶然，这正是为什么大语言模型推理（token-by-token 解码）的速度瓶颈始终在 HBM 带宽而非 Tensor Core 算力。提升 batch size 本质上是在提升矩阵乘的 m 维，从而把 kernel 从内存受限区推向计算受限区。

一个容易忽视的陷阱是：Roofline 模型给出的是**性能上界**，而不是性能预测。即使 kernel 处于计算受限区，也可能因为寄存器溢出、warp 调度不均、共享内存 bank conflict 等原因远低于 P_{peak} 。因此脊点的意义在于「排除」而非「保证」：一个 $I < I^*$ 的 kernel，无论你怎么优化计算逻辑，**理论上也不可能触及 P_{peak}** ；而一个 $I \geq I^*$ 的 kernel，才有资格谈「是否压榨出了算力」。

6.1.1 算术强度的精确定义

设某计算核 (kernel) 执行 W 次浮点运算，在此过程中从最慢存储层级（对现代 GPU 即 HBM）读写共计 Q 字节数据。定义**算术强度** (arithmetic intensity) 为

$$I = \frac{W}{Q} \quad [\text{FLOP/byte}].$$

Williams、Waterman 与 Patterson 在 2009 年的原始论文中使用术语「operational intensity」，本章与数学卷其余各章统一改用更通行的「算术强度」，两者同义。

若存储层级选取不同（如 L2 而非 HBM），则得到**层次化 Roofline** (hierarchical Roofline)，但本章

专注于 HBM 层级，以与主本第六章的产业链叙事对应。

6.1.2 性能上界与 Roofline 公式

设硬件的浮点峰值算力为 P_{peak} (FLOP/s)，HBM 带宽为 β (byte/s)。对一个算术强度为 I 的 kernel，其可达性能上界为

$$P \leq \min(P_{\text{peak}}, I \cdot \beta).$$

直觉：若 kernel 是**内存受限** (memory-bound)，则每秒可喂入算力单元的数据量为 β byte/s，携带算力仅 $I \cdot \beta$ FLOP/s；若 kernel 是**计算受限** (compute-bound)，则上限即为 P_{peak} 。两者取 min 给出严格上界。

6.1.3 脊点 (Ridge Point) 的推导

两个上界相交的算术强度称为**脊点** I^* ，由

$$I^* \cdot \beta = P_{\text{peak}}$$

解得

$$I^* = \frac{P_{\text{peak}}}{\beta}.$$

当 $I < I^*$ 时 kernel 内存受限；当 $I \geq I^*$ 时计算受限。

H100 SXM5 的具体数值：依据 NVIDIA 官方规格，H100 的 FP16 张量核 (Tensor Core) 峰值算力 $P_{\text{peak}} = 989$ TFLOPS (TF32, dense) 或 1,979 TFLOPS (FP16, dense)。主本与本章使用 TF32/FP16 混合精度训练场景中最常被引用的 989 TFLOPS 作为代表值，HBM3 带宽 $\beta = 3.35$ TB/s，因此

$$I_{\text{H100}}^* = \frac{989 \times 10^{12} \text{ FLOP/s}}{3.35 \times 10^{12} \text{ byte/s}} \approx 295 \text{ FLOP/byte}.$$

这意味着：一个算术强度低于 295 FLOP/byte 的 kernel 在 H100 上永远是内存受限的，无论如何优化 CUDA 核心的利用率都无济于事——**瓶颈在带宽，不在算力**。

6.1.4 GEMM 的算术强度分析

考虑矩阵乘 $C = AB$ ，其中 $A \in \mathbb{R}^{m \times k}$ ， $B \in \mathbb{R}^{k \times n}$ ， $C \in \mathbb{R}^{m \times n}$ 。

浮点运算数：每个输出元素 C_{ij} 需做 k 次乘加，共 $2mnk$ FLOP。

数据搬运量（假设矩阵在 HBM 中各读一次、 C 写一次）：

$$Q_{\text{GEMM}} = (mk + kn + mn) \times \text{sizeof(element)}.$$

忽略数据精度系数，定义算术强度为

$$I_{\text{GEMM}} = \frac{2mnk}{mk + kn + mn}.$$

大 batch 极限 ($m, n, k \rightarrow \infty$ 且同阶): 分子 $\sim 2n^3$, 分母 $\sim 3n^2$, 故 $I_{\text{GEMM}} \rightarrow \frac{2n}{3} \rightarrow \infty$ ——大矩阵乘是计算受限的, 不受内存墙威胁。

小 batch 极限 ($m = n = 1$, 即矩阵-向量乘):

$$I_{\text{GEMM}}|_{m=n=1} = \frac{2k}{k+k+1} = \frac{2k}{2k+1} \xrightarrow{k \rightarrow \infty} 1 \approx 1 \text{ FLOP/byte (FP32)}.$$

若采用 FP16 (每元素 2 byte), 则

$$I_{\text{GEMM}}|_{m=n=1, \text{FP16}} \approx \frac{2k}{2(2k+1)} \rightarrow 1 \text{ FLOP/byte}.$$

这一数值与 H100 脊点 295 相差 **295 倍**——LLM 推理中的 token-by-token (batch size = 1) 解码阶段, 矩阵乘完全内存受限, 这正是 HBM 带宽对 LLM 推理速度如此关键的根本原因。

§6.2 I/O 复杂度的下界 (Hong-Kung 1981)

这一节在讲什么

为什么需要一个关于「搬数据次数」的下界定理? 光优化算法是不够的——你需要知道最优算法的极限在哪里, 才能判断自己的实现还有多少空间。Hong 和 Kung 1981 年的定理给出了这个极限的数学表达式。

第一步: 把算法变成图, 把快速存储变成筹码。 计算依赖关系用有向无环图 (DAG) 表示, 快速存储 (SRAM) 大小为 S 颗红色卵石的容量。「红蓝卵石游戏」把「搬数据」这个物理动作变成了可以严格计数的博弈规则。

第二步: S-划分论证。 将整个计算过程按 I/O 次数切分为若干阶段, 每阶段至多 S 次 I/O, 因此每阶段的「外部边界」至多 $2S$ 个值。这个约束限制了每阶段能计算的节点数, 进而推出总 I/O 下界。

第三步: 对矩阵乘用 AM-GM 不等式。 快速存储中同时持有 N_A 个 A 元素、 N_B 个 B 元素、 N_C 个 C 元素时, 可完成的乘加数至多 $\sqrt{N_A N_B N_C} \leq (2S/3)^{3/2}$, 由此导出 $T_{\text{I/O}} = \Omega(n^3/\sqrt{M})$ 。

本节要拿走的一件事: $n \times n$ 矩阵乘的 I/O 下界是 $\Omega(n^3/\sqrt{M})$, 且分块 GEMM 恰好达到此界——这是算法最优性的信息论证明, 而非工程猜测。

直觉

Hong-Kung 定理的出发点是一个极其朴素的观察: 不管算法多聪明, 如果快速存储 (SRAM) 只能装 S 个数, 那么每次在快速存储和慢速存储 (HBM) 之间的「通勤」最多只能携带有限的计算价值。红蓝卵石游戏把这个物理约束翻译成了一套可以严格计数的博弈规则——红色卵石代表「数在快速存储里」, 蓝色卵石代表「数被持久化到慢速存储」, 每次搬移计一次 I/O。这样, 「最少需要搬多少次数据」就变成了一个有限博弈的最优策略问题, 可以用组合论证来给出下界。

S-划分论证是本节的核心技巧, 它的思路像是「信息传输的路费」: 把整个计算过程按每 S 次 I/O 切成一段一段, 每段的「入境」和「出境」加起来至多 $2S$ 个值。这 $2S$ 个值决定了这一段内部能完成多少有效计算——即 $H(2S, G)$, DAG 中在 $2S$ 个外部输入限制下最多能覆盖的节点数。段数乘以每

段最大计算量，必须至少等于总计算量，由此倒推出 I/O 总次数的下界。整个论证的优美之处在于：它对任意正确的计算顺序都成立，无论你采用什么花哨的循环调度，都逃不出这个下界。

对矩阵乘应用这个框架时，关键步骤是用 AM-GM 不等式估计 $H(2S, G)$ ：快速存储里同时持有 N_A 、 N_B 、 N_C 个元素时，最多完成 $\sqrt{N_A N_B N_C}$ 次乘加，而三者之和受 $2S$ 约束，所以乘积的立方根至多 $(2S/3)^{3/2}$ ，即 $H(2S, G) = O(S^{3/2})$ 。代入下界公式得 $T_{I/O} = \Omega(n^3/\sqrt{M})$ 。一个值得注意的地方是：这个下界是可达的——分块矩阵乘（block size $b \approx \sqrt{M/3}$ ）恰好实现 $\Theta(n^3/\sqrt{M})$ 次 I/O，说明分块算法在渐近意义下已是最优，不存在更聪明的数据布局能突破这个界。

6.2.1 计算模型与红蓝卵石游戏

两级存储模型：系统有容量无限的慢速存储（对应 HBM）和容量为 S 个字的快速存储（对应 SRAM/shared memory）。在任意时刻，快速存储中最多驻留 S 个值。一次 I/O 操作将一个值在两级存储之间搬移。

红蓝卵石游戏 (Red-Blue Pebble Game)：将算法表示为有向无环图 (DAG) $G = (V, E)$ ，节点对应计算值，边对应数据依赖。两种卵石的语义如下：

- **红色卵石**：该值当前在快速存储 (SRAM) 中；最多同时存在 S 颗。
- **蓝色卵石**：该值在慢速存储 (HBM) 中持久化。

博弈规则：1. **计算**：若某节点的所有前驱节点均有红色卵石，可在该节点放置红色卵石（计算一步）。2. **读 (HBM→SRAM)**：将蓝色卵石替换为红色卵石，计 1 次 I/O。3. **写 (SRAM→HBM)**：将红色卵石替换为蓝色卵石，计 1 次 I/O。4. **驱逐**：若快速存储已满，可无代价地移除某红色卵石（丢弃，若以后需要则须重新读入）。

目标：对 DAG 的所有输出节点放置红色卵石，同时使 I/O 操作总数 $T_{I/O}$ 最小。

6.2.2 关键引理：S-划分与路径数

定义 ($H(S, G)$ ——S-碎石数)：在 DAG G 中，使用 S 颗红色卵石可以覆盖的最大节点数称为 $H(S, G)$ ——即红蓝游戏的「路径复杂度」。更形式化地， $H(2S, G)$ 是 DAG 的 **2S-块流量 (2S-dominator set size)**：每个「流」至多穿过 $2S$ 个节点。

引理 6.1 (Hong-Kung 基本不等式)：设 DAG G 有 $|V|$ 个非输入节点，输入节点数为 $|\text{Inputs}(G)|$ 。则在 S 个红色卵石约束下，任意正确博弈策略满足

$$\left\lceil \frac{T_{I/O}}{S} \right\rceil \cdot H(2S, G) \geq |V| - |\text{Inputs}(G)|.$$

证明梗概 (S-划分论证)：

将整个博弈过程按 I/O 操作划分为**阶段 (phases)**，每个阶段包含至多 S 次 I/O。在阶段 t 开始与结束时，快速存储至多各含 S 个值，故阶段 t 的「边界」上至多有 $2S$ 个值（进入 + 离开）。

在阶段 t 内，所有被计算的节点形成一个子图；其所有外部输入（来自 HBM 读入或上一阶段留存）至多 $2S$ 个。由 $H(2S, G)$ 的定义，该子图中最多包含 $H(2S, G)$ 个节点。

阶段总数为 $\lceil T_{I/O}/S \rceil$ ，故

$$(\text{阶段数}) \times H(2S, G) \geq |V| - |\text{Inputs}(G)|,$$

即

$$\left\lceil \frac{T_{I/O}}{S} \right\rceil \geq \frac{|V| - |\text{Inputs}(G)|}{H(2S, G)}.$$

将两边乘以 S 即得引理。□

6.2.3 矩阵乘的 I/O 下界

定理 6.2 (矩阵乘 I/O 下界): 对 $n \times n$ 矩阵乘 $C = AB$ ，在快速存储大小为 M 的两级存储模型下，任意正确算法的 I/O 复杂度满足

$$T_{I/O} = \Omega\left(\frac{n^3}{\sqrt{M}}\right).$$

证明梗概:

矩阵乘的标准算法可由以下 DAG 表示：节点集为 $\{(i, j, k) : 1 \leq i, j, k \leq n\}$ （即 n^3 个部分乘积节点），共 $|V| = n^3$ 个内部节点（忽略 n^2 个累加节点的低阶项），输入节点为 A 与 B 的 $2n^2$ 个元素。

关键一步 · 估计 $H(2S, G)$: 若快速存储中驻留 N_A 个 A 的元素、 N_B 个 B 的元素、 N_C 个 C 的部分和，则在这批元素上能完成的 FMA 数至多为 $\sqrt{N_A N_B N_C}$ （此为 Irony、Toledo 与 Tiskin 于 2004 年给出的精确估计，Hong-Kung 原文以略松的论证给出相同渐近结果）。由 **AM-GM 不等式**，

$$N_A + N_B + N_C \leq 2S \implies \sqrt{N_A N_B N_C} \leq \left(\frac{2S}{3}\right)^{3/2} = O(S^{3/2}).$$

故 $H(2S, G) = O(S^{3/2})$ 。代入引理 6.1:

$$T_{I/O} \geq S \cdot \frac{n^3 - 2n^2}{H(2S, G)} = S \cdot \frac{\Theta(n^3)}{O(S^{3/2})} = \Omega\left(\frac{n^3}{S^{1/2}}\right) = \Omega\left(\frac{n^3}{\sqrt{M}}\right). \quad \square$$

可达性: 分块矩阵乘（block size $b = \lfloor \sqrt{M/3} \rfloor$ ）实现 $\Theta(n^3/\sqrt{M})$ 次 I/O，与下界匹配，故该下界是紧的（tight）。

硬件含义: H100 的 SRAM（每 SM 约 256 KB shared memory，108 个 SM）总计 ≈ 27 MB。对 $n = 4096$ （典型大语言模型的注意力维度量级）:

$$T_{I/O} = \Omega\left(\frac{4096^3}{\sqrt{27 \times 10^6}}\right) \approx \Omega\left(\frac{6.87 \times 10^{10}}{5196}\right) \approx \Omega(1.3 \times 10^7) \text{ word transfers}.$$

即使算法最优，大矩阵也必须反复在 HBM 和 SRAM 之间搬移数据——这不是实现缺陷，而是**信息论意义上的必然**。

§6.3 FlashAttention 的 I/O 分析 (Dao et al. 2022)

这一节在讲什么

为什么标准 Attention 慢得出奇，即使算力充足？因为它把一个 $N \times N$ 的中间矩阵反复写入再读出 HBM，而这个矩阵的大小是 $O(N^2)$ ——序列翻倍，I/O 增加四倍。FlashAttention 的贡献是在不改变数学结果的前提下，让这个矩阵永远不出现在 HBM 里。

第一步：在线 softmax 递推。 softmax 需要知道全行的最大值和指数和才能归一化。FlashAttention 的核心技巧是：把这两个统计量做成「流式更新」——看到新的一块 K, V 就更新，不需要等到读完全行。这个递推在数学上等价于完整 softmax，但无需把整行存到 HBM。

第二步：分块 tiling 把 SRAM 装满。 把 Q, K, V 分成若干小块，每块大小 $B \times d$ ，约束 $Bd = O(M)$ 保证小块能驻留 SRAM。这样每次内迭代的 HBM I/O 只是搬小块，而 $N \times N$ 矩阵的全局写入被彻底消除。

第三步：I/O 从 $\Theta(N^2)$ 降到 $\Theta(N^2 d^2 / M)$ 。 当 $M \gg d^2$ 时（H100 上约差 600 倍），同样的计算结果，HBM 搬运量缩小近两个数量级——这不是优化，是重新推导。

本节要拿走的一件事：FlashAttention 的加速来源不是近似，而是通过「在线归一化 + 分块 tiling」重排计算顺序，将 I/O 复杂度从 $\Theta(N^2)$ 降至信息论最优的 $\Theta(N^2 d^2 / M)$ 。

直觉

标准 Attention 的 I/O 瓶颈直白得令人沮丧：计算 $S = QK^\top$ 之后，这个 $N \times N$ 的矩阵必须写入 HBM；softmax 再从 HBM 读回来，写出归一化结果；最后 $O = PV$ 再读一次 P 。三步走， $N \times N$ 矩阵在 HBM 上被反复搬运，I/O 总量是 $\Theta(N^2)$ 。当 $N = 8192$ （典型长上下文场景），这相当于在每次前向传播中搬运约 500 MB 的中间状态——而 H100 的 SRAM 只有 27 MB，根本装不下，必须落地 HBM。

FlashAttention 的突破是认识到：softmax 的本质只是「计算每一行的最大值、指数和，再做除法」，这三个统计量可以在流式扫描中逐块更新，不需要等到整行数据就位。引理 6.3 的归纳证明说明，只要维护好 m_i （当前最大值）和 ℓ_i （当前指数和），就可以在扫描完所有 K, V 块之后得到与完整 softmax 完全相同的结果——误差为零，不是近似。这个在线统计量的递推是 FlashAttention 数学上最优雅的地方，它的形式看似复杂，实质就是「带数值稳定性的流式求和」。

从复杂度角度理解这个改进，关键在于块大小 B 的选择： $B = O(M/d)$ 恰好让每个 Q, K, V 块驻留 SRAM，从而把每次内迭代的 HBM I/O 从「写整个 $N \times N$ 矩阵」变成「搬运一个 $B \times d$ 小块」。总 I/O 变成 $(N/B)^2 \times O(Bd)$ ，代入 $B = O(M/d)$ 后得 $\Theta(N^2 d^2 / M)$ 。这个表达式与标准 Attention 的比值是 $\Theta(M/d^2)$ ——H100 上约 600 倍的渐近加速比。实测约 7.6 倍的实际加速，说明常数项和序列长度的有限性还有不小的影响，但渐近分析已经精准指出了方向。

6.3.1 标准注意力 (Attention) 的 I/O 复杂度

设序列长度 N ，head 维度 d ，查询/键/值矩阵 $Q, K, V \in \mathbb{R}^{N \times d}$ 。标准 attention 的前向计算分三步：

1. $S = QK^\top \in \mathbb{R}^{N \times N}$ ；写入 HBM。
2. $P = \text{softmax}(S) \in \mathbb{R}^{N \times N}$ ；从 HBM 读 S ，写回 P 。
3. $O = PV \in \mathbb{R}^{N \times d}$ ；从 HBM 读 P 和 V 。

各步主要 I/O：

步骤	读 (byte)	写 (byte)
$S = QK^\top$	$2Nd$	N^2
$\text{softmax}(S)$	N^2	N^2
$O = PV$	$N^2 + Nd$	Nd

总 $\text{I/O} = \Theta(Nd + N^2)$ 。

极限假设 $N \gg d$ (例如 $N = 8192, d = 128$) 下, **主导项为** N^2 ; 而 Q, K, V, O 的搬运仅为 $Nd \ll N^2$ 。这意味着标准 attention 的 I/O 瓶颈在于 $N \times N$ 注意力矩阵在 HBM 上的多次读写, 而非矩阵乘本身的计算。

6.3.2 在线归一化指数 (Softmax) 的递推公式

Dao 等人的核心技巧是**分块 tiling + 在线 softmax**, 使得 $N \times N$ 矩阵永远无需完整写入 HBM。设将 K, V 沿序列维度分成 $T_c = \lceil N/B_c \rceil$ 块, 每块大小 $B_c \times d$; 将 Q, O 分成 $T_r = \lceil N/B_r \rceil$ 块, 每块大小 $B_r \times d$ 。对 Q 的第 i 行块, 算法遍历 K, V 的所有列块, 维护以下三个**在线统计量**:

$$m_i = \max(m_{i-1}, \text{rowmax}(S_i)),$$

$$\ell_i = e^{m_{i-1}-m_i} \ell_{i-1} + \text{rowsum}(e^{S_i-m_i}),$$

$$O_i = \text{diag}(e^{m_{i-1}-m_i}) O_{i-1} + e^{S_i-m_i} V_i.$$

其中 $S_i = Q \cdot K_i^\top \in \mathbb{R}^{B_r \times B_c}$ 是当前块的注意力分数, $m_0 = -\infty$, $\ell_0 = 0$, $O_0 = 0$ 。

引理 6.3 (在线 softmax 正确性): 设总共有 T 个键块, 则经过所有 T 次递推后,

$$O_T = \text{diag}(\ell_T)^{-1} \cdot \sum_{j=1}^T e^{S_j-m_T} V_j = \text{softmax}(S_{\text{full}}) \cdot V,$$

其中 $S_{\text{full}} = [S_1, S_2, \dots, S_T]$ 为完整注意力分数行向量。

证明: 采用数学归纳法, 对块数 t 归纳。

基础: $t = 1$: $m_1 = \text{rowmax}(S_1)$, $\ell_1 = \text{rowsum}(e^{S_1-m_1})$, $O_1 = e^{S_1-m_1} V_1$ 。注意

$$\frac{O_1}{\ell_1} = \frac{e^{S_1-m_1} V_1}{\text{rowsum}(e^{S_1-m_1})} = \text{softmax}(S_1) V_1,$$

归一化后正确。✓

归纳步: 设在处理 $t-1$ 块后, 归一化输出

$$\hat{O}_{t-1} = \frac{O_{t-1}}{\ell_{t-1}} = \text{softmax}(S_{1:t-1}) V_{1:t-1}$$

成立 (归纳假设)。处理第 t 块时, 令 $m_t = \max(m_{t-1}, \text{rowmax}(S_t))$, 则

$$\ell_t = e^{m_{t-1}-m_t} \ell_{t-1} + \text{rowsum}(e^{S_t-m_t}).$$

注意 $e^{m_{t-1}-m_t} \ell_{t-1} = \text{rowsum}(e^{S_{1:t-1}-m_t})$ (指数中的 m_{t-1} 被消去), 故

$$\ell_t = \text{rowsum}(e^{S_{1:t}-m_t}).$$

同理,

$$O_t = e^{m_{t-1}-m_t} O_{t-1} + e^{S_t-m_t} V_t = e^{S_{1:t-1}-m_t} \cdot \mathbf{1} \cdot \hat{O}_{t-1} \cdot \ell_{t-1} + e^{S_t-m_t} V_t.$$

精确地,

$$\frac{O_t}{\ell_t} = \frac{\sum_{j=1}^t e^{S_j-m_t} V_j}{\text{rowsum}(e^{S_{1:t}-m_t})} = \text{softmax}(S_{1:t}) V_{1:t}. \quad \square$$

6.3.3 FlashAttention 的 I/O 复杂度

命题 6.4 (FlashAttention 前向 I/O 复杂度): 设 SRAM 大小为 M , 满足 $d \leq M \leq Nd$, 块大小取 $B_c = \Theta(M/d)$, $B_r = \Theta(M/d)$ 。FlashAttention 前向传播的 HBM I/O 复杂度为

$$T_{I/O}^{\text{FlashAttn}} = \Theta\left(\frac{N^2 d^2}{M}\right).$$

证明:

外循环遍历 $T_c = N/B_c$ 个键块, 内循环遍历 $T_r = N/B_r$ 个查询块。

- **内循环每次:** 从 HBM 读入 Q 块 ($B_r \times d$) + K_j, V_j 块 (各 $B_c \times d$), 计算 $S_i = QK_j^\top$ (纯 SRAM 内计算, 无额外 HBM I/O), 更新 O_i (SRAM 内), 写出 O 块 ($B_r \times d$)。每次内迭代 HBM I/O 量 = $O(B_r d + B_c d)$ 。
- **总 I/O:** 共 $T_r \times T_c = (N/B_r)(N/B_c)$ 次内迭代, 每次 $O(B_r d + B_c d)$, 故

$$T_{I/O} = O\left(\frac{N}{B_r} \cdot \frac{N}{B_c} \cdot (B_r + B_c) \cdot d\right).$$

取 $B_r = B_c = B$, 则

$$T_{I/O} = O\left(\frac{N^2}{B^2} \cdot 2Bd\right) = O\left(\frac{N^2 d}{B}\right).$$

消元条件 · SRAM 容量约束: SRAM 需同时容纳 Q, K, V 块及 O, S 块: $O(Bd)$ 个元素, 故约束 $Bd = O(M)$, 即 $B = O(M/d)$ 。代入:

$$T_{I/O}^{\text{FlashAttn}} = O\left(\frac{N^2 d}{M/d}\right) = O\left(\frac{N^2 d^2}{M}\right).$$

下界同理可得 $\Omega(N^2 d^2/M)$ (见原文 Theorem 2 的最优性证明), 故 $T_{I/O}^{\text{FlashAttn}} = \Theta(N^2 d^2/M)$ 。□

复杂度跃迁 · 这一步是全文的关键转折: 标准 attention 的 $\Theta(N^2)$ 被换写为 $\Theta(N^2 d^2/M)$ ——当 $M \gg d^2$ (H100 上 $M \approx 10^7$, $d^2 \approx 1.6 \times 10^4$), 同一计算结果的 HBM I/O 几乎下降两个数量级。I/O 复杂度不是被「优化」出来的, 是被「重新推导」出来的。

6.3.4 FlashAttention vs. 标准 Attention 的复杂度比较

$$\frac{T_{I/O}^{\text{standard}}}{T_{I/O}^{\text{FlashAttn}}} = \frac{\Theta(Nd + N^2)}{\Theta(N^2 d^2 / M)} = \Theta\left(\frac{M}{d^2} \cdot \frac{Nd + N^2}{N^2}\right) \underset{N \gg d}{\approx} \Theta\left(\frac{M}{d^2}\right).$$

代入 A100 的 SRAM 大小 $M \approx 20 \text{ MB} = 20 \times 10^6 \text{ byte}$, head dimension $d = 128$, 每 FP16 元素 2 byte, 则 $M \approx 10^7$ 元素, $d^2 = 16384$, 故加速比约 $\Theta(10^7/16384) \approx 610$ 倍 (渐近)。实测加速约 7.6 倍 (主要原因是序列长度有限, 常数项不可忽略)。

6.3.5 FlashAttention-2 的工作划分 (Work Partitioning) 改进

Dao (2023) 在 FlashAttention 的基础上提出三项改进, 不改变 I/O 复杂度量级, 但大幅提升硬件利用率:

1. **减少非矩阵乘 FLOP**: FlashAttention 原版的缩放与 rescaling 操作引入额外逐元素运算; FA-2 重新推导递推公式, 将 $\text{diag}(e^{m_{i-1}-m_i})$ 的显式展开推迟到最终输出时执行, 减少中间 rescaling 次数。
2. **序列维度并行化**: FA-1 仅对 batch 和 head 维度并行调度 thread block, 导致长序列小 batch 时 GPU 占用率 (occupancy) 低。FA-2 对外循环 (T_r 个行块) 额外并行, 使 SM 利用率从约 40% 提升至 50–73%。
3. **Warp 内 Work Partitioning**: FA-1 将 K, V 切分至 4 个 warp (「split-K」方案), 各 warp 需写中间结果至 shared memory 后同步累加; FA-2 改为将 Q 切分至 warp, K, V 共享, 消除 warp 间同步, 降低 shared memory 访问延迟。

这三项改进使 FA-2 在 A100 上达到理论峰值的 50–73%, 在 H100 上可达 335 TFLOPS (FlashAttention-2 本身无需特殊 Hopper 指令), 是截至 2023 年实现最高 GPU 利用率的 attention 实现之一。

§6.4 环形注意力 (RingAttention) 的通信复杂度 (Liu et al. 2024)

这一节在讲什么

为什么把 FlashAttention 直接搬到多卡上会出问题? 单卡 SRAM 装不下超长序列的 K, V , 多卡分片后每张卡的 Q_p 需要访问所有卡的 K, V ——朴素的 AllGather 会让每张卡内存随 N 线性增长, 同时通信量也不随卡数增加而分摊。RingAttention 用环形流水线同时解决这两个问题。

第一步: 把 GPU 排成环, K/V 块像接力棒一样传递。 每一步, 每张卡把当前的 K_j, V_j 发给下一张卡, 同时接收上一张卡传来的新块, 并用当前块更新本地的在线 softmax 统计量。计算与通信同步进行。

第二步: 总通信量与卡数无关。 P 张卡跑 $P-1$ 步, 每步传 $(N/P) \times d$ 个元素, 总计 $\approx Nd$ 元素——和只有 1 张卡时搬运整个 K, V 的量相同。卡越多, 每步通信越少, 总量不变。

第三步: 完全重叠的条件是「局部脊点」。 要让通信被计算完全掩盖, 每卡局部序列长度 N/P 必须不低于 $2F/\beta_{\text{NVLink}}$ ——这正是单卡 Roofline 脊点在多卡通信场景下的类比。

本节要拿走的一件事: RingAttention 的通信复杂度是 $\Theta(Nd)$, 与卡数 P 无关; 但完全重叠所要求的「局部脊点」给出了 P 的理论上限 $P_{\max} \approx N\beta_{\text{NVLink}}/(2F)$ 。

直觉

超长序列 (百万 token 量级) 的 Attention 需要多卡协作, 根本原因不只是计算量, 更是内存: 单张 H100 的 HBM 只有 80 GB, 而 $N = 10^6$ 、 $d = 128$ 、FP16 下的 K, V 矩阵各占约 256 GB——远超单卡容量。RingAttention 的设计目标是: 让每张卡只需要持有 $1/P$ 的序列, 同时仍然能计算出完整的、精确的 Attention 输出, 而不是近似或截断。这个目标的实现依赖于 §6.3 中的在线 softmax 递推: 只要统计量 m_p 和 ℓ_p 可以流式更新, 就可以把「遍历全局 K, V 」拆分成「一块一块地接收并更新」, 而无需一次性看到完整的 K, V 。

环形通信方案的数学优雅之处在于: 每张卡每步只需收发大小为 $(N/P) \times d$ 的矩阵, 总通信量 $(P-1) \times 2(N/P) \times d \approx 2Nd$, 与卡数 P 完全无关。这意味着添加更多卡不会增加总通信量, 只是把每步的通信量等比缩小。对比朴素的 AllGather 方案 (每张卡都要收全局 K, V , 通信量正比 P), RingAttention 在通信复杂度上有本质优势。直觉上, 这类似于「流水线」工厂: 产品在工位间依次传递, 总传递次数等于工位数, 而不是每个工位各自复制一份完整产品。

然而, 通信量与卡数无关并不意味着可以无限加卡。完全重叠的条件要求每步计算时间不短于通信时间, 推导出 $N/P \geq 2F/\beta_{\text{NVLink}}$, 即每卡局部序列长度不得过短。这个不等式在形式上与 Roofline 的脊点条件同构: F/β_{NVLink} 是一个「多卡版算术强度阈值」, 超过 $P_{\max}$ 后每步计算来不及掩盖通信, 系统陷入「通信受限」状态, 与单卡内存受限具有完全相同的数学结构。这提示了一个更深的统一性: 无论是单卡的 Roofline, 还是多卡的 RingAttention 重叠条件, 本质上都是「两种资源 (算力 vs 数据搬运) 的速率匹配」问题。

6.4.1 多卡场景设定

设总序列长度 N , 分布在 P 张 GPU 上, 每张 GPU 持有

$$Q_p \in \mathbb{R}^{(N/P) \times d}, \quad K_p, V_p \in \mathbb{R}^{(N/P) \times d}, \quad p = 1, 2, \dots, P.$$

完整注意力需要每张 GPU 的 Q_p 对全局 K, V 做 attention, 即 $O_p = \text{softmax}(Q_p K^\top / \sqrt{d}) V$ 。

朴素做法: 先用 AllGather 将完整 K, V (各 Nd 元素, $2Nd \cdot \text{sizeof}$ byte) 汇聚至每张卡, 通信量 $\Theta(Nd)$ 字节。这与 P 无关, 但实际上每张卡需存储完整 K, V , 内存随 N 增长, 无法扩展至超长序列。

6.4.2 环形通信方案

RingAttention 将 P 张 GPU 排列为**逻辑环**, 在计算与通信之间实现**流水线重叠 (pipeline overlap)**:

第 t 步 ($t = 1, 2, \dots, P-1$):

- GPU p 将其持有的当前 K_j, V_j (大小各 $(N/P) \times d$) 发送至 GPU $p+1 \pmod{P}$, 同时从 GPU $p-1 \pmod{P}$ 接收下一批 K, V 块。
- 发送/接收过程中, GPU p 同时用 FlashAttention 计算当前 K_j, V_j 对本地 Q_p 的贡献, 更新在线 softmax 统计量。

每步通信量: 发送 K_j 和 V_j 各 $(N/P) \times d \times \text{sizeof} = 2 \cdot (N/P) \cdot d$ 元素。

总通信量 ($P-1$ 步):

$$T_{\text{comm}} = (P-1) \cdot 2 \cdot \frac{N}{P} \cdot d \cdot \text{sizeof} \approx 2Nd \cdot \text{sizeof},$$

即总通信量 $\Theta(Nd)$ 字节, **不依赖** P ——设备数越多, 每步通信量越小, 总量不变。

6.4.3 计算与通信重叠条件

每步**计算时间**: GPU p 需完成 Q_p (大小 $N/P \times d$) 与 K_j (大小 $N/P \times d$) 的矩阵乘, 计算量为 $2(N/P)^2 d$ FLOP, 耗时

$$t_{\text{comp}} = \frac{2(N/P)^2 d}{F},$$

其中 F 为单卡浮点算力 (FLOP/s)。

每步**通信时间**: 传输 $2 \times (N/P) \times d$ 个元素 (FP16, 每元素 2 byte), 耗时

$$t_{\text{comm}} = \frac{4(N/P)d}{\beta_{\text{NVLink}}},$$

其中 β_{NVLink} 为单向 NVLink 带宽 (H100 SXM: ≈ 450 GB/s 单向)。

完全重叠条件 (计算时间 $\geq$ 通信时间):

$$\frac{2(N/P)^2 d}{F} \geq \frac{4(N/P)d}{\beta_{\text{NVLink}}},$$

化简得

$$\frac{N}{P} \geq \frac{2F}{\beta_{\text{NVLink}}}.$$

即每卡持有的局部序列长度 N/P 不得小于 $2F/\beta_{\text{NVLink}}$ (类似一个「局部 ridge point」)。

6.4.4 环路最大规模推导

由上式解出 P 的上界：

$$P \leq \frac{N \cdot \beta_{\text{NVLink}}}{2F} = \frac{N \cdot \beta_{\text{NVLink}}}{2F}.$$

若希望通信完全被计算掩盖，GPU 数量不得超过

$$P_{\max} = \left\lfloor \frac{N \cdot \beta_{\text{NVLink}}}{2F} \right\rfloor.$$

H100 SXM5 的具体数值： $F = 989 \times 10^{12}$ FLOP/s (TF32), $\beta_{\text{NVLink}} = 450 \times 10^9$ byte/s (单向), 序列长度 $N = 1,000,000$ (百万 token):

$$P_{\max} = \frac{10^6 \times 450 \times 10^9}{2 \times 989 \times 10^{12}} \approx \frac{4.5 \times 10^{17}}{1.978 \times 10^{15}} \approx 227.$$

即对百万 token 的序列，约 227 张 H100 可令 RingAttention 通信完全被计算掩盖。超过此数目时通信将成为瓶颈，需进一步的通信压缩或序列分片策略。

与 Roofline 的类比： $P_{\max}$ 的公式 $\sim \sqrt{N \cdot F / \beta}$ (若同时考虑计算量正比 N^2/P^2 、通信量正比 N/P) 提示了一个多卡版本的「ridge point」：设备数增多使局部计算强度下降，超过 $P_{\max}$ 后即陷入「通信受限」状态，与单卡内存受限具有同构的数学结构。

§6.5 算术强度的演化：从 V100 到 B200

这一节在讲什么

为什么每一代 GPU 升级，「需要多高的算术强度才能用满算力」这个门槛也在升高？算力增速长期快于带宽增速，脊点 $I^* = P_{\text{peak}}/\beta$ 就随之上移——上一代计算受限的 kernel，在这一代可能已经变成内存受限。

第一步：量化每一代的增速比。从 V100 到 H100，算力增长了约 7.9×，带宽增长了约 3.7×，比值 > 1 使脊点从 139 升至 295。理解这个比值比记住绝对数字更有价值。

第二步：B200 是一个有趣的例外。Blackwell 架构首次出现带宽增速 (2.39×) 超过算力增速 (2.27×) 的情形，脊点从 295 微降至 281。这不是退步，而是 HBM3e 在本代实现了相对更大幅度的带宽跃升。

第三步：量化对 I 的影响。从 FP16 换到 FP8，相同计算量下字节搬运减半， $I \rightarrow 2I$ ——相当于「人工提升算术强度」。这和 FlashAttention 通过 tiling 减少 HBM 访问是同一逻辑的不同实现。

本节要拿走的一件事：脊点随代际上移意味着算法必须持续「主动提升 I 」——更大 tile、算子融合、低精度量化，三条路共同指向同一个目标：在带宽不变的约束下，从每一字节数据中榨取更多计算。

直觉

Roofline 模型的脊点 $I^* = P_{\text{peak}}/\beta$ 不是一个固定常数，它随着每一代 GPU 架构的迭代而移动。历史数据表明，从 V100 到 H100 的三代中，算力增长始终快于带宽增长（增速比均大于 1），脊点从 139 单调上升到 295。这个趋势有一个深刻含义：曾经在旧架构上「计算受限」的 kernel，在新架构上可能已经滑入「内存受限」区——哪怕你的代码一字未改，新硬件反而让它变慢了（相对于峰值的比例下降）。算法设计者必须意识到，提升算术强度不是一次性工作，而是随硬件迭代持续进行的。

从数字上看，V100 的脊点 139 意味着「只要算术强度超过 139，就能用满算力」；H100 把这个门槛推高到 295，几乎翻倍。这对 Transformer 的核心算子有直接影响：大矩阵乘（GEMM）的算术强度随矩阵规模增长，在足够大的 batch 下依然计算受限；但推理解码时 batch size 通常为 1，矩阵-向量乘的 $I \approx 1$ ，相比任何一代的脊点都低了两个数量级以上，永远内存受限。B200 的带宽增速首次超过算力增速是个值得注意的信号：它提示硬件厂商也意识到了「带宽滞后」的瓶颈，开始在 HBM 带宽上投入更多资源。

量化（从 FP16 到 FP8 或 FP4）是「提升算术强度」的第三条路，与算法层面的 tiling 和算子融合并列。换成 FP8 后，相同矩阵的字节搬运量减半，算术强度翻倍，相当于把 kernel 在 Roofline 图上向右平移了一倍——有可能从内存受限区越过脊点进入计算受限区。H100 在 FP8 精度下的脊点约达 1181 FLOP/byte，这意味着即便是相当密集的计算任务，也仍然需要精心设计的 tiling 才能真正触及计算上界。三条路（更大 tile、算子融合、低精度量化）在数学上指向同一目标：让每一字节的 HBM 访问对应更多的有效 FLOP。

6.5.1 各代 GPU 的 Roofline 参数

以下数据来源于 NVIDIA 官方规格表与公开基准测试，均为 dense（无稀疏加速）FP16 Tensor Core 性能：

GPU	架构	P_{peak} (TFLOPS, FP16)	β (TB/s, HBM)	$I^* = P_{\text{peak}}/\beta$ (FLOP/byte)
V100	Volta	125	0.90	139
SXM2	(2018)			
A100	Ampere	312	2.00	156
SXM4	(2020)			
H100	Hopper	989 [†]	3.35	295
SXM5	(2022)			
B200	Blackwell	1250 [‡]	8.00 [§]	281
SXM	(2025)			

[†] 989 TFLOPS 为 TF32 dense；FP16 dense 为 1,979 TFLOPS，对应 $I^* \approx 590$ FLOP/byte。本章与主本一致，使用 TF32 基准以便对比训练场景。

[‡] B200 FP16 Tensor Core dense 值；FP4 dense 可达 9,000 TFLOPS，仅用于推理量化场景，不计入此处 Roofline 对比。

[§] B200 HBM3e 带宽官方数值因文献版本不同在 7.7–8.0 TB/s 之间；本章取 8.0 TB/s。

6.5.2 Ridge Point 的单调上行论证

命题 6.5 (Ridge Point 单调上行)：设在相邻两代 GPU 之间，算力增长因子为 $\alpha_F = P_{\text{peak}}^{(t+1)}/P_{\text{peak}}^{(t)}$ ，带宽增长因子为 $\alpha_\beta = \beta^{(t+1)}/\beta^{(t)}$ 。若 $\alpha_F > \alpha_\beta$ ，则脊点单调上升：

$$I^{*(t+1)} = \frac{P_{\text{peak}}^{(t+1)}}{\beta^{(t+1)}} = I^{*(t)} \cdot \frac{\alpha_F}{\alpha_\beta} > I^{*(t)}.$$

验证：

- V100 → A100: $\alpha_F = 312/125 = 2.50$, $\alpha_\beta = 2.00/0.90 = 2.22$, 比值 $1.12 > 1$ 。 I^* 从 139 升至 156。✓
- A100 → H100: $\alpha_F = 989/312 = 3.17$, $\alpha_\beta = 3.35/2.00 = 1.68$, 比值 $1.89 > 1$ 。 I^* 从 156 升至 295。✓
- H100 → B200: $\alpha_F = 2250/989 = 2.27$, $\alpha_\beta = 8.00/3.35 = 2.39$, 比值 $0.95 < 1$ 。 I^* 从 295 微降至 281。

注意 B200 的带宽增速 ($2.39\times$) 首次超过算力增速 ($2.27\times$)，脊点出现轻微下降。这并非 Blackwell 架构设计失误，而是 HBM3e 技术在本代实现了相对更大幅度的带宽跃升（叠片数增加、引脚速率 8 Gbps vs. 5.23 Gbps）。

结构性含义：从历史趋势看，V100→H100 三代间 α_F/α_β 均大于 1，脊点从 139 升至 295，**翻倍有余**。这意味着：

1. 曾经在 V100 上计算受限的 kernel，在 H100 上可能已变为内存受限（因为脊点右移，需要更高算术强度才能逃离内存墙）。
2. 算法设计必须不断「主动提升 I 」——或通过更大 tile（如 FlashAttention-2 的更大 B_r, B_c ），或通过算子融合（kernel fusion），或通过量化（FP8/FP4）降低数据搬运字节数。

量化对 I 的影响：若将 FP16（2 byte/元素）改为 FP8（1 byte/元素），相同计算量下字节搬运减半，算术强度 $I \rightarrow 2I$ 。结合 H100 FP8 下 3,958 TFLOPS 的峰值，脊点

$$I_{\text{H100, FP8}}^* = \frac{3958}{3.35} \approx 1181 \text{ FLOP/byte}.$$

在此精度下，即使是 batch size = 1 的矩阵-向量乘 ($I \approx 2$) 依然深度内存受限，带宽仍然是主要瓶颈。

§6.6 反类比与边界

这一节在讲什么

为什么一个精确的数学模型也需要「反类比」章节？因为 Roofline、Hong-Kung 定理、FlashAttention 的 I/O 分析各自都有隐含假设，在这些假设失效的场景下，模型的预测会系统性地偏乐观。知道边界在哪里，和知道模型本身一样重要。

第一步：Roofline 的三个假设局限。它假设（1）内存访问与计算可以完全流水——实际有延迟隐藏限制；（2）用单个 β 代表所有层级——L1/L2/shared memory 与 HBM 差 3-4 个数量级；（3）所有浮点运算同质——逐元素运算走 CUDA Core，吞吐低 4-8 倍。

第二步：Hong-Kung 定理的适用边界。它隐藏了常数因子（小矩阵时常数优化比渐近改进更重要），假设单一快慢存储层级（多级存储需要 Ballard et al. 2012 的扩展），且只适用于确定性算法。

第三步：FlashAttention 和 RingAttention 的隐含前提。FlashAttention 要求 $d \leq M \leq Nd$ ；若 $M < d$ ，分块策略失效。RingAttention 要求通信与计算可以完全重叠，实际中 NCCL 延迟和 InfiniBand 带宽共享往往使 $P_{\max}$ 远低于理论值。

本节要拿走的一件事：本章每个模型都有一个「适用区间」，超出区间时应转向更精细的工具（层次化 Roofline、多级 I/O 下界、实测 profiling），而不是强行套用渐近结论。

直觉

数学模型的强大之处在于它的简洁性，但简洁性来自抽象——忽略某些细节以换取可分析性。Roofline 模型把复杂的 GPU 微架构压缩成两个数字（ P_{peak} 和 β ），Hong-Kung 定理把多级存储层级压缩成一快一慢，FlashAttention 的 I/O 分析把 SRAM 当作均匀、无延迟的存储。在这些简化成立的范围内，这些模型极为有用；一旦跨出边界，它们的预测就会系统性地偏乐观——而工程师如果不知道边界在哪里，往往会把「理论加速比 600 倍」和「实测加速比 7.6 倍」之间的差距归结为「实现不够好」，而不是「模型本身的假设在此失效」。

Roofline 最常被忽视的局限是延迟隐藏（latency hiding）：HBM 的访问延迟约 500–800 ns，要充分利用带宽，必须同时有足够多的 outstanding memory requests「在途」，这要求 warp 数量和内存访问模式满足特定条件。如果 kernel 的工作量太小，或者内存访问模式导致 bank conflict，实际带宽利用率可能远低于 β ，Roofline 的「内存受限上界」就高估了实际性能。类似地，逐元素密集型 kernel（如 softmax、LayerNorm）中大量使用超越函数（exp、sqrt），这些运算走 CUDA Core 而非 Tensor Core，吞吐量低 4–8 倍，而 Roofline 的 P_{peak} 通常指 Tensor Core 峰值，对这类 kernel 完全不适用。

Hong-Kung 定理的「信息论必然」有时被误读为「下界就是实际性能的参照」。需要强调的是， $\Omega(\cdot)$ 符号隐藏了常数因子，且这些常数在小规模时不可忽略——对 $n \leq 512$ 的矩阵乘，常数项的差异可能比渐近阶更重要。FlashAttention 分析中的 $d \leq M$ 假设在 head dimension 极小（如某些线性注意力变体中 $d = 16$ ）或 SRAM 受极端限制时可能不成立，此时分块策略退化，需要重新推导适合该参数范围的算法。这些边界案例说明，「理解定理的证明」和「理解定理的适用条件」同样重要，甚至后者在工程实践中更为关键。

6.6.1 Roofline 模型的假设局限

假设一：完美重叠（perfect overlap）。Roofline 模型假设内存访问与计算可以完全流水，实际 GPU 中存在 latency hiding 限制：若 outstanding memory requests 不足以填满内存子系统的延迟（约

500–800 ns on HBM), 则 kernel 会出现 latency-bound 而非纯 bandwidth-bound 的停顿。此时实际带宽利用率可能显著低于 β , Roofline 的预测偏于乐观。

假设二：单一带宽值。标准 Roofline 用单个 β 代表所有内存访问, 忽视了 L1 cache、L2 cache、shared memory 与 HBM 之间 **3–4 个量级**的带宽差异。层次化 Roofline (hierarchical Roofline) 可缓解此问题, 但引入更多硬件参数, 分析复杂度成倍增加。

假设三：FP 运算同质。峰值算力 P_{peak} 通常指 Tensor Core 的矩阵乘算力, 而逐元素运算 (softmax 中的 exp、LayerNorm 等) 需走 CUDA Core, 吞吐量低 4–8 $\times$ 。对于逐元素密集的 kernel, 实际峰值远低于 P_{peak} 。

6.6.2 Hong-Kung 下界的适用范围

限制一：渐近下界, 常数不可忽略。定理 6.2 中的 $\Omega(\cdot)$ 隐藏了常数因子。分块 GEMM 的实际 I/O 为 $2\sqrt{3}n^3/\sqrt{M}$ (Irony et al. 2004 给出), 而 cuBLAS 的实现常数约为 2–4, 因此对小矩阵 ($n \leq 512$), 渐近下界的渐近优化意义有限, 常数项的优化往往更关键。

限制二：顺序存储层级假设。Hong-Kung 定理假设单一快速存储与单一慢速存储, 现代 GPU 的存储层级为 **Register $\rightarrow$ L1/Shared Memory $\rightarrow$ L2 Cache $\rightarrow$ HBM**, 各级之间存在非均匀延迟与带宽。真实 I/O 下界应针对多级存储层级分别建立, 见 Ballard 等 (2012) 对多级层次的扩展。

限制三：仅适用于确定性算法。对于随机化算法 (如随机稀疏注意力), Hong-Kung 框架需要修正, 访问模式的随机性可能允许绕过某些下界。

6.6.3 FlashAttention I/O 分析的隐含前提

FlashAttention 的 $\Theta(N^2d^2/M)$ 复杂度依赖 $d \leq M \leq Nd$ 的假设。若 $M < d$ (极端 SRAM 受限), 则整个 head 无法驻留 SRAM, 分块策略失效, 复杂度退化至 $\Theta(N^2)$ 。反之若 $M \geq Nd$ (即整个 K, V 可放入 SRAM), 则标准 attention 的第 1–3 步均可在 SRAM 内完成, I/O 降至 $\Theta(Nd)$, FlashAttention 的优势消失。

FlashAttention 的最优性定理 (Dao et al. 2022, Theorem 2) 同时给出了下界 $\Omega(N^2d^2/M)$, 证明**不存在**在所有 M 范围内均优于 FlashAttention 的精确 attention 算法——它在渐近意义下已是最优的。

6.6.4 RingAttention 的通信模型假设

本章 §6.4 的推导假设: (1) NVLink 为**全双工、无竞争**的点对点通信; (2) 计算与通信可以**完全重叠** (需要硬件对计算流与通信流的并发调度支持, 如 CUDA Stream 与 NCCL 的集成); (3) 序列在设备间均匀分布 (causal masking 下负载不均衡, Striped Attention 等变体通过重排 token 索引加以缓解)。

在实际部署中, 由于 NCCL 通信延迟、InfiniBand 带宽共享等因素, 可达 P_{max} 可能远低于理论值, 需通过 profiling 工具 (如 NVIDIA Nsight Systems) 实测通信/计算重叠率。

§6.7 参考文献

这一节在讲什么

为什么这份文献表按此顺序排列？前两篇（Hong-Kung 1981、Williams et al. 2009）是本章两条主线的理论奠基；之后的 Dao 等人系列给出 FlashAttention 及其改进的原始证明；Liu et al. 2024 对应 RingAttention；Irony et al. 2004 和 Ballard et al. 2012 给出 I/O 下界的精确常数与多级推广。最后几篇提供硬件规格来源、在线 softmax 算法出处，以及 Striped Attention 等工程变体的参考。凡本章引用数值或引理，均可在此找到一手来源。

本节要拿走的一件事：每个推论的数学权威出处都在这里——遇到疑问时，先查对应原文，而不是依赖二手综述。

1. **Hong, J.-W., & Kung, H. T.** (1981). I/O complexity: The red-blue pebble game. *Proceedings of the 13th Annual ACM Symposium on Theory of Computing (STOC)*, 326–333. <https://doi.org/10.1145/800076.802486>
2. **Williams, S., Waterman, A., & Patterson, D.** (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 65–76. <https://doi.org/10.1145/1498765.1498785>
3. **Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C.** (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems (NeurIPS)* 35. <https://arxiv.org/abs/2205.14135>
4. **Dao, T.** (2023). FlashAttention-2: Faster attention with better parallelism and work partitioning. *International Conference on Learning Representations (ICLR 2024)*. <https://arxiv.org/abs/2307.08691>
5. **Liu, H., Zaharia, M., & Abbeel, P.** (2024). Ring attention with blockwise transformers for near-infinite context. *International Conference on Learning Representations (ICLR 2024)*. <https://arxiv.org/abs/2310.01889>
6. **Irony, D., Toledo, S., & Tiskin, A.** (2004). Communication lower bounds for distributed-memory matrix multiplication. *Journal of Parallel and Distributed Computing*, 64(9), 1017–1026. <https://doi.org/10.1016/j.jpdc.2004.03.021>
7. **Ballard, G., Demmel, J., Holtz, O., & Schwartz, O.** (2012). Graph expansion and communication costs of fast matrix multiplication. *Journal of the ACM*, 59(5), 1–23. <https://doi.org/10.1145/2344422.2344424>
8. **Aggarwal, A., & Vitter, J. S.** (1988). The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9), 1116–1127. <https://doi.org/10.1145/48529.48535>
9. **Milakov, M., & Gimelshein, N.** (2018). Online normalizer calculation for softmax. *arXiv preprint*. <https://arxiv.org/abs/1805.02867>
10. **NVIDIA Corporation.** (2022). NVIDIA H100 Tensor Core GPU Architecture White Paper. <https://resources.nvidia.com/en-us-tensor-core>
11. **NVIDIA Corporation.** (2025). NVIDIA B200 GPU Product Specifications. <https://www.nvidia.com/en-us/data-center/b200/>

12. Brandon, W., Mishra, M., Nrusimha, A., Panda, R., & Ragan-Kelley, J. (2023). Striped attention: Faster ring attention for causal transformers. *arXiv preprint*. <https://arxiv.org/abs/2311.09431>
13. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)* 30. <https://arxiv.org/abs/1706.03762>
14. Demmel, J., Grigori, L., Hoemmen, M., & Langou, J. (2012). Communication-optimal parallel and sequential QR and LU factorizations. *SIAM Journal on Scientific Computing*, 34(1), A206–A239. <https://doi.org/10.1137/080731992>

本章与主本第六章相互镜像：主本讲产业逻辑（HBM 为何成为万亿生意、CoWoS 封装与超大规模云厂商（Hyperscaler）采购），本章专注数学骨干——不等式如何推、下界如何建立、算法如何在严格意义上最优。两章合读方能得到完整的「硬件感知」图景。

数学卷·第7章：后变换器（Transformer）时代的数学——状态空间模型、线性注意力与扩散语言模型的数理基础

说明：本章为《算法的低语·数学卷》的第七章，专注严格数学推导。与主本第七章的架构评述相互补充，读者需具备线性代数、常微分方程与概率论的基础知识。本章中所有内联公式使用 $\cdot$ 记号，展示公式使用独立段落。

§7.1 状态空间模型的连续形式

这一节在讲什么

为什么要用「状态空间」描述序列？序列模型面临一个根本矛盾：历史无限长，但内存和算力有限。状态空间模型的答案是：把无限长的历史压缩进固定大小的向量 $h(t)$ ，并用一条微分方程来维护它。

第一步：线性递推即连续极限。工程师熟悉的指数加权滑动平均 $h_t = \alpha h_{t-1} + \beta x_t$ ，令时间步长 $\Delta t \rightarrow 0$ ，就自然变成一阶线性 ODE $h'(t) = Ah(t) + Bx(t)$ 。连续化不是魔法，是「让步长无穷小」的极限。

第二步：闭式解揭示卷积结构。ODE 的解是 $h(t) = e^{At}h(0) + \int_0^t e^{A(t-s)}Bx(s)ds$ ，输出 $y(t) = Ch(t)$ 因此等于输入 x 与卷积核 $K(t) = Ce^{At}B$ 的卷积。这一结构使训练可以用 FFT 在 $O(L \log L)$ 内完成。

第三步：选什么样的 A 至关重要。 A 的特征值决定了系统的记忆长度和稳定性；什么样的 A 能让状态 $h(t)$ 最优地压缩历史，正是下一节 HiPPO 框架要回答的核心问题。

本节要拿走的一件事：状态空间模型的本质是「用一条线性 ODE 把无限历史压进固定向量」——卷积核 $K(t) = Ce^{At}B$ 将这一想法的所有信息都藏在矩阵 A 里。

直觉

序列建模的核心矛盾在于：历史序列可以无限长，但模型的计算和内存必须有限。循环神经网络的解法是维护一个「缓存状态」 h_t ，每一步用递推更新它；Transformer 的解法是存下所有历史并用注意力机制查询。状态空间模型走的是第一条路，但给它加上了严格的数学基础：用一阶线性常微分方程来刻画状态如何演化。这与工程领域的「指数加权滑动平均」完全同源——把时间步长推向零，滑动平均就变成了连续 ODE。

连续化的好处是数学工具更丰富。一旦把递推写成 ODE，就可以用矩阵指数 e^{At} 求闭式解，进而发现「输出 = 输入 * 卷积核」的结构。卷积核 $K(t) = Ce^{At}B$ 完全由 A, B, C 三个参数矩阵决定。LTI（线性时不变）假设保证了这个卷积核不随时间变化，因此整个序列的计算可以批量用 FFT 完成，训练效率大幅提升。

这里有一个常见的混淆点：「连续时间」并不意味着输入必须是连续信号。在实际使用中，序列仍然是离散的 token，连续 ODE 只是一个**建模框架**，真正运行在计算机上的是离散化后的递推（见 §7.3）。连续视角的价值在于让矩阵 A 的选取有了严格的数学理论可依——要在连续时间里「最优地」压缩历史，该选什么样的 A ，正是 HiPPO 框架 (§7.2) 的任务。

7.1.0 工程直觉引子：从「滑动窗口 + 缓存状态」走到连续 ODE

在跳进连续时间、一阶 ODE 和勒让德多项式之前，我们先用一个工程老兵都熟悉的画面开场。这个引子并不严格，但能为后面所有那些「凭空出现」的记号预存一个心锚。

设想这样一个年轻的服务。它要实时处理一条无限长的 token 流，但只允许维护一个**固定大小的缓存状态** $h \in \mathbb{R}^N$ 。每来一个新输入 x_t ，服务需要决定：旧状态保留多少，新输入注入多少。最自然的写法是一条带遗忘因子的线性递推：

$$h_t = \alpha h_{t-1} + \beta x_t, \quad y_t = c^\top h_t, \quad (7.0)$$

其中 $\alpha \in (0, 1)$ 控制「旧状态衰减多快」， β 控制「新输入注入多重」。QA 老兵一眼就能认出：这就是监控系统里近 30 年被反复使用的**指数加权滑动平均 (EWMA)**——Prometheus 的 `rate()`、Kafka 的 consumer lag 告警、高频交易系统的 tick-by-tick 指标都用这条递推。它有三个友好的性质：

- **记忆有限**：远古输入的权重衰减为 α^k ， k 越大越被遗忘；
- **状态恒定**：不论流多长， h_t 始终为 N 维向量；
- **线性叠加**：两条输入流的状态可以独立计算后相加。

连续化只需一步跳跃。如果我们不再假设 token 是按 $\Delta t = 1$ 均匀到达，而是令 $\Delta t \rightarrow 0$ ，记 $\alpha = 1 + A\Delta t$ 、 $\beta = B\Delta t$ ，那么

$$\frac{h_t - h_{t-1}}{\Delta t} = A h_{t-1} + B x_t.$$

取极限就得到

$$h'(t) = A h(t) + B x(t), \quad y(t) = C h(t),$$

这正是后面 §7.1.1 要介绍的一阶线性 ODE。换句话说，连续状态空间模型不是什么新东西，它是「带衰减的滑动窗口」在 $\Delta t \rightarrow 0$ 下的连续极限。全章将反复使用的连续信号、状态转移矩阵、双线性变换，都只是在问一个问题：应该选什么样的 A ，才能让这个「无限长流·固定状态」的缓存尽可能多地保留历史信息？§7.2 HiPPO 的答案会是：选一个与勒让德多项式正交的 A 。

一句话总结: 接下来所有最奇怪的记号—— e^{At} 、勒让德多项式、双线性变换——都只是「滑动窗口 + 缓存状态」这个工程原型的**连续极限版本**。请带着这个心锚走进后面的推导。

7.1.1 线性时不变系统的基本方程

状态空间模型 (State Space Model, SSM) 的出发点是经典的线性时不变 (Linear Time-Invariant, LTI) 系统。给定一维输入信号 $x(t) \in \mathbb{R}$ 和 N 维隐状态 $h(t) \in \mathbb{R}^N$, 系统方程为

$$h'(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t), \quad (7.1)$$

其中 $A \in \mathbb{R}^{N \times N}$ 是状态转移矩阵, $B \in \mathbb{R}^{N \times 1}$ 是输入投影向量, $C \in \mathbb{R}^{1 \times N}$ 是输出投影向量。此方程描述了隐状态如何随时间演化以及如何从隐状态恢复输出。

在信号处理与控制理论中, 式 (??) 已有数十年研究历史。将其引入序列建模的关键洞察在于: 若选取合适的矩阵 A , 则 $h(t)$ 能够以固定内存代价维护对任意长历史的压缩表示。LTI 性质保证了卷积结构, 而卷积结构则保证了 $O(L \log L)$ 的训练效率。

7.1.2 闭式解的推导

方程 (??) 是一阶线性常微分方程组。将其改写为

$$h'(t) - Ah(t) = Bx(t).$$

左乘积分因子 e^{-At} , 得

$$\frac{d}{dt}(e^{-At}h(t)) = e^{-At}Bx(t).$$

在 $[0, t]$ 上积分, 并利用初始条件 $h(0)$,

$$e^{-At}h(t) - h(0) = \int_0^t e^{-As}Bx(s) ds.$$

两边乘以 e^{At} , 得到**闭式解**:

$$h(t) = e^{At}h(0) + \int_0^t e^{A(t-s)}Bx(s) ds. \quad (7.2)$$

其中 e^{At} 是矩阵指数 (matrix exponential), 定义为 $e^{At} = \sum_{k=0}^{\infty} \frac{(At)^k}{k!}$ 。

稳定性条件: 若 A 的所有特征值 λ_i 满足 $\text{Re}(\lambda_i) < 0$, 则 $\|e^{At}\| \rightarrow 0$ 当 $t \rightarrow \infty$, 系统渐近稳定。HiPPO 矩阵的特征值恰好位于左半复平面, 保证了状态的有界演化。

7.1.3 卷积视角

将闭式解代入输出方程, 并设 $h(0) = 0$ (零初始状态), 得

$$y(t) = Ch(t) = \int_0^t Ce^{A(t-s)}Bx(s) ds = (K * x)(t), \quad (7.3)$$

其中卷积核定义为

$$K(t) = Ce^{At}B. \quad (7.4)$$

这一卷积视角至关重要：在离散化之后，若参数 A, B, C 不依赖于输入（LTI 假设），则整个序列的输出可通过**单次全局卷积**在频域以 $O(L \log L)$ 时间内计算，其中 L 是序列长度。这也是 S4 等模型实现高效训练的核心。

注记：式 (??) 中的卷积核 $K(t)$ 完全由矩阵 A 的特征结构决定。若 A 具有负实部的特征值，则 $K(t)$ 随时间指数衰减；若 A 有纯虚特征值，则 $K(t)$ 表现为振荡记忆。HiPPO 框架 (§7.2) 将系统地回答：什么样的 A 能够最优地压缩历史信息？

7.1.4 频域分析与传递函数

对式 (??) 作单边拉普拉斯变换（设零初始条件），令 s 为复频率变量：

$$sH(s) = AH(s) + BX(s), \quad Y(s) = CH(s),$$

解出 $H(s) = (sI - A)^{-1}BX(s)$ ，故传递函数为

$$G(s) = \frac{Y(s)}{X(s)} = C(sI - A)^{-1}B. \quad (7.5)$$

这是有理分式，分母的零点即为矩阵 A 的特征值（系统极点）。S4 中利用此结构，在 z 域上用 DPLR 分解将 $(zI - \bar{A})^{-1}$ 的计算化为柯西（Cauchy）矩阵向量积（见 §7.3）。

§7.2 HiPPO 框架的严格推导

这一节在讲什么

为什么要研究「最优压缩历史」？ 上一节论证了状态向量 $h(t)$ 能用一阶 ODE 维护，但没有告诉你该持续配置哪个 A 。HiPPO 的回答是：让 $h(t)$ 成为历史信号在某组正交多项式基上的最优投影系数。

第一步：把压缩问题化为投影问题。 与其说「记住历史」，不如说「用 N 个坐标表示历史在某个函数空间里的最优近似」。勒让德多项式在 $[-1, 1]$ 上关于均匀测度正交，确保系数之间不干扰、信息不冗余。

第二步：不同测度导出不同的 A 。 对历史序列的「最优源」是个选择：关注近期（滑动窗口均匀测度）就得到 LegT 矩阵；关注全历史（全局缩放均匀测度）就得到 LegS 矩阵。两者都能用小于 $N \times N$ 的小矩阵表示。

第三步： A 矩阵的具体形式来自多项式递推。 利用勒让德多项式的导数展开公式，将对 t 求导转化为内积系数的线性组合，最终得到 $h'(t) = A_{\text{HiPPO}}h(t) + Bx(t)$ 的显式表达式。

本节要拿走的一件事：HiPPO 矩阵不是猜出来的，而是「要让固定大小的向量 $h(t)$ 在每一时刻都对历史信号给出最小二乘最优投影」这一要求强制推导出来的。

直觉

上一节确立了状态空间模型的基本方程，但没有回答一个关键问题：应该选什么样的矩阵 A ？随意选一个 A 并不能保证状态 $h(t)$ 对历史信息的有效压缩。HiPPO 框架把这个问题转化为一个明确的最优化目标：找一个 A ，使得 $h(t)$ 在每一时刻都是历史信号在某个正交多项式基上的最小二乘最优投影。这个想法的巧妙之处在于：投影到正交基上，不同基向之间不干扰， N 个坐标各自承载不同频率和尺度的历史信息。

不同的测度选择对应不同的「最优 · 历史的定义」。HiPPO-LegT 使用常宽滑动窗口（均匀分布在最近的 θ 步内），对「近期历史」最优；HiPPO-LegS 使用全局缩放均匀测度（当前时刻 t 前的所有时刻均等对待），对「全局历史」最优。庞大直觉：如果你更关心近期的事件，用 LegT；如果你要平均地对待整个历史，用 LegS。S4 选择使用 LegS 作为初始化，就是因为它对全局历史给出了均匀的关注。

实际推导中， A 矩阵的具体形式来自对投影系数 $h_n(t) = \langle x(s), P_n(s/t) \rangle$ 关于 t 求导。求导过程要用到莱布尼茨积分法则和勒让德多项式的导数展开公式，局部线性代数最终给出一个可以写尺的下三角矩阵 A 。关键在于： A 是弱导出来的，不是猜的，这使得 HiPPO 不同于之前那些「凭经验设计 A 」的方法。

本节的关键论证链：(1) 问题陈述——压缩历史信号；(2) 选 Legendre 多项式做基函数（正交、闭式系数）；(3) 推导 LegT/LegS 矩阵 A ；(4) 证明所得 A 正是「让有限维状态 $h(t)$ 在每一时刻都对历史的最优最小二乘投影」的那一个。**关键转折点会在每个小节标题后用粗体提示。**

7.2.1 问题陈述：在线最优投影

HiPPO(High-order Polynomial Projection Operators) 由 Gu et al. (NeurIPS 2020) 提出，其核心问题是：

给定连续信号 $x : [0, t] \rightarrow \mathbb{R}$ ，如何在线地 (online) 将其历史压缩到固定维向量 $h(t) \in \mathbb{R}^N$ ，使得 $h(t)$ 以某种意义最优地近似 x 在 $[0, t]$ 上的全部历史？

形式化：设 $\mu^{(t)}$ 是 $[0, t]$ 上的测度 (measure)， $\{g_n\}_{n=0}^{N-1}$ 是关于 $\mu^{(t)}$ 的正交多项式基。定义**投影算子**

$$\text{proj}_t : L^2(\mu^{(t)}) \rightarrow \text{span}\{g_0, \dots, g_{N-1}\},$$

令 $h_n(t) = \langle x, g_n \rangle_{\mu^{(t)}}$ 为 $x(s)$ (视为 $[0, t]$ 上的函数, $s \leq t$) 在第 n 个基函数上的内积系数。则 $h(t) = (h_0(t), \dots, h_{N-1}(t))^T$ 是最优 N 维近似的系数向量。

关键结论：当测度 $\mu^{(t)}$ 随时间演化时， $h(t)$ 满足线性微分方程 $h'(t) = Ah(t) + Bx(t)$ ，矩阵 A 由测度的具体形式决定。

7.2.2 Legendre 多项式的正交性

勒让德多项式 $P_n : [-1, 1] \rightarrow \mathbb{R}$ 由 Rodrigues 公式定义：

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} [(x^2 - 1)^n].$$

关于标准 Lebesgue 测度 dx 在 $[-1, 1]$ 上满足正交性：

$$\int_{-1}^1 P_n(x) P_m(x) dx = \frac{2}{2n+1} \delta_{nm}. \quad (7.6)$$

归一化后的正交基为 $\tilde{P}_n(x) = \sqrt{\frac{2n+1}{2}} P_n(x)$, 满足 $\int_{-1}^1 \tilde{P}_n \tilde{P}_m dx = \delta_{nm}$ 。

前几阶勒让德多项式为 $P_0(x) = 1, P_1(x) = x, P_2(x) = \frac{1}{2}(3x^2 - 1)$, 满足递推关系:

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x). \quad (7.7)$$

这一递推关系是 HiPPO 矩阵推导中的核心工具。

7.2.3 HiPPO-LegT: 平移 Legendre 测度

LegT(Translated Legendre) 使用滑动窗口测度: 对固定窗口大小 $\theta > 0$,

$$\mu^{(t)} = \frac{1}{\theta} \mathbf{1}_{[t-\theta, t]} ds \quad (\text{均匀分布在长度为 } \theta \text{ 的窗口上}).$$

通过变量替换将 $[t-\theta, t]$ 映射到 $[-1, 1]$, 令 $u = \frac{2(s-(t-\theta))}{\theta} - 1$, 则系数 $h_n(t) = \sqrt{\frac{2n+1}{\theta}} \int_{t-\theta}^t P_n\left(\frac{2(s-t)}{\theta} + 1\right) x(s) ds$ 对时间 t 求导, 利用莱布尼茨积分法则和勒让德多项式的递推关系, 可以推导出:

$$h'(t) = A_{\text{LegT}} h(t) + B_{\text{LegT}} x(t),$$

其中显式矩阵为

$$(A_{\text{LegT}})_{nk} = -\frac{1}{\theta} \begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & \text{若 } n \geq k, \\ (-1)^{n-k}(2n+1)^{1/2}(2k+1)^{1/2} & \text{若 } n < k, \end{cases} \quad (7.8)$$

以及 $(B_{\text{LegT}})_n = \frac{1}{\theta}(2n+1)^{1/2}(-1)^n$ 。此处为简洁起见设 $\theta = 1$ 。

物理意义: 矩阵 A_{LegT} 的下三角块使得低阶系数影响高阶系数, 而上三角块 (反号) 体现了窗口滑动时旧信息的遗忘机制。

7.2.4 引理 (HiPPO-LegS 矩阵的推导)

引理 7.1 (HiPPO-LegS 矩阵的显式形式): 设测度为累积 (缩放 Legendre) 测度:

$$\mu^{(t)} = \frac{1}{t} \mathbf{1}_{[0, t]} ds,$$

即在 $[0, t]$ 上的均匀测度, 随时间整体缩放。则最优投影系数 $h(t)$ 满足的 SSM 矩阵为

$$(A_{\text{LegS}})_{nk} = \begin{cases} -\sqrt{(2n+1)(2k+1)} & \text{若 } n > k, \\ -(n+1) & \text{若 } n = k, \\ 0 & \text{若 } n < k, \end{cases} \quad (7.9)$$

以及 $(B_{\text{LegS}})_n = \sqrt{2n+1}$ 。

证明:

设归一化基函数 $\tilde{P}_n(x) = \sqrt{2n+1}P_n(2x-1)$, 在 $[0, 1]$ 上关于均匀测度正交, 即

$$\int_0^1 \tilde{P}_n(x)\tilde{P}_m(x)dx = \delta_{nm}.$$

将 $[0, t]$ 通过变量替换 $u = s/t$ 映射到 $[0, 1]$, 系数为

$$h_n(t) = \int_0^1 \tilde{P}_n(u)x(tu)du = \frac{1}{t} \int_0^t \tilde{P}_n(s/t)x(s)ds.$$

对 t 求导 (应用莱布尼茨法则):

$$h'_n(t) = -\frac{1}{t^2} \int_0^t \tilde{P}_n(s/t)x(s)ds + \frac{1}{t} \tilde{P}_n(1)x(t) - \frac{1}{t^2} \int_0^t \frac{s}{t} \tilde{P}'_n(s/t)x(s)ds.$$

第一项化简为 $-\frac{1}{t}h_n(t)$; 第三项中利用勒让德多项式导数的展开公式:

$$P'_n(x) = \sum_{\substack{k=0 \\ n-k \text{ 奇}}}^{n-1} (2k+1)P_k(x),$$

因此 $\tilde{P}'_n(u) = \sqrt{2n+1} \sum_{\substack{k < n \\ n-k \text{ 奇}}} (2k+1)P_k(2u-1) = \sqrt{2n+1} \sum_{k < n} \sqrt{2k+1} \tilde{P}_k(u) \cdot [(n-k \text{ 奇})?1:0]$.

将上式代入第三项并整理, 对角项系数汇总为 $-(n+1)/t$, 次对角项系数为 $-\sqrt{(2n+1)(2k+1)}/t$ ($n > k$), 上三角项为 0. 注意 $\tilde{P}_n(1) = \sqrt{2n+1}$ (因为 $P_n(1) = 1$), 故 $B_n = \sqrt{2n+1}$.

此系统通常写为 $th'(t) = \hat{A}h(t) + \hat{B}x(t)$, 并通过时间变换 $\tau = \log t$ 化为标准 LTI 形式。□

数值示例: 对 $N = 4$ 的 LegS 矩阵:

$$A_{\text{LegS}} = - \begin{pmatrix} 1 & 0 & 0 & 0 \\ \sqrt{3} & 2 & 0 & 0 \\ \sqrt{5} & \sqrt{15} & 3 & 0 \\ \sqrt{7} & \sqrt{21} & \sqrt{35} & 4 \end{pmatrix}, \quad B_{\text{LegS}} = \begin{pmatrix} 1 \\ \sqrt{3} \\ \sqrt{5} \\ \sqrt{7} \end{pmatrix}.$$

矩阵是严格下三角占优的, 这保证了在适当归一化后矩阵指数的有界性。

7.2.5 最优投影的精确陈述

定理 7.2 (HiPPO 最优近似保证): 设 $\hat{x}_t(s) = \sum_{n=0}^{N-1} h_n(t)\tilde{P}_n(s/t)$ 为 $x|_{[0,t]}$ 的 N 阶 Legendre 多项式近似。则

$$\hat{x}_t = \arg \min_{p \in \mathcal{P}_{N-1}} \int_0^t |p(s) - x(s)|^2 \frac{ds}{t},$$

其中 $\mathcal{P}_{N-1}$ 是次数不超过 $N-1$ 的多项式空间。即 $h(t)$ 给出了 $x|_{[0,t]}$ 在 $[0, t]$ 上的均方意义下的最优 N 维 Legendre 系数近似。

注记: 不同的测度 $\mu^{(t)}$ 导致不同的“最优性”——LegT 最优近似的是最近的窗口, LegS 最优近似的是全历史的加权 (均匀) 版本。§7.8 将讨论这一点对模型性质的影响。

§7.3 S4 的离散化与卷积核

这一节在讲什么

为什么需要离散化? 连续 ODE 是理论框架, 但计算机处理的是离散 token 序列。必须将 $h'(t) = Ah(t) + Bx(t)$ 转换成离散递推 $h_k = \bar{A}h_{k-1} + \bar{B}x_k$, 才能在 GPU 上运行。

第一步: ZOH 离散化给出精确的 $\bar{A}, \bar{B}$ 。 零阶保持假设输入在每个步长内为常数, 积分闭式可算, 得到 $\bar{A} = e^{\Delta A}$ 。这不是近似, 而是直接对 ODE 的较确矩阵和。

第二步: 展开递推得到卷积。 $y_k = \sum_{j=0}^k \bar{C}\bar{A}^{k-j}\bar{B}x_j$, 这是一个离散卷积运算。卷积核 $\bar{\mathbf{K}} = (\bar{C}\bar{B}, \bar{C}\bar{A}\bar{B}, \dots)$ 一旦知道, 整个序列可以用 FFT 在 $O(L \log L)$ 内完成。

第三步: DPLR 结构把卷积核计算从 $O(N^2L)$ 降至 $\tilde{O}(N+L)$ 。 将 A 写成对角加低秩形式, 用 Woodbury 恒等式解析矩阵逆, 再把卷积核的乾☐函数评估问题转化为 Cauchy 矩阵向量积。

本节要拿走的一件事: S4 的效率来自两个叠加: LTI 保证卷积结构可用 FFT, DPLR 结构保证卷积核可快速计算。去掉任一个, 效率就崩塌。

直觉

连续状态空间模型在理论上很优美, 但它运行在连续时间内而序列建模处理的是离散 token。离散化步骤就是把连续 ODE “翻译” 成 GPU 可运行的离散递推。最直接的方法是零阶保持 (ZOH): 假设输入在每个步长内为常数, 然后积分闭式直接可算, 得到 $\bar{A} = e^{\Delta A}$ 。这不是近似——在 ZOH 假设下它是精确的。

离散递推展开后就变成卷积: $y_k = \sum_{j=0}^k \bar{K}_{k-j}x_j$, 其中 $\bar{K}_m = \bar{C}\bar{A}^m\bar{B}$ 。卷积结构意味着可以用 FFT 批量计算整个序列, 训练复杂度从 $O(NL)$ (每步递推) 降至 $O(L \log L)$ (每次清气 FFT)。但有个前提: 只有在 $\bar{A}$ 不依赖输入 (LTI 假设) 时, 卷积核 $\bar{\mathbf{K}}$ 才是固定的, 才能预先计算。

现在还有一个瓶颈: 算 $\bar{\mathbf{K}}$ 需要计算 L 个 $\bar{A}^m\bar{B}$, 若 $\bar{A}$ 是稠密 $N \times N$ 矩阵, 复杂度为 $O(N^2L)$, $N = L = 4096$ 时就是天文数字。S4 的销魏千金在于 DPLR 参数化: 把 A 写成对角加低秩的形式 $A = \Lambda - PQ^*$, 用 Woodbury 恒等式解析矩阵逆, 再把卷积核的生成函数在 FFT 直点处的估值问题转化为 Cauchy 矩阵向量积。最终复杂度从 $O(N^2L)$ 降至 $\tilde{O}(N+L)$, 速度提升几个数量级。

本节的关键骨架: 连续 ODE $h'(t) = Ah(t) + Bx(t) \rightarrow$ 零阶保持 (ZOH) 离散化 $\rightarrow$ 离散递推 $h_k = \bar{A}h_{k-1} + \bar{B}x_k \rightarrow$ 展开为卷积 $y = \bar{\mathbf{K}} * x \rightarrow$ **关键消元** Cauchy 核 + Vandermonde 矩阵把卷积从 $O(NL)$ 降至 $O((N+L) \log^2(N+L))$ 。

7.3.1 零阶保持离散化

在实际应用中, 序列数据是离散的: 时刻 $k = 0, 1, \dots, L-1$, 步长 (time step) 为 $\Delta > 0$ 。最常用的离散化方案是**零阶保持 (Zero-Order Hold, ZOH)**, 假设输入在每个时间步内保持常数。

引理 7.3 (ZOH 离散化的精确形式): 在步长 Δ 的 ZOH 假设下, 连续 SSM (A, B, C) 离散化为

$$\bar{A} = e^{\Delta A}, \quad \bar{B} = (\Delta A)^{-1}(e^{\Delta A} - I) \cdot \Delta B, \quad \bar{C} = C. \quad (7.10)$$

离散递推为 $h_k = \bar{A}h_{k-1} + \bar{B}x_k, y_k = \bar{C}h_k$ 。

证明:

将时间区间 $[k\Delta, (k+1)\Delta]$ 上的连续解 (??) 应用于 $h((k+1)\Delta)$:

$$h((k+1)\Delta) = e^{A\Delta}h(k\Delta) + \int_{k\Delta}^{(k+1)\Delta} e^{A((k+1)\Delta-s)} Bx(s) ds.$$

ZOH 假设 $x(s) = x_{k+1}$ 为常数, 令 $\tau = (k+1)\Delta - s$, 则

$$\int_{k\Delta}^{(k+1)\Delta} e^{A((k+1)\Delta-s)} Bx_{k+1} ds = \left(\int_0^\Delta e^{A\tau} d\tau \right) Bx_{k+1}.$$

利用矩阵指数的积分公式:

$$\int_0^\Delta e^{A\tau} d\tau = A^{-1}(e^{A\Delta} - I),$$

故

$$\bar{B} = A^{-1}(e^{A\Delta} - I) \cdot B = (\Delta A)^{-1}(e^{\Delta A} - I) \cdot \Delta B.$$

这正是式 (??) 中的 $\bar{B}$ 。□

与双线性方法的比较: S4 原始论文中也使用双线性 (Tustin/bilinear) 离散化 $\bar{A} = (I - \Delta A/2)^{-1}(I + \Delta A/2)$, 其精度为 $O(\Delta^2)$ 而 ZOH 同阶。两者实践差异较小, 但 Mamba 重新引入 ZOH 以支持更直观的选择机制——因为 ZOH 下 $\bar{A} = e^{\Delta A}$ 具有更清晰的“遗忘率”语义。

7.3.2 离散卷积核

在 LTI 设定下, 离散 SSM 的输入-输出关系展开为卷积。将递推展开:

$$y_k = \bar{C}\bar{A}^k\bar{B}x_0 + \bar{C}\bar{A}^{k-1}\bar{B}x_1 + \cdots + \bar{C}\bar{B}x_k,$$

即 $y_k = \sum_{j=0}^k \bar{K}_j x_{k-j}$, 其中**离散卷积核向量**为

$$\bar{\mathbf{K}} = (\bar{C}\bar{B}, \bar{C}\bar{A}\bar{B}, \bar{C}\bar{A}^2\bar{B}, \dots, \bar{C}\bar{A}^{L-1}\bar{B}). \quad (7.11)$$

向量 $\bar{\mathbf{K}} \in \mathbb{R}^L$ 称为 SSM 的卷积核 (SSM kernel)。一旦计算出 $\bar{\mathbf{K}}$, 整个序列的输出可由 $\mathbf{y} = \bar{\mathbf{K}} * \mathbf{x}$ (离散卷积) 在频域通过 FFT 以 $O(L \log L)$ 计算。

7.3.3 计算复杂度的降低: Cauchy 核与范德蒙 (Vandermonde) 矩阵

瓶颈: 直接计算 $\bar{\mathbf{K}}$ 需要 L 次矩阵幂 $\bar{A}^k B$, 若 $\bar{A}$ 是稠密 $N \times N$ 矩阵, 则复杂度为 $O(N^2 L)$, 在 $N = L = 4096$ 的典型设定下为 $\sim 7 \times 10^{10}$ 次运算, 不可接受。

DPLR 参数化: S4 的核心技术是将 A 参数化为**对角加低秩 (Diagonal Plus Low-Rank, DPLR)**形式:

$$A = \Lambda - PQ^*, \quad (7.12)$$

其中 $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_N)$ 是对角矩阵, $P, Q \in \mathbb{C}^{N \times r}$ 是低秩矩阵 ($r \ll N$)。HiPPO-LegS 矩阵恰好可以写成 DPLR 形式: 其正规部分为斜对称矩阵 $S_{nk} = -\frac{1}{2}[(2n+1)(2k+1)]^{1/2} \text{sgn}(n-k)$, 可对角化为纯虚特征值; 非正规部分为秩 1 修正 $-\frac{1}{2}pp^*$, 其中 $(p)_n = (n+1/2)^{1/2}$ 。

Woodbury 恒等式: 设 $M = \Lambda - PQ^*$ 为 DPLR 矩阵, 则

$$M^{-1} = \Lambda^{-1} + \Lambda^{-1}P(I - Q^*\Lambda^{-1}P)^{-1}Q^*\Lambda^{-1}. \quad (7.13)$$

由于 Λ^{-1} 是对角矩阵, 右侧所有运算均可以 $O(N + r^3)$ 时间完成。这使得在对角形上计算 $(zI - \bar{A})^{-1}$ 变得可行。

关键算法: 设 Λ 的特征值为 $\{\lambda_j\}_{j=1}^N$, 离散化后 $\bar{\Lambda} = e^{\Delta\lambda}$ 。卷积核的第 k 个元素可写为

$$\bar{K}_k = \bar{C}\bar{A}^k\bar{B} = \sum_{j=1}^N c_j \bar{\lambda}_j^k,$$

其中系数 $\{c_j\}$ 由伍德伯里 (Woodbury) 恒等式修正低秩项后得到。将全部 L 个核元素的生成函数求值:

$$\hat{K}(z) = \sum_{k=0}^{L-1} \bar{K}_k z^k \approx C(zI - \bar{A})^{-1}\bar{B} = \sum_{j=1}^N \frac{c_j}{z - \bar{\lambda}_j}. \quad (7.14)$$

在 L 个单位根 ω^j (其中 $\omega = e^{2\pi i/L}$) 处求值:

$$\hat{K}(\omega^l) = \sum_{j=1}^N \frac{c_j}{\omega^l - \bar{\lambda}_j}, \quad l = 0, 1, \dots, L-1.$$

这等价于 **Cauchy 矩阵向量积** $\text{diag}(\hat{K}) = M_{\text{Cauchy}}\mathbf{c}$, 其中 $(M_{\text{Cauchy}})_{lj} = 1/(\omega^l - \bar{\lambda}_j)$ 。利用快速多极方法或 Vandermonde 矩阵的 FFT 类算法, 此计算可在 $\tilde{O}(N + L)$ 时间内完成。最终通过逆 FFT 恢复时域卷积核 $\bar{\mathbf{K}}$ 。

整体复杂度: 从 $O(N^2L)$ 降至 $\tilde{O}(N + L)$, 提升约 $N \cdot L / \log(N + L)$ 倍。

DPLR 参数化的必要性: 若 A 是一般稠密矩阵, 则其矩阵指数的计算为 $O(N^3)$, 后续矩阵幂亦为 $O(N^2)$ per step, 无法实现 Cauchy 核约化。DPLR 结构保证:(i) 矩阵逆由 Woodbury 恒等式精确解析;(ii) 离散化后 $\bar{A}$ 仍保持 DPLR 结构;(iii) 所有操作在数值稳定的复对角形上进行。S4D(Diagonal S4) 进一步将 DPLR 中 P, Q 简化为零, 直接用纯对角矩阵 $A = \Lambda$, 实现更简洁的实现, 并在实践中与 S4 性能相当。

§7.4 Mamba 的选择性扫描

这一节在讲什么

为什么 S4 还不够？ S4 的 LTI 假设意味着模型对所有位置的相同输入给出相同响应，无法分辨「这个 token 需要记住」还是「这个 token 需要忽略」。Mamba 的解决方案是将 Δ_t, B_t, C_t 设为输入 x_t 的函数。

第一步： Δ_t 控制记忆与遗忘的弹性。 $\Delta_t \rightarrow 0$ 时 $\bar{A}_t \rightarrow I$ ，当前输入被忽略、历史状态保持； $\Delta_t \rightarrow +\infty$ 时 $\bar{A}_t \rightarrow 0$ ，历史全部清除、当前输入优先。模型学会根据内容动态调节这个「分辨率」。

第二步：选择性打破了 LTI，全局卷积核不再有效。 $\bar{A}_t$ 现在依赖 t ，序列不再是时不变系统，无法预先计算卷积核。但递推中存在二元结合律操作，可用 Blelloch 并行扫描将深度从 $O(L)$ 压到 $O(\log L)$ 。

第三步：硬件感知实现将状态和参数都放进 SRAM。 避免反复读写 HBM，实现训练吞吐量约 5 倍于 Transformer。

本节要拿走的一件事： Mamba = S4 的选择性版本。用输入依赖的 Δ_t, B_t, C_t 换掉静态参数，获得内容感知的记忆，代价是失去全局卷积核、需要并行扫描弥补。

直觉

S4 建立在 LTI（线性时不变）假设上：矩阵 A, B, C 对所有位置和输入都相同。这意味着它无法根据内容决定「应该把哪个 token 写入记忆」——与 Transformer 的注意力机制相比，这是一个根本缺陷。Mamba 的选择性机制就是用输入依赖的 Δ_t, B_t, C_t 替换静态参数，让每个时刻的「离散化分辨率」都由当前内容决定。 Δ_t 通过 softplus 保证为正： Δ_t 小则遗忘慢、输入被忽略； Δ_t 大则历史快速被清、模型聚焦于当前输入。这与 LSTM 的遗忘门在概念上相似，但数学形式更严格、更易于确保稳定性。

选择性的代价是失去全局卷积核。因为 $\bar{A}_t$ 现在随时间变化，序列不再是时不变系统，「预先计算一个卷积核多次使用」的技巧失效了。必须对每条序列逐步递推。但 Mamba 的关键发现是：递推 $h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$ 具有一个可证明满足结合律的二元操作，所有前缀积因此可以用 Blelloch 并行扫描在 $O(\log L)$ 层 GPU 并行内完成。结合硬件感知实现（将状态和参数清退到 SRAM，避免读写 HBM），训练吞吐量约 5 倍于 Transformer。

这里一个常见的误解是：「Mamba 弃用了卷积，所以它就是纯 RNN」。实际上，**Mamba 在训练时使用并行扫描，推理时则以递推模式运行**。训练和推理可以切换模式，这是它同时具备并行训练效率和常数推理时间复杂度的关键。

7.4.1 选择性机制：从 LTI 到输入依赖

S4 的根本局限在于参数 A, B, C 与输入无关 (LTI)，因此模型在不同位置对相同幅度的输入给出相同响应，无法进行基于内容的选择性记忆。Mamba(Gu & Dao, 2023) 通过使参数依赖于当前输入来打破这一限制。

选择性参数化： 对输入序列 $\{x_t\}_{t=1}^L$, Mamba 令

$$\Delta_t = \text{softplus}(W_\Delta x_t + b_\Delta) \in \mathbb{R}^D, \quad B_t = W_B x_t \in \mathbb{R}^N, \quad C_t = W_C x_t \in \mathbb{R}^N, \quad (7.15)$$

其中 $W_\Delta \in \mathbb{R}^{D \times D}, W_B \in \mathbb{R}^{N \times D}, W_C \in \mathbb{R}^{N \times D}$ 是可学习参数, D 是输入维度。步长 Δ_t 决定了当前时刻离散化的“分辨率”, 由 $\text{softplus}(x) = \ln(1 + e^x) > 0$ 保证正性。

选择性离散化: 对每个时刻 t , 独立执行 ZOH 离散化:

$$\bar{A}_t = e^{\Delta_t A}, \quad \bar{B}_t = (\Delta_t A)^{-1}(e^{\Delta_t A} - I)\Delta_t B_t, \quad (7.16)$$

其中 A 保持固定 (仅结构参数化)。选择性 SSM 的递推为:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \quad y_t = C_t h_t. \quad (7.17)$$

直觉分析: 当 $\Delta_t \rightarrow 0^+$ 时, $\bar{A}_t = e^{\Delta_t A} \rightarrow I$ 而 $\bar{B}_t \rightarrow 0$, 模型忽略当前输入、保持历史状态; 当 $\Delta_t \rightarrow +\infty$ 时, 若 A 有负特征值, 则 $\bar{A}_t \rightarrow 0$ 且 $\bar{B}_t \rightarrow -A^{-1}B_t$, 模型聚焦于当前输入、遗忘所有历史。 Δ_t 由输入内容决定, 实现了“token-wise 的记忆/遗忘控制”。

7.4.2 选择性 SSM 的计算挑战

由于 $\bar{A}_t$ 依赖于 t , 序列不再是 LTI 系统, **全局卷积核不再有效**。朴素的顺序递推需要 $O(LN)$ 时间但无法在 GPU 上并行化, 成为训练的瓶颈。解决方案是**并行扫描算法** (Parallel Scan)。

7.4.3 并行扫描与关联算子

引理 7.4 (选择性 SSM 的关联算子): 定义元素 $(a, b) \in \mathbb{R}^{N \times N} \times \mathbb{R}^N$, 其中 a 表示状态转移矩阵, b 表示驱动项。定义二元运算

$$(a_1, b_1) \otimes (a_2, b_2) = (a_2 a_1, a_2 b_1 + b_2). \quad (7.18)$$

则此运算满足结合律, 且递推 (??) 可以写成

$$(h_t, 1) \xleftrightarrow{\text{对应}} (\bar{A}_t, \bar{B}_t x_t) \otimes (\bar{A}_{t-1}, \bar{B}_{t-1} x_{t-1}) \otimes \cdots \otimes (\bar{A}_1, \bar{B}_1 x_1). \quad (7.19)$$

证明 (结合律):

对三个元素 $(a_1, b_1), (a_2, b_2), (a_3, b_3)$, 验证左右结合相等。

左结合: 先计算 $(a_1, b_1) \otimes (a_2, b_2) = (a_2 a_1, a_2 b_1 + b_2)$, 再与 (a_3, b_3) 结合:

$$(a_2 a_1, a_2 b_1 + b_2) \otimes (a_3, b_3) = (a_3 (a_2 a_1), a_3 (a_2 b_1 + b_2) + b_3) = (a_3 a_2 a_1, a_3 a_2 b_1 + a_3 b_2 + b_3).$$

右结合: 先计算 $(a_2, b_2) \otimes (a_3, b_3) = (a_3 a_2, a_3 b_2 + b_3)$, 再与 (a_1, b_1) 结合:

$$(a_1, b_1) \otimes (a_3 a_2, a_3 b_2 + b_3) = ((a_3 a_2) a_1, (a_3 a_2) b_1 + (a_3 b_2 + b_3)) = (a_3 a_2 a_1, a_3 a_2 b_1 + a_3 b_2 + b_3).$$

两者相等, 故结合律成立。□

验证式 (??): 归纳地, 若 $h_0 = 0$, 令 $a_i = \bar{A}_i, b_i = \bar{B}_i x_i$, 则

$$h_t = a_t a_{t-1} \cdots a_1 \cdot 0 + a_t \cdots a_2 b_1 + a_t \cdots a_3 b_2 + \cdots + b_t,$$

恰好由运算 (??) 的前缀积给出。此式也可解释为：隐状态是带指数衰减权重的输入历史加权和，权重由路径乘积 $\prod_{s=j+1}^t \bar{A}_s$ 决定。

7.4.4 Blelloch 并行前缀扫描的复杂度

给定长度为 L 的序列 $\{(a_t, b_t)\}_{t=1}^L$ ，计算所有前缀积 $\{(a_t, b_t) \otimes \cdots \otimes (a_1, b_1)\}_{t=1}^L$ ：

- 朴素串行： $O(L)$ 步骤，无法并行化。
- Blelloch 并行扫描 (Blelloch, 1990)：利用结合律，将前缀积分为上扫 (up-sweep) 和下扫 (down-sweep) 两个阶段，在 $O(\log L)$ 深度的并行计算中完成，总工作量 (work) 为 $O(L)$ 。

具体地，每个 GPU 线程处理 $O(1)$ 个元素，利用 $O(\log L)$ 层二叉树归并：

$$\text{Work} = O(L), \quad \text{Depth} = O(\log L).$$

这使得长度 10^6 的序列可以在 ~ 20 层并行操作中完成状态扫描。Mamba 的硬件感知实现将参数和中间状态直接存储在 SRAM 中，避免反复读写 HBM(High Bandwidth Memory)，实现了训练时约 5 倍于 Transformer 的吞吐量提升。

§7.5 线性注意力的核函数视角

这一节在讲什么

为什么要把注意力近似成线性的？ 标准 Softmax 注意力的 Q, K, V 矩阵乘法需要 $O(L^2)$ 内存和 $O(L^2 d)$ 时间，在长序列上无法扩展。线性注意力的目标是把它压成 $O(Ld)$ 而不丧失太多表达能力。

第一步：用核函数分解代替 Softmax。 把 $\exp(Q_i \cdot K_j)$ 替换为 $\phi(Q_i) \cdot \phi(K_j)$ ，其中 $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^r$ 是特征映射。这让注意力矩阵变成两个低维矩阵的外积，突破了 L^2 的依赖。

第二步：利用矩阵乘法结合律改变计算顺序。 先算 $S = \sum_j \phi(K_j)^\top V_j$ ($O(Lrd)$)，再对每个 i 算 $\phi(Q_i)S$ ，无需显式构造 $L \times L$ 注意力矩阵。

第三步：推理时状态 S_t 可在线递推。 $S_t = S_{t-1} + \phi(K_t)^\top V_t$ ，每步 $O(rd)$ ，内存 $O(rd)$ ，与序列长度无关。

本节要拿走的一件事：线性注意力不是「注意力的近似」，而是用核技巧把注意力的 $O(L^2)$ 瓶颈变成一个固定大小的递推状态——这把它和 SSM、RNN 统一到了同一框架下（见 §7.6）。

直觉

Softmax 注意力的 $O(L^2)$ 瓶颈来自一个根本原因：要计算每个查询对每个键的相似度，必须构造完整的 $L \times L$ 注意力矩阵。线性注意力的突破思路是：用一个核函数分解 $\exp(Q_i \cdot K_j) \approx \phi(Q_i) \cdot \phi(K_j)$ 把注意力矩阵写成两个低维矩阵的外积，然后利用矩阵乘法的结合律改变计算顺序——先把所有键值对折叠成一个固定大小的矩阵 $S = \sum_j \phi(K_j)^\top V_j$ ，再用查询去检索它。这样既不需要显式构造 $L \times L$ 矩阵，也不需要随序列长度增长的内存。

推理时的优势更为显著。在自回归生成中，每来一个新 token，状态 S_t 只需做一次加法更新，然后直接用 $\phi(Q_t)$ 检索。每步的时间和内存都是 $O(rd)$ ，而不是 Softmax 注意力的 $O(td)$ ——后者随生成

长度线性增长。这个「固定大小状态 + 在线递推」的模式与 SSM 完全同构，也是 §7.6 统一视角的切入点。

然而核函数的选择并非无代价。ELU+1 这样的简单特征映射虽然快速，但并不精确近似 Softmax；FAVOR+ 的随机特征理论上无偏但需要更多特征维度；RetNet 的固定指数衰减则本质上是把键值对的贡献按位置加权衰减。每种选择都在「近似质量」与「计算效率」之间做不同的权衡。更根本的限制在于：有限秩的特征映射无法模拟 Softmax 的「赢者通吃」尖锐化能力，这导致线性注意力在需要精确键值检索的任务上存在系统性不足（见 §7.9.3）。

7.5.1 标准归一化指数 (Softmax) 注意力的复杂度

标准自注意力机制 (Vaswani et al., 2017) 定义为：

$$A_{ij} = \frac{\exp(Q_i \cdot K_j / \sqrt{d})}{\sum_{j'} \exp(Q_i \cdot K_{j'} / \sqrt{d})}, \quad O_i = \sum_j A_{ij} V_j, \quad (7.20)$$

其中 $Q, K, V \in \mathbb{R}^{L \times d}$ 分别是查询、键、值矩阵。计算 $L \times L$ 注意力矩阵 A 需要 $O(L^2 d)$ 时间和 $O(L^2)$ 空间，在长序列上形成瓶颈。

7.5.2 核函数分解

线性注意力的核心思想 (Katharopoulos et al., ICML 2020): 用正半定核函数 $k(Q_i, K_j) = \phi(Q_i) \cdot \phi(K_j)^\top$ 替换 $\exp(Q_i \cdot K_j)$ ，其中 $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^r$ 是特征映射 (feature map)。则

$$O_i = \frac{\sum_j \phi(Q_i) \cdot \phi(K_j)^\top V_j}{\sum_j \phi(Q_i) \cdot \phi(K_j)^\top}. \quad (7.21)$$

关键代数变换: 利用矩阵乘法的结合律

$$O_i = \frac{\phi(Q_i) \left(\sum_j \phi(K_j)^\top V_j \right)}{\phi(Q_i) \left(\sum_j \phi(K_j)^\top \right)}. \quad (7.22)$$

令 $\mathbf{S} = \sum_j \phi(K_j)^\top V_j \in \mathbb{R}^{r \times d}$ (累积 KV 状态), $\mathbf{z} = \sum_j \phi(K_j)^\top \in \mathbb{R}^r$ (归一化项), 则

$$O_i = \frac{\phi(Q_i) \mathbf{S}}{\phi(Q_i) \cdot \mathbf{z}}. \quad (7.23)$$

在因果 (causal) 设定下 (自回归生成), $\mathbf{S}_t = \sum_{j \leq t} \phi(K_j)^\top V_j$ 可在线更新:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \phi(K_t)^\top V_t, \quad O_t = \frac{\phi(Q_t) \mathbf{S}_t}{\phi(Q_t) \cdot \mathbf{z}_t}. \quad (7.24)$$

这使得推理时每个 token 的计算量为 $O(rd)$ (仅依赖固定大小的状态 $\mathbf{S}_t \in \mathbb{R}^{r \times d}$), 而非 $O(td)$ 。推理复杂度从 $O(L)$ per token 降至 $O(1)$ per token。

复杂度分析:

- **训练 (全序列):** 先计算全局 $\mathbf{S} = \sum_j \phi(K_j)^\top V_j$ (复杂度 $O(Lrd)$), 再对所有 i 计算 $O_i = \phi(Q_i) \mathbf{S} / (\phi(Q_i) \cdot \mathbf{z})$ (复杂度 $O(Lrd)$), 总体为 $O(Lrd)$ 。相比 $O(L^2 d)$, 当 $r \ll L$ 时有显著优势。
- **推理 (自回归):** 每步 $O(rd)$, 状态 $\mathbf{S}_t$ 占 $O(rd)$ 内存, 与序列长度无关。

7.5.3 特征映射的选择

不同 ϕ 的选择决定了近似质量与效率的权衡:

1. ELU+1 (Katharopoulos, 2020):

$$\phi(x)_i = \text{elu}(x_i) + 1 = \begin{cases} x_i + 1 & x_i \geq 0, \\ e^{x_i} & x_i < 0. \end{cases} \quad (7.25)$$

此映射保证 $\phi(x)_i > 0$, 使分母 $\phi(Q_i) \cdot \mathbf{z}$ 保持正定, 避免数值不稳定。但此 ϕ 并不对应 softmax 核的精确近似, 故可视为一种启发式替代。

2. FAVOR+ 随机特征 (Performer, Choromanski et al., ICLR 2021):

利用 softmax 核的正值随机特征分解。注意到

$$\exp(Q \cdot K) = \exp\left(\frac{\|Q\|^2 + \|K\|^2}{2}\right) \cdot \exp\left(-\frac{\|Q - K\|^2}{2}\right),$$

后者是高斯核。利用 Bochner 定理, 高斯核有无偏随机特征展开。FAVOR+ 使用正交正值特征:

$$\phi_\omega^+(x) = \frac{e^{-\|x\|^2/2}}{\sqrt{r}} [\exp(\omega_1^\top x), \dots, \exp(\omega_r^\top x)], \quad \omega_k \text{ 来自正交矩阵的列。} \quad (7.26)$$

可以证明 $\mathbb{E}_\omega[\phi^+(x) \cdot \phi^+(y)^\top] = \exp(x \cdot y)$, 即无偏估计 softmax 核。使用 r 个随机特征时, 方差为 $O(1/r)$ 。

3. RetNet 的固定衰减 (Sun et al., 2023):

RetNet 使用固定的指数衰减, 定义保留 (Retention) 机制:

$$A_{ij} = \gamma^{i-j} (Q_i K_j^\top / \sqrt{d}) \cdot \mathbf{1}_{i \geq j}, \quad \gamma \in (0, 1), \quad (7.27)$$

等价于 $\phi(Q)_i = Q_i, \phi(K)_j = \gamma^{-j} K_j$, 加上按位置的衰减因子 γ^i 。多头设定下每个头有不同的 γ (“多尺度保留”), 提供不同长度的记忆窗口。

7.5.4 线性注意力与 Fast Weight Memory 的等价性

Schlag et al. (2021) 指出, 式 (??) 的更新

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \phi(K_t)^\top V_t$$

可以解释为**快速权重存储器 (fast weight memory)**: 状态矩阵 $\mathbf{S}_t$ 以外积 $\phi(K_t)^\top V_t$ 形式存储键值关联, 查询时用 $\phi(Q_t)$ 检索。这与 Hopfield 网络和联想记忆 (associative memory) 具有深刻联系, 揭示了线性注意力的记忆机制本质。在此视角下, 外积存储的容量为 $O(rd)$ 比特, 对应约 r 个精确键值对; 当待检索的键值对数目超过 r 时, 干扰效应导致检索精度下降, 这从机制上解释了 §7.8 中讨论的检索任务劣势。

§7.6 Mamba、线性注意力 (Attention) 与 RNN 的统一视角

这一节在讲什么

为什么要统一这三类模型？ Mamba、线性注意力、门控 RNN 看似来自三条完全不同的研究路线，但它们的推理时递推结构高度相似。找到统一框架能指导新模型的设计，并解释为什么它们有相似的长处和短处。

第一步：所有模型都是同一广义线性递推的特例。 $\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t$ ，区别仅在于门控矩阵 $\mathbf{G}_t$ 的参数化方式：线性注意力取 $\mathbf{G}_t = \mathbf{1}$ ，RetNet 取 $\mathbf{G}_t = \gamma \mathbf{1}$ ，GLA 取数据依赖的 sigmoid，Mamba 取 $e^{\Delta_t A}$ 。

第二步：Mamba-2 的 SSD 对偶性把递推视角和矩阵视角打通。 SSM 的输入-输出映射等价于一个「半分离矩阵」乘向量，这将递推模型与结构化注意力精确对应，揭示了它们是同一运算的两个计算视角。

第三步：统一框架指明改进方向。 GLA 增加了数据依赖门，Mamba-2 扩大了状态维度，都是在这个框架下对门控 $\mathbf{G}_t$ 做更丰富的参数化。

本节要拿走的一件事： Mamba、线性注意力、门控 RNN 都是「广义线性递推」的实例，不同之处仅在于门控的参数化——从这个角度看，它们在同一条数学流形上，不是三类不同的模型。

直觉

在 §7.4 和 §7.5 各自的语境下，Mamba 和线性注意力看起来是从不同出发点推导出来的不同模型。但两者的推理时递推结构惊人地相似：都是用一个固定大小的状态矩阵 $\mathbf{S}_t$ 、用新的键值对更新它、用查询向量检索它。这暗示了一个更深层的统一：三类模型（Mamba、线性注意力、经典门控 RNN）可能只是同一广义递推框架的不同实例化。

统一的关键概念是门控矩阵 $\mathbf{G}_t$ 。把递推写成 $\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t$ ，三类模型的区别就只在 $\mathbf{G}_t$ 如何参数化：线性注意力不做遗忘 ($\mathbf{G}_t = \mathbf{1}$)，RetNet 用固定衰减 ($\mathbf{G}_t = \gamma \mathbf{1}$)，GLA 用数据依赖的 sigmoid，Mamba 用输入依赖的矩阵指数 $e^{\Delta_t A}$ 。从 Yang et al. (GLA) 和 Dao & Gu (Mamba-2/SSD) 从两条独立路线殊途同归地发现这一统一，这本身就说明了框架的自然性。

Mamba-2 的 SSD 对偶性更进一步，从代数角度精确描述了递推视角和矩阵视角的等价性：SSM 的输入-输出映射等价于一个「半分离矩阵」乘向量，这与结构化注意力精确对应。这一对偶性不仅是理论上的优美，也有工程意义：Mamba-2 利用它允许更大的状态维度并实现了 2–8 倍的速度提升。**统一框架的价值不在于帮你选择一个最好的模型，而在于指明了哪些维度（门控的参数化、状态维度、秩约束）是可以调节的旋钮。**

7.6.1 通用递推形式

Yang et al. (GLA, ICML 2024) 和 Dao & Gu (Mamba-2/SSD, ICML 2024) 分别从不同方向指出：Mamba、线性 Attention 及经典门控 RNN 可以统一到同一框架下。

定义 7.5 (广义线性递推)： 定义隐状态 $\mathbf{S}_t \in \mathbb{R}^{N \times d}$ 的递推

$$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t, \quad (7.28)$$

其中 $\mathbf{G}_t \in \mathbb{R}^{N \times d}$ 是门控矩阵， $\odot$ 表示 Hadamard 积， $\mathbf{k}_t, \mathbf{q}_t \in \mathbb{R}^N, \mathbf{v}_t \in \mathbb{R}^d$ 。

三类模型在此框架中的实例化：

模型	门控 $\mathbf{G}_t$ 的参数化
线性注意力 (Katharopoulos)	$\mathbf{G}_t = \mathbf{1}$ (恒等, 无遗忘)
RetNet (Sun et al., 2023)	$\mathbf{G}_t = \gamma \cdot \mathbf{1}$ (固定标量衰减)
GLA (Yang et al., 2024)	$\mathbf{G}_t = \sigma(W_g x_t)$ (数据依赖门)
Mamba (Gu & Dao, 2023)	$\mathbf{G}_t = e^{\Delta_t A}$ (输入依赖离散化, 结构化对角)

7.6.2 Mamba-2 的结构化状态空间对偶性

Dao & Gu (SSD, ICML 2024) 在更严格的数学框架下建立了 SSM 与结构化注意力之间的精确对应:

定理 7.6 (结构化状态空间对偶): 考虑标量结构的 SSM(A 为标量乘以单位矩阵), 对每个特征维度:

$$h_t = \alpha_t h_{t-1} + \beta_t x_t, \quad y_t = c_t h_t, \quad (7.29)$$

其中 $\alpha_t, \beta_t, c_t \in \mathbb{R}$ 为标量。则此模型的输入-输出映射与如下**半分离 (semiseparable)** 矩阵乘向量等价:

$$Y = MX, \quad M_{ij} = c_i \left(\prod_{t=j+1}^i \alpha_t \right) \beta_j \cdot \mathbf{1}_{i \geq j}, \quad (7.30)$$

矩阵 M 是 1 阶半分离矩阵。在多头设定下, 矩阵 M 升维为具有更丰富块结构的半分离矩阵族, 统一了 SSM 的递推视角和注意力的矩阵视角。

此对偶性揭示:SSM 可以被视为结构化、低秩化的注意力矩阵运算。Mamba-2 利用此对偶性设计了新的 SSD(Structured State Space Duality) 层, 允许更大的状态维度 (dN 而非 Mamba 的 dN), 相较 Mamba 实现了 2-8 倍的速度提升。

7.6.3 GLA 的数据依赖门控

GLA(Gated Linear Attention, Yang et al., ICML 2024) 在线性注意力框架中引入数据依赖的门控:

$$\mathbf{S}_t = (\mathbf{G}_t \odot \mathbf{S}_{t-1}) + \mathbf{k}_t^\top \mathbf{v}_t, \quad \mathbf{G}_t = \sigma(W_\alpha x_t) \in (0, 1)^{N \times d}, \quad (7.31)$$

其中 σ 是 sigmoid 函数。GLA 中对 $\mathbf{G}_t$ 施加了“低秩”结构约束: $\mathbf{G}_t = \alpha_t \beta_t^\top$ ($\alpha_t \in \mathbb{R}^N, \beta_t \in \mathbb{R}^d$), 减少参数量同时保持表达能力。GLA 可以证明等价于选择性 SSM: 门控 $\mathbf{G}_t$ 对应 Mamba 中的 $\bar{A}_t$, 而 $\mathbf{k}_t^\top \mathbf{v}_t$ 对应 $\bar{B}_t x_t \cdot C_t^\top$ 。这一等价性表明, 从线性注意力和 SSM 两条独立路径, 研究者们殊途同归地发现了同一类数据依赖线性递推。

§7.7 扩散语言模型的数学

这一节在讲什么

为什么要把扩散模型用于语言？ 图像扩散模型在连续像素空间加高斯噪声；语言是离散 token，不能直接加连续噪声。离散扩散的解决方案是用「掩码替换」代替「加噪声」，从而把扩散的数学框架搬到 token 空间。

第一步：连续 DDPM 复习——前向加噪 + 反向去噪。 前向过程把 x_0 逐步混入高斯噪声直至变成纯噪声；反向过程学习从噪声恢复 x_0 ；训练目标简化为「预测加入的噪声 ϵ 」，等价于分数匹配。

第二步：离散扩散用吸收态 (MASK) 替代高斯噪声。 前向过程以概率 β_t 将 token 替换为 [MASK]；边际分布有闭式：时刻 t 时 token 要么仍是原值（概率 $\bar{\alpha}_t$ ），要么已被 MASK（概率 $1 - \bar{\alpha}_t$ ）。

第三步：MDLM 把 ELBO 化简为加权 MLM 损失。 对被 MASK 的位置同时预测原始 token，权重系数 $|\alpha'_t|/\alpha_t$ 对不同噪声水平重要性加权——本质上是一个多尺度的遮蔽语言模型目标。

本节要拿走的一件事： 离散扩散语言模型的训练目标是「加权 MLM 损失」，生成时可以并行对所有被 MASK 的位置同时预测——这与自回归模型严格左到右的单步生成形成数学上的本质对立。

直觉

自回归语言模型以严格的左到右顺序生成文本：每一步预测一个 token，条件于所有已生成的前缀。这是一个非常特殊的生成方向——为什么不能同时生成所有位置，或者从中间开始生成？扩散语言模型提供了另一条路：从全部被 MASK 的序列出发，通过多步去噪逐步揭示真实 token，任意位置可以同时被预测。连续扩散模型 (DDPM) 在图像上取得了巨大成功，但语言是离散 token，不能直接加高斯噪声。离散扩散的关键创新是用「掩码替换」代替「加高斯噪声」，把扩散的前向-反向框架搬到 token 空间。

离散扩散的数学形式依然干净。前向过程通过吸收态转移矩阵 Q_t 以概率 β_t 把每个 token 变成 [MASK]；边际分布有闭式形式：时刻 t 的 token 要么还是原始值 x_0 （概率 $\bar{\alpha}_t$ ），要么已被 MASK（概率 $1 - \bar{\alpha}_t$ ）。后向条件分布也有闭式，这使得 ELBO 的计算变得高效。MDLM 进一步将 ELBO 化简为加权 MLM 损失——仅对被 MASK 的位置计算预测损失，权重由噪声调度决定。这个目标形式上与 BERT 的掩码语言模型非常相似，但有严格的变分推导支撑。

自回归与离散扩散之间的数学差异不仅是技术细节，而是根本性的。自回归模型因为训练时只看 $x_{<i}$ 而对「逆向查询」存在系统性弱点（如「谁发明了 X」→「X 是谁发明的」的不对称）。扩散模型以全序列为条件进行去噪，对所有位置对称建模，在逆向任务上有理论优势。代价是生成需要多步迭代（通常 64–256 步），每步都需要完整的前向传播，延迟高于自回归的每步单 token 生成。

7.7.1 连续扩散:DDPM 回顾

去噪扩散概率模型 (DDPM, Ho et al., NeurIPS 2020) 通过马尔可夫链定义数据的渐进加噪过程。给定数据 $x_0 \sim q_{\text{data}}$ ，前向过程为：

$$q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \quad t = 1, \dots, T, \quad (7.32)$$

其中 $\{\beta_t\}_{t=1}^T$ 是方差调度 (variance schedule)。令 $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$, 则边际分布有闭式形式:

$$q(x_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I). \quad (7.33)$$

反向去噪: 学习参数化的反向过程 $p_\theta(x_{t-1}|x_t)$ 。通过变分下界 (ELBO) 与去噪分数匹配的联系, 最优化目标简化为

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t, x_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2 \right], \quad (7.34)$$

其中 $\epsilon \sim \mathcal{N}(0, I)$ 。这等价于在每个噪声水平 t 上的分数匹配 (score matching):

$$\nabla_{x_t} \log q_t(x_t) \approx -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}. \quad (7.35)$$

分数函数的直觉: 在样本 x_t 处, 分数 $\nabla_{x_t} \log q_t(x_t)$ 指向高概率密度方向。去噪网络 ϵ_θ 估计添加的噪声, 等价于估计分数函数的负值, 从而引导采样轨迹朝数据流形靠拢。

7.7.2 离散扩散: 吸收态过渡矩阵

连续扩散对连续向量加高斯噪声, 但语言模型处理的是离散 token。**离散扩散 (Discrete Diffusion)** 通过马尔可夫链在 token 空间 $\{1, \dots, V\}^L$ 上定义噪声过程。

吸收态 (Absorbing State) 扩散: 引入特殊 [MASK] token(索引 M), 定义前向转移矩阵:

$$Q_t = (1 - \beta_t)I + \beta_t \mathbf{1} e_M^\top, \quad (7.36)$$

其中 e_M 是 [MASK] 的 one-hot 向量。即以概率 β_t 将当前 token 替换为 MASK, 以概率 $1 - \beta_t$ 保持不变。边际分布为

$$q(x_t|x_0) = \bar{\alpha}_t \delta_{x_t, x_0} + (1 - \bar{\alpha}_t) \delta_{x_t, M}. \quad (7.37)$$

其中 $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ 。在时刻 t , token 要么仍是原始 token x_0 (概率 $\bar{\alpha}_t$), 要么已被 MASK (概率 $1 - \bar{\alpha}_t$)。

后向条件分布由贝叶斯公式给出:

$$q(x_{t-1}|x_t, x_0) = \begin{cases} \delta_{x_{t-1}, x_0} & \text{若 } x_t \neq M, \\ \frac{(1 - \bar{\alpha}_{t-1}) \delta_{x_{t-1}, M} + (\bar{\alpha}_{t-1} - \bar{\alpha}_t) \delta_{x_{t-1}, x_0}}{1 - \bar{\alpha}_t} & \text{若 } x_t = M. \end{cases} \quad (7.38)$$

此闭式形式使得训练中的 ELBO 计算非常高效。

7.7.3 MDLM 的简化目标

MDLM (Masked Diffusion Language Model, Sahoo et al., NeurIPS 2024) 推导了吸收态离散扩散的变分下界的 **Rao-Blackwellized 简化形式**。

由于吸收态的特殊结构, ELBO 化简为:

$$\mathcal{L}_{\text{MDLM}} = -\mathbb{E}_{t, x_0} \left[\frac{|\alpha'_t|}{\alpha_t} \sum_{i: x_t^{(i)} = M} \log p_\theta(x_0^{(i)} | x_t) \right], \quad (7.39)$$

其中 $\alpha'_t = d\alpha_t/dt$, 求和仅对被 MASK 的位置 i 进行。此目标本质上是加权的 MLM 损失的混合, 权重系数 $|\alpha'_t|/\alpha_t$ 对不同噪声水平重要性加权。

LLaDA(2025) 将此框架扩展至 8B 参数规模, 证明了离散 Mask 扩散与自回归语言模型在同等工程实践下具有可比性能。

7.7.4 与自回归模型的本质差异

自回归 (AR) 语言模型基于因果分解 $p(x) = \prod_i p(x_i | x_{<i})$, 生成时严格左到右顺序进行, 时间复杂度为 $O(L)$ 步。

离散扩散模型则有以下数学差异:

1. **并行解码**: 在任意一步去噪 $p_\theta(x_0 | x_t)$ 中, 所有被 MASK 的位置**同时**预测, 不存在左到右的顺序依赖。理论上, 仅需 1 步去噪即可并行生成整个序列, 但质量较差; 实际使用 $T = 64 \sim 256$ 步迭代精化。
2. **迭代精化**: 每步将部分 MASK token 替换为预测值, 逐步精化全序列。理论上, 给定足够的步数 T , 采样质量随 T 单调提升, 等价于更精确地模拟真实反向马尔可夫链。
3. **消除逆向诅咒 (Reversal Curse)**: AR 模型因训练时只看 $x_{<i}$ 而对逆向查询存在系统性弱点。扩散模型以全序列为条件进行去噪, 对所有位置对称建模, 在逆向任务上有理论优势, 这在 LLaDA 实验中已得到验证。

§7.8 世界模型与预测编码的数学

这一节在讲什么

为什么「世界模型」需要数学化? 仅凭「让智能体预测未来」的目标太模糊, 不同的形式化选择——预测哪个空间、用什么先验、如何防止 collapse——导致截然不同的模型行为和优化难度。本节拆解四种支撑性数学框架。

第一步: 层级预测编码把视觉皮层建模为误差传递网络。 每层对下层做「自顶向下预测」, 预测误差逐层上传, 权重更新仅依赖相邻层信号——这在数学上等价于反向传播, 但完全局部。

第二步: 变分自由能把感知与行动统一为同一极小化目标。 最小化自由能 $F = D_{\text{KL}}(q \| p(z|x)) - \log p(x)$, 同时做感知推断 (让 q 逼近后验) 和「惊奇最小化」(最大化证据)。这与 VAE 的 ELBO 完全等价。

第三步: JEPA 在表示空间预测以规避像素级重建的高方差。 能量函数 $E(x, y) = \|s_\phi(y) - P_\psi(s_\phi(x), z)\|^2$, 配合 VICReg 三项正则化防止表示 collapse。

本节要拿走的一件事: 世界模型的数学骨架不是全新发明, 而是把变分推断、Kalman 滤波、对比学习按新的工程目标重新组装——Kalman 增益的误差修正规则与预测编码的局部更新在结构上完全对应。

直觉

「世界模型」这个词在强化学习和认知科学中有不同含义，但数学层面的核心问题是共同的：给定当前观测，如何在表示空间中对未来状态进行预测，并用预测误差来驱动学习？本节的形式化框架——预测编码、变分自由能、JEPA、Kalman 滤波——从不同角度回答这一问题，但相互之间有深刻的数学关联。预测编码的层级误差传递规则、Kalman 滤波的更新步、JEPA 的预测残差，在形式上高度对应：都是「预测值 - 实际值 = 误差信号，用它来更新表示」。

变分自由能原理提供了最宏观的统一：感知、行动、学习都是「最小化自由能 F 」这一目标的不同分支。这个框架与 VAE 的训练目标完全等价——VAE 的重构损失加 KL 正则项就是 F 的蒙特卡洛近似。活跃推断 (active inference) 更进一步把行动也纳入 F 的极小化：感知更新对应 E 步，行动更新对应本体感觉通道的预测误差抑制。这个框架理论上优美，但计算上仍有大量未解决的扩展问题（如高维连续动作空间）。

JEPA 的洞察则针对世界模型的一个具体痛点：在像素空间预测未来帧需要模型精确重建视觉细节，方差极高、收敛困难。把预测目标移到表示空间，模型只需预测「语义等价的表示」而非「像素级精确重建」，大幅降低了学习难度。但表示空间预测面临 collapse 问题——所有输入映射到同一点就能让误差为零。VICReg 的三项正则化（不变性、方差、协方差）从信息论角度强制表示空间保持「充盈性」，这与白化、去冗余原理一脉相承。

世界模型的工程目标是让智能体在表示空间中对未来状态进行预测，并以此驱动规划与控制。本节从数学角度拆解支撑这一目标的四类形式化：层级预测编码、变分自由能、JEPA 能量函数与防 collapse 正则化、以及 Kalman 滤波作为线性高斯 SSM 是让智能体在表示空间中对未来状态进行预测，并以此驱动规划与控制。本节从数学角度拆解支撑这一目标的四类形式化：层级预测编码、变分自由能、JEPA 能量函数与防 collapse 正则化、以及 Kalman 滤波作为线性高斯 SSM 的精确推断。它们共同构成当代世界模型的数学骨架。

7.8.1 预测编码：层级生成模型与误差传递

Rao & Ballard (1999) 提出的层级预测编码框架将视觉皮层建模为一个层级生成系统。设共有 $L + 1$ 层，第 l 层持有表示 x_l ，第 l 层对第 $l - 1$ 层的自顶向下预测为

$$\hat{x}_{l-1} = f_l(x_l),$$

其中 f_l 是可学习的生成映射。预测误差 (prediction error) 定义为

$$\varepsilon_{l-1} = x_{l-1} - \hat{x}_{l-1}.$$

联合生成模型写作

$$p(x_0, x_1, \dots, x_L) = p(x_L) \prod_{l=0}^{L-1} p(x_l | x_{l+1}),$$

其中每个条件因子在高斯假设下取 $p(x_l | x_{l+1}) = \mathcal{N}(x_l; f_{l+1}(x_{l+1}), \Pi_l^{-1})$ ，精度矩阵 Π_l 刻画该层预测的置信度。

在此高斯假设下，最大化对数似然等价于最小化精度加权误差平方和。对表示 x_l 求负对数似然的梯度，得到表示更新规则：

$$\Delta x_l \propto \Pi_l \varepsilon_l - \frac{\partial f_l(x_l)}{\partial x_l} \Pi_{l-1} \varepsilon_{l-1}.$$

右侧第一项是**来自上方的误差信号** (当前层的预测与实际值的差异), 第二项是**来自下方的误差信号** (当前层对下一层的预测残差通过雅可比 (Jacobian) 反传回来)。这一局部更新规则完全依赖相邻层之间的信号, 无需全局梯度通路。

Whittington & Bogacz (2017) 严格证明: 当网络满足”误差单元与表示单元分离”以及”固定预测权重”等条件时, 预测编码的权重更新与标准误差反向传播算法在数值上收敛到相同结果。这一等价性表明预测编码并非生物学的特殊曲解, 而是反向传播在局部学习规则约束下的一个有效近似。

7.8.2 自由能原理与变分下界

Friston (2010) 将感知与行动统一为一个极小化目标: **变分自由能** (variational free energy)。设观测为 x , 隐变量为 z , 变分后验为 $q(z)$, 变分自由能定义为

$$F[q, x] = \mathbb{E}_{q(z)}[\log q(z) - \log p(x, z)].$$

将联合概率分解为 $\log p(x, z) = \log p(x | z) + \log p(z)$ 并重组, 得到与证据下界 (ELBO) 的关系:

$$F[q, x] = D_{\text{KL}}(q(z) \| p(z | x)) - \log p(x) = -\text{ELBO}(q, x).$$

由于 $\log p(x)$ 对 q 不可优化, 最小化 F 同时实现两个目标: 其一, 令 $q(z)$ 靠近真实后验 $p(z | x)$ (感知推断); 其二, 间接最大化边际似然 $\log p(x)$ (避免惊奇, 即 surprise minimization)。与变分自编码器 (VAE) 的训练目标完全等价——VAE 的重构损失加 KL 正则项正是 F 的蒙特卡洛近似。

主动推断 (active inference) 在此框架中把行动 a 纳入自由能极小化: 感知更新通过 $\partial F / \partial z = 0$ 调整隐变量表示, 行动更新通过

$$\frac{\partial F}{\partial a} = \sum_l \frac{\partial F}{\partial \varepsilon_l} \cdot \frac{\partial \varepsilon_l}{\partial a}$$

调整运动输出, 使感觉预测误差在本体感觉通道中得到抑制。感知与行动因此成为同一极小化问题的两个分支, 分别对应变分 E 步与 M 步的连续时间版本。

7.8.3 JEPA 的能量函数与防 collapse 机制

LeCun (2022) 提出**联合嵌入预测架构** (Joint-Embedding Predictive Architecture, JEPA), 其核心思想是在**表示空间**而非像素空间中执行预测, 从而规避生成模型在像素级重建上的高方差问题。设 x 为上下文观测, y 为目标观测, s_ϕ 为编码器, P_ψ 为预测器, z 为描述 x 与 y 差异的隐变量, 能量函数定义为

$$E(x, y) = \|s_\phi(y) - P_\psi(s_\phi(x), z)\|^2.$$

最小化 E 要求预测器能够从上下文表示与差异变量 z 中复原目标表示。V-JEPA (Bardes et al., 2024) 将此框架应用于视频时序预测, 在屏蔽的时空块上执行表示层预测。

Collapse 问题: 若编码器退化为常数映射 $s_\phi(\cdot) \equiv \mathbf{c}$, 则 $E \equiv 0$ 对任意 x, y 成立, 训练目标被平凡解满足。VICReg (Bardes et al., ICLR 2022) 通过三项显式正则化阻止 collapse:

- **不变性项 (Invariance):** $I(Z, Z') = \frac{1}{n} \sum_{i=1}^n \|Z_i - Z'_i\|^2$, 最小化两视图嵌入之差;
- **方差项 (Variance):** $V(Z) = \frac{1}{d} \sum_{j=1}^d \max\left(0, \gamma - \sqrt{\text{Var}(Z_{:,j})} + \epsilon\right)$, 惩罚任意维度标准差低于阈值 γ ;
- **协方差项 (Covariance):** $C(Z) = \frac{1}{d} \sum_{i \neq j} [\text{Cov}(Z)]_{ij}^2$, 惩罚不同维度之间的线性相关性。

VICReg 目标函数为

$$L_{\text{VICReg}} = \lambda I(Z, Z') + \mu V(Z) + \nu C(Z),$$

其中 λ, μ, ν 为超参数。方差项直接对抗维度坍缩, 协方差项鼓励嵌入维度间信息不冗余, 二者共同维持表示空间的”充盈性”(informativeness)。从信息论角度看, $C(Z)$ 的极小化是对表示进行近似白化, 与去冗余原理 (redundancy reduction) 一脉相承。

7.8.4 Kalman 滤波作为线性高斯 SSM 的精确推断, 以及神经过程视角

§7.1 已将状态空间模型 (SSM) 写为 $x_t = Ax_{t-1} + w_t, y_t = Cx_t + v_t$, 其中 $w_t \sim \mathcal{N}(0, Q), v_t \sim \mathcal{N}(0, R)$ 。当 A, C, Q, R 全部已知且线性高斯时, 后验 $p(x_t | y_{1:t})$ 保持高斯, 可用 Kalman (1960) 滤波器在 $\mathcal{O}(n^3)$ 时间内精确递推。

预测步 (predict):

$$\hat{x}_{t|t-1} = A \hat{x}_{t-1|t-1}, \quad P_{t|t-1} = A P_{t-1|t-1} A^\top + Q.$$

更新步 (update):

$$K_t = P_{t|t-1} C^\top (C P_{t|t-1} C^\top + R)^{-1},$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t (y_t - C \hat{x}_{t|t-1}), \quad P_{t|t} = (I - K_t C) P_{t|t-1}.$$

Kalman 增益 K_t 是协方差矩阵的函数: 当预测不确定性 $P_{t|t-1}$ 大时, 新观测权重高; 当观测噪声 R 大时, 权重低。更新步在结构上与预测编码的误差修正规则 $\Delta x \propto K(y - C\hat{x})$ 完全对应—— $y_t - C\hat{x}_{t|t-1}$ 正是观测层的预测误差。

Mamba (§7.2) 可视为 Kalman 滤波的**数据依赖非线性推广**: 用输入门控的选择矩阵 $\bar{A}(x_t), \bar{B}(x_t)$ 替代固定的 A, B , 使得噪声协方差的角色由网络动态承担, 但预测-更新的递推结构保持不变。

神经过程 (Neural Processes) Garnelo et al., ICML 2018 将贝叶斯滤波的后验近似交给神经网络, 以**摊销推断** (amortized inference) 替代逐步递推: 一个编码器将上下文集合 $\{(x_i, y_i)\}_{i=1}^n$ 映射为全局潜在表示 z , 解码器再条件于 z 生成目标预测。这一机制在计算上等价于把 Kalman 增益的迭代计算”压缩”进一次前向传播。

DreamerV3 Hafner et al. (2023) 的循环状态空间模型 (RSSM) 将上述路线推至强化学习世界模型: 隐状态分为**确定性分量** h_t (由 GRU 递推维护, 对应 $\hat{x}_{t|t-1}$) 和**随机分量** $z_t \sim q_\phi(z_t | h_t, o_t)$ (对应后验更新)。变分目标最小化

$$\mathcal{L}_{\text{RSSM}} = \mathbb{E}_q \left[\sum_t -\log p_\theta(o_t | h_t, z_t) + \beta D_{\text{KL}}(q_\phi(z_t | h_t, o_t) \| p_\theta(z_t | h_t)) \right],$$

其中先验 $p_\theta(z_t | h_t)$ 由确定性递推给出, 与 §7.8.2 的自由能框架直接对应。智能体在学到的隐空间中展开纯想象 (imagination) 轨迹以进行策略优化, 无需与真实环境交互。

以上四类形式化共同指向一个判断: 世界模型的数学骨架并非全新发明, 而是把变分推断、Kalman 滤波、对比学习这些已有工具按新的工程目标重新组装。下一节给出每种形式化的边界与失效场景。

§7.9 反类比与边界

这一节在讲什么

为什么要专门讲「边界」? 前几节推导了 HiPPO、S4、Mamba、线性注意力各自的数学保证, 但每个保证都在某个假设成立时才有效。本节系统检查: 当假设失效时, 模型在哪里会垮掉, 失败是随机的还是系统性的?

第一步: HiPPO 的「最优性」是测度相对的。 LegS 在全历史均匀测度下最优, LegT 在滑动窗口下最优——拿错测度用对方的矩阵就不是最优。实践中 S4 训练后 A 会偏离 LegS 结构, 说明任务最优性和理论最优性有差距。

第二步: Mamba 并行扫描有数值稳定性隐患。 长前缀积中谱范数大于 1 则指数增长, 小于 1 则指数衰减; 与 Softmax 注意力的内建归一化不同, 状态扫描没有类似机制, 极长序列下状态数值范围可能漂移。

第三步: 线性注意力在精确检索任务上有数学上界 $O(r/L)$ 。 有限秩的特征映射无法模拟 Softmax 的「赢者通吃」尖锐化, 当序列长度 L 增大时, 正确键分配到的权重期望不超过 $O(r/L)$, 这是系统性不足, 不是调参能解决的。

本节要拿走的一件事: 每个数学保证背后都有一个假设; 把假设和结论分开记, 遇到新任务时先检查假设是否成立, 再判断方法是否适用。

直觉

本章前面的推导为每类模型建立了数学保证: HiPPO 给出最优压缩, S4 实现高效卷积, Mamba 实现选择性记忆, 线性注意力将推理压缩到 $O(1)$ 。但每个保证都有它的隐含假设, 而现实中假设常常不成立。这一节的价值在于把「保证」和「前提」分开: 知道一个方法在什么时候会系统地失败, 和知道它在什么时候会成功, 同等重要。

HiPPO 的最优性依赖于测度选择。LegS 在全历史均匀测度下是最优的, 但如果任务真正重要的是近期历史 (比如对话的最近几轮), LegS 给所有时刻均等权重可能并不理想。实践中 S4 训练后 A 矩阵会偏离理论的 LegS 结构, 这说明「训练后的最优」和「数学推导的理论最优」之间存在差距。后续工作 (S4D、How to Train Your HiPPO) 探索了更灵活参数化, 放弃了严格的 HiPPO 最优性但

在实践中表现更好——这是一个典型的「理论严格但实践灵活」的权衡。

线性注意力的检索上界 $O(r/L)$ 是最直接的例子。当序列中有 L 个随机键时，分配给正确键的权重期望不超过 $O(r/L)$ ，随序列长度线性衰减。这不是实现缺陷，而是有限秩特征映射的数学必然： r 个维度最多能「精确区分」约 r 个键，多出来的键之间会互相干扰。这从机制上解释了为什么 GLA、Mamba 等模型在 RULER 等精确检索基准上稳定落后于 Transformer——这是一个需要理论改变才能解决的问题，不是工程调优的范畴。

7.9.1 HiPPO”最优性”的测度依赖

§7.2 的最优性结论具有根本的相对性：

命题 7.7: HiPPO-LegT 和 HiPPO-LegS 给出的 A 矩阵各自在其特定测度下是最优的——LegT 在长度为 θ 的滑动窗口均匀测度下最优，LegS 在整个历史 $[0, t]$ 的均匀测度下最优。在另一种测度下使用对方的矩阵将不是最优近似。

例如，若使用指数衰减测度 $\mu^{(t)}(ds) = e^{-(t-s)}ds$ ，则最优矩阵 A_{LagT} 是 Laguerre 多项式对应的矩阵，与 LegT、LegS 均不同。实践中，S4 使用 LegS 初始化（因为它对全局历史最优），但在学习过程中 A 矩阵会偏离 LegS 结构，表明任务-特定最优性与理论最优性存在差距。

推论: HiPPO 框架的成功部分来自”良好的初始化”，而非全局最优。后续工作 (How to Train your HIPPO, 2023) 表明，S4D(对角 S4) 等变体虽然放弃了严格的 HiPPO 最优性，但通过更灵活参数化在实践中表现更好。此外，不同测度对应不同”时间分辨率”：LegT 在近期历史上有更高精度，LegS 对所有时刻均等对待。没有一种测度在所有任务上都最优。

7.9.2 Mamba 并行扫描的数值稳定性

选择性 SSM 的并行扫描在数值上面临以下挑战：

问题 1: 矩阵乘积的数值范围。 在式 (??) 的长前缀积中，若状态转移矩阵 $\bar{A}_k$ 的谱范数大于 1，则乘积指数增长（梯度爆炸）；若远小于 1，则乘积指数衰减（梯度消失）。Mamba 通过限制 A 为具有负实部特征值的对角矩阵（从而 $\|\bar{A}_t\| = \|e^{\Delta_t A}\| < 1$ ）来缓解此问题。

问题 2: 与 Softmax Attention 的对比。 Softmax 注意力的归一化操作保证注意力权重之和为 1，这是一种内建的数值归一化机制。对数-求和-指数 (log-sum-exp) 技巧 (FlashAttention) 进一步在数值稳定的对数域中计算，避免上溢。状态扫描没有类似的归一化，状态 h_t 的数值范围可能随序列长度增长而漂移。Mamba 的对数域扫描技巧是部分解决方案，但未被所有实现采用。

问题 3: 长序列下的梯度传播。 在极长序列 ($L > 10^5$) 上，即使使用并行扫描，反向传播时 $O(\log L)$ 层二叉树的激活值存储仍需 $O(L)$ 显存，可能成为瓶颈。FlashAttention 通过重计算前向激活避免存储 $O(L^2)$ 注意力矩阵，SSM 的类似技巧 (FlashSSM) 也在被探索。

7.9.3 线性注意力在检索任务上的系统性劣势

实验规律: 在需要精确键值检索的任务（如 Passkey Retrieval、Copy、归纳头 (Induction Heads)）上，线性注意力一致地输给 softmax 注意力，差距在长序列下尤为显著。

理论解释:

设查询集中有某个精确匹配的查询 Q^* ，对应键 $K^* = Q^*$ 。在 softmax 注意力中：

$$A_{i^*,j^*} = \frac{e^{\|Q^*\|^2/\sqrt{d}}}{\sum_j e^{Q^* \cdot K_j/\sqrt{d}}} \rightarrow 1 \quad \text{当 } \|Q^*\|^2/\sqrt{d} \rightarrow \infty,$$

通过增大 $\|Q^*\|$ (由学习控制) 可以任意接近 1, 实现”赢者通吃”的精确检索。

而在线性注意力中, 分配给 j^* 的权重为:

$$w_{j^*} = \frac{\phi(Q^*) \cdot \phi(K^*)}{\sum_j \phi(Q^*) \cdot \phi(K_j)} = \frac{\phi(Q^*) \cdot \phi(K^*)}{\phi(Q^*) \cdot \mathbf{z}}.$$

命题 7.8 (线性注意力的检索上界): 设特征维度为 r , 序列中有 L 个键均匀随机分布。则在最坏情况下, 线性注意力分配给正确键的权重期望不超过 $O(r/L)$ 。

证明思路: 由柯西-施瓦茨不等式, $|\phi(Q^*) \cdot \phi(K)| \leq \|\phi(Q^*)\| \|\phi(K)\|$ 。当 L 个键均匀分布时, $\|\mathbf{z}\| = \|\sum_j \phi(K_j)\| \geq \sqrt{L} \|\phi(K^*)\|$ (大数定律), 故 $w_{j^*} \leq \|\phi(K^*)\| / (\sqrt{L} \|\phi(K^*)\|) = 1/\sqrt{L}$ 。更精细的分析给出 $O(r/L)$ 的界。□

这从数学上解释了线性注意力在检索任务上的系统性不足: 有限秩的特征映射无法模拟 softmax 的无界尖锐化能力。实践中, 这一限制导致 GLA、Mamba 等模型在 RULER 等需要长距离精确检索的基准上显著落后于 Transformer。

§7.10 参考文献

这一节在讲什么

为什么列这些文献? 本章涵盖的数学内容跨越控制理论 (SSM、Kalman)、逼近论 (HiPPO、Legendre)、深度学习 (S4、Mamba、线性注意力)、概率生成模型 (DDPM、离散扩散) 和世界模型 (JEPA、DreamerV3) 五条研究线索。所列文献是每条线索的原始来源或最具代表性的综述, 适合在理解本章正文后深入追溯推导细节和实验验证。

1. Gu, A., Dao, T., Ermon, S., Rudra, A., & Ré, C. (2020). HiPPO: Recurrent Memory with Optimal Polynomial Projections. *Advances in Neural Information Processing Systems (NeurIPS)*, 33. <https://proceedings.neurips.cc/paper/2020/hash/102f0bb6efb3a6128a3c750dd16729be-Abstract.html>
2. Gu, A., Goel, K., & Ré, C. (2022). Efficiently Modeling Long Sequences with Structured State Spaces (S4). *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2111.00396>
3. Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv preprint arXiv:2312.00752*. <https://arxiv.org/abs/2312.00752>
4. Dao, T., & Gu, A. (2024). Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality (Mamba-2/SSD). *International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2405.21060>
5. Katharopoulos, A., Vyas, A., Pappas, N., & Fleuret, F. (2020). Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. *International Conference on Machine Learning (ICML)*, PMLR 119. <https://proceedings.mlr.press/v119/katharopoulos20a.html>

6. Choromanski, K., Likhoshesterov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., ... & Weller, A. (2021). Rethinking Attention with Performers (FAVOR+). *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2009.14794>
7. Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., ... & Wei, F. (2023). Retentive Network: A Successor to Transformer for Large Language Models. *arXiv preprint arXiv:2307.08621*. <https://www.microsoft.com/en-us/research/publication/retentive-network-a-successor-to-transformer-for-large-language-models/>
8. Yang, S., Wang, B., Shen, Y., Panda, R., & Kim, Y. (2024). Gated Linear Attention Transformers with Hardware-Efficient Training (GLA). *International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2312.06635>
9. Sahoo, S. S., Arriola, M., Schiff, Y., Gokaslan, A., Marroquin, E., Chiu, J. T., Rush, A., & Kuleshov, V. (2024). Simple and Effective Masked Diffusion Language Models (MDLM). *Advances in Neural Information Processing Systems (NeurIPS)*, 38. <https://neurips.cc/virtual/2024/poster/95622>
10. Nie, S., Zhu, F., Du, C., Zhang, T., Ou, Z., Muennighoff, N., ... & Li, C. (2025). Large Language Diffusion Models (LLaDA). *arXiv preprint arXiv:2502.09992*. <https://arxiv.org/abs/2502.09992>
11. Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models (DDPM). *Advances in Neural Information Processing Systems (NeurIPS)*, 33. <https://arxiv.org/abs/2006.11239>
12. Blelloch, G. E. (1990). Prefix Sums and Their Applications. *Technical Report CMU-CS-90-190*, Carnegie Mellon University. <https://www.cs.cmu.edu/~guyb/papers/Ble93.pdf>
13. Schlag, I., Irie, K., & Schmidhuber, J. (2021). Linear Transformers Are Secretly Fast Weight Programmers. *International Conference on Machine Learning (ICML)*, PMLR 139. <https://arxiv.org/abs/2102.11174>
14. Gu, A., Johnson, I., Timalina, A., Rudra, A., & Ré, C. (2023). How to Train Your HiPPO: State Space Models with Generalized Orthogonal Basis Projections. *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=klK17OQ3KB>
15. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30. <https://arxiv.org/abs/1706.03762>
16. Gu, A., Goel, K., Gupta, A., & Ré, C. (2022). On the Parameterization and Initialization of Diagonal State Space Models (S4D). *Advances in Neural Information Processing Systems (NeurIPS)*, 35. <https://arxiv.org/abs/2206.11893>
17. Austin, J., Johnson, D. D., Ho, J., Tarlow, D., & van den Berg, R. (2021). Structured Denoising Diffusion Models in Discrete State-Spaces (D3PM). *Advances in Neural Information Processing Systems (NeurIPS)*, 34. <https://arxiv.org/abs/2107.03006>
18. Voelker, A., Kajić, I., & Eliasmith, C. (2019). Legendre Memory Units: Continuous-Time Representation in Recurrent Neural Networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 32. <https://proceedings.neurips.cc/paper/2019/hash/952285b9b7e7a1be5aa7849f32ffff05-Abstract.html>

本章完。数学卷至此与主本第 1-7 章逐章镜像，公式与推导补完。

后记

本附册的写作目标是让读者「在主本里看到几何，在附册里看到公式」。如果你读完主本之后愿意翻开这里，并在某一节停下来推一遍——那这本书就达成了它最大的野心。

数学之所以重要，不是因为它能预测一切，而是因为它能让我们看清「哪里是确定的、哪里是猜测的、哪里是隐喻」。第 5 章的渗流相变是隐喻；第 1 章的交叉熵是确定的；第 4 章的 P 是数学定理但前提需要满足；第 6 章的 I/O 下界是经过 1981 年 Hong-Kung 证明的渐近铁律；第 7 章的 HiPPO 是在「平移 Legendre 测度」这一特定选择下最优——换个测度，矩阵就换。把这些边界讲清楚，是这本附册的本职工作。

如果你在阅读中发现任何公式错误、推导疏漏、或者认为某节应该更深入，欢迎寄信至 fqsx@mail.us tc.edu.cn 指正。

——大队长，2026 年 6 月